> **NOTE**
>
> Join the Astra Discord server!
> There is now a dedicated **Discord server** for Astra Framework and its extensions. Join to **receive notifications** about new extension releases and important updates, **ask for new features**, **report bugs**, **share ideas**, and **showcase your Astra creations** with other developers.
>
> **Join the Discord Server: https://discord.gg/nJVRMkGrZg**

Introduction

Astra Health extends the base framework, [](Astra Framework), by adding functionality for managing health and calculating damage for entities. The package is designed with the same design philosophy as the base asset: a Scriptable Object-based architecture to encourage flexibility, modularity, and testability. If you're already familiar with the base package, you'll feel right at home with its features.

You can define your own damage types, designate defensive statistics used to reduce each damage type, configure the damage calculation pipeline with the desired steps, configure both generic and damage-type-specific lifesteal, define strategies to execute upon entity death, and much more.

Astra Health Vocabulary

Damage Type

Damage types represent the different categories of damage that can be inflicted on entities. Common examples include "Physical", "Magical", "Bleeding", "Drowning", etc. You can create custom damage types to suit your game's needs.

Damage Source

Damage sources represent the origin of the damage inflicted. This is highly specific to your game's context. For example, one game might have damage sources such as "Entity", "Environment", "Potion", etc., while another might want to define more specific damage sources like "Attack", "Spell", "Equipment", "Trap", "Environment", "Damage Over Time", etc. The main difference between damage source and damage type is that damage types categorize damage based on its nature, while damage sources identify where the damage originates. The distinction can be subtle, but it's important for game logic and mechanics. We'll return to these concepts later, where we'll see the practical differences between the two.

Heal Source

Heal sources represent the origin of healing. Similar to damage sources, heal sources are specific to your game's context. Common examples include "Potion", "Spell", "Lifesteal", "Ability", "Environment", etc. In

some games, a classic Heal Source definition might include "Self" and "Ally", as these enable mechanics for increasing healing provided/received based on the caster and target.

Barrier

The concept of temporary HP can take various names across different games, though the underlying mechanic remains the same: provide an amount of extra and ephemeral hit points that are deducted instead of health when damage is taken. In Astra, these temporary hit points are called "Barrier".

Raw and Net Damage

Raw Damage refers to the damage that a certain attack or skill intends to inflict. This damage does not account for defense, critical hits, modifiers, etc. Net Damage is the result of processing Raw Damage while accounting for damage modifiers, damage mitigation, barriers, critical hits, etc.

Damage Mitigation and Defense Penetration

Primary damage mitigation occurs through defensive statistics, which reduce damage based on their value and the assigned damage mitigation formula. Each Damage Type can be configured with a specific defensive statistic that will be used to reduce damage of that type. For example, a `Physical Damage` Damage Type could be configured to be reduced by an `Armor` statistic.

Defense Penetration, instead, represents the ability to ignore part or all of the defensive statistic assigned to a Damage Type. Each Damage Type can be configured with a specific piercing statistic that will be used to ignore a portion of the defense when calculating damage. For example, the `Armor Penetration` statistic could be used to determine how much of the `Armor` to ignore when calculating the net damage of an attack that inflicts `Physical Damage`.

Damage Modifiers

Damage modifiers are a low-level abstraction that can alter net damage. They represent either flat or percentage-based modifications to the damage an entity receives. They can be general, applying to all damage received by the entity, or specific to a certain damage type or source.
They are the foundation for implementing a wide range of mechanics — such as elemental resistances and vulnerabilities, status effects, buffs, and debuffs — that affect the damage received based on varying conditions and contexts. Whether a modifier is temporary or permanent depends entirely on the higher-level logic built on top of them. Damage modifiers are generally utilized by the damage calculation pipeline (which we'll see shortly).

Damage Modifiers vs. Defensive Stat-Based Damage Mitigation

These are distinct concepts. Stat-based damage mitigation (via defensive stats) is the **baseline**: entities are always expected to carry a meaningful value for each defensive stat, and that value **always reduces** incoming damage of the associated type. Damage Modifiers, on the other hand, are backed by stats that

default to zero — meaning they have no effect unless a higher-level system (e.g. a resistance, a weakness, a status effect) changes their value. Additionally, while defensive stats can only ever reduce damage, damage modifiers can go in either direction: **reducing or amplifying** the damage received. Finally, defensive stats often also carry an explicit semantic meaning in the game world. For example, an `Armor` stat clearly indicates a defensive capability that reduces physical damage, and an heavier equipped armor values grants a greater `Armor` value.

Damage modifiers are a more abstract, low-level tool that can be used to implement many different mechanics; they do not carry an inherent, universal meaning. They act as a shared container for damage modifiers coming from various game mechanics. For example, a `Stone Skin` buff, a `Iron Elixir` potion, and a `Barbarian Temper` passive ability, could all rely on the same `Flat Physical Damage Modifier` stat to reduce physical damage. In this case, the `Flat Physical Damage Modifier` stat would be a shared resource that all these mechanics modify to achieve their effect on damage mitigation.

The **origin of val values** also differs between the two. Defensive stat values are typically set either as a fixed base value on the entity, or derived from its Class's `GrowthFormula` if classes are used — meaning they scale naturally with the entity's progression. Damage modifier stats, by contrast, usually rely on **flat stat modifiers**: each game mechanic (potion, passive, buff, debuff, etc.) contributes its effect by adding or removing a stat modifier on the underlying stat, temporarily or permanently altering the total modifier applied to incoming damage.

	Defensive Stat Damage Mitigation	**Damage Modifiers**
Default effect	Always active — entities carry a (usually) non-zero defensive stat that continuously reduces incoming damage	Neutral by default — the underlying stat starts at zero and has no effect until explicitly changed
Direction	Can only **reduce** damage	Can **reduce or increase** damage
Scope	Specific to a **Damage Type** — each type can have at most one defensive stat assigned. No reduction exists for Damage Sources	Can target **all damage** (any type and source), a **specific Damage Type**, or a **specific Damage Source**
Nature	Derived from a defensive stat value, processed through a damage mitigation formula	Flat or percentage-based modifications driven by higher-level systems (resistances, weaknesses, status effects, etc.)
Duration	Permanent — tied to the entity's stat value	Temporary or permanent, depending on the higher-level systems that apply them

	Defensive Stat Damage Mitigation	**Damage Modifiers**
Value origin	Set as a fixed base value on the entity, or derived from its Class's `GrowthFormula`	Driven by **flat stat modifiers** added or removed by individual game mechanics (potions, passives, buffs, debuffs, etc.)

How is Astra Health organized and how does it work?

 Astra Health Config

The `AstraHealthConfigSO` is a `ScriptableObject` that serves as the central configuration point for the Astra Health package. It has several properties that allow you to define how the health and damage systems should behave in your game.

Astra Health needs to be configured to work around the actual instances of the base framework's components defined for your game. For example, if you defined a certain statistic for general damage mitigation in your game with Astra Framework, you need to inform Astra Health about it so that it can use it when calculating damage. The needed configuration is kept minimal, and convention-over-configuration is applied where possible to reduce the amount of setup required.
Configuration will be deeply discussed later in Package Configuration.

 EntityHealth

`EntityHealth` is a brand new `MonoBehaviour` that you can add to your entities to provide them with health management capabilities. Worth mentioning, it exposes the most important method of the package: `TakeDamage()`, which allows you to inflict damage on the entity.

 Damage Type

`DamageType` is a `ScriptableObject` that represents a specific type of damage. Each damage type can be optionally configured with a defensive `Stat` that will be used to reduce incoming damage of that type. If a defensive stat is assigned, also a piercing stat can be assigned to ignore a portion of the defense when calculating damage.
Both for the defensive and piercing stat, you can select a `DamageMitigationFormula` and a `DefensePenetrationFormula` respectively, to define how the stats will affect damage mitigation and defense penetration. Each `DamageType` also contains its own optional lifesteal configuration, allowing specific damage types to add a dedicated lifesteal contribution on top of any generic lifesteal configured globally.

 Damage Source

`DamageSource`, derived from `ScriptableObject`, represents the origin of the damage inflicted. They don't have any specific properties, but they can be assigned to other objects of the package to create specific

behaviors based on the damage source. We will see for what and how in the Workflows.

 Damage Mitigation Functions

The package comes with three built-in `DamageMitigationFunction`s that you can use to define how defensive stats reduce incoming damage:

-  `FlatDamageMitigationFn`: Reduces damage by a flat amount based on the defense stat value.
-  `PercentageDamageMitigationFn`: Reduces damage by a percentage based on the defense stat value.
-  `LogarithmicDamageMitigationFn`: Reduces damage using a logarithmic scale based on the defense stat value.

In case of need, custom damage mitigation functions can be created by extending the `DamageMitigationFn` class.

 Defense Penetration Functions

As for damage mitigation functions, the package comes with three built-in `DefensePenetrationFunction`s that you can use to define how piercing stats ignore a portion of the defense stat:

-  `FlatDefensePenetrationFn`: Ignores a flat amount of the defense stat based on the piercing stat value.
-  `PercentageDefensePenetrationFn`: Ignores a percentage of the defense stat based on the piercing stat value.
-  `LogarithmicDefensePenetrationFn`: Ignores a portion of the defense stat using a logarithmic scale based on the piercing stat value.

Also in this case, custom defense penetration functions can be created by extending the `DefensePenetrationFn` class.

 Heal Source

`HealSource`, derived from `ScriptableObject`, represents the origin of healing. Similar to `DamageSource`, they don't have any specific properties, but they can be assigned to other objects of the package to create specific behaviors based on the heal source. We will see for what and how in the Workflows.

 Damage Calculation Strategy

The damage calculation pipeline is the component of the framework responsible for processing raw damage and producing net damage. The pipeline will use a given `DamageCalculationStrategy` to determine the sequence of steps to apply when calculating damage.
Therefore, a `DamageCalculationStrategy` is a `ScriptableObject` that defines a sequence of `DamageStep`s to be executed in order when calculating damage.

On-death & on-resurrection `GameAction`s

Astra Framework v1.4.0 introduced the concept of GameAction: a modular and reusable unit of game logic that can be assigned via the inspector to various components to define custom behaviors.

Astra Health leverages Game Actions to define custom behaviors when an entity dies or is resurrected. The framework allows you to assign a default on-death Game Action that will be executed when any entity dies, unless the entity has its own custom on-death Game Action assigned. Similarly, a default on-resurrection Game Action can be assigned to be executed when an entity is resurrected.

Lifesteal

Lifesteal is configured through two complementary layers. `AstraHealthConfigSO` can define a **Generic Lifesteal** that applies to all outgoing damage, while each `DamageTypeSO` can define its own **Lifesteal** configuration that applies only to that specific damage type.
Both layers use the same data structure — lifesteal stat, heal source, and amount selector — and can stack on the same hit. When both are active, you can choose whether they should remain two separate heals or be merged into a single heal through the `Unify Lifesteal Heals` flag in the global configuration. We will see this in detail later in the Workflows.

Health Scaling Component

Astra Health provides a brand new `HealthScalingComponent` that you can use in your `ScalingFormula`s to have skills or abilities scale based on either the attacker or the target's health. You can choose to scale upon one or more among Maximum HP, Current HP, and Missing HP.

Experience Collector

An `ExpCollector` is a `MonoBehaviour` that you can add to your entities to allow them to collect experience points from entities that die and have an `ExpSource` component attached. The `ExpCollector` can be configured with different strategies (`ExpCollectionStrategy`) to define under which conditions experience is collected from the dead entity.

Experience Collection Strategy

An `ExpCollectionStrategy` is a `ScriptableObject` that defines the logic for determining if an entity collects experience upon the death of another entity.

More Game Events

Astra Health comes with many new Game Events that you can use to react to health&damage-related events in your game. Some of the most important ones are:

- `PreDamageGameEvent`: Triggered before damage is applied to an entity. Useful for modifying or canceling damage. Use this for implementing custom passives or effects that need to react before damage is taken.
- `DamageResolutionGameEvent`: Triggered when an entity takes damage. Can be used to react to damage being applied.
- `EntityDiedGameEvent`: Triggered when an entity dies.
- `EntityHealedGameEvent`: Triggered when an entity is healed.
- `EntityResurrectedGameEvent`: Triggered when an entity is resurrected.

And many more events. We will discuss them in detail later in the Workflows.

Package Configuration

Astra RPG Framework needed no configuration. The health package, however, needs to be configured to have the system work around the specific instances of your game. For example, if you defined a "Super Duper All-damage resistance" `Stat` in your game, you need to tell Astra Health to use it when calculating damage.

The package uses a flexible configuration system that balances convenience with explicit control. This page explains how to set up and configure the health system for your game.

Configuration Overview

The Astra Health system uses a **two-tier configuration architecture**:

1. **Global Settings** (`AstraHealthGlobalSettings`) - A lightweight pointer stored in `Resources` that references your active configuration
2. **Gameplay Configuration** (`AstraHealthConfigSO`) - The actual configuration containing all gameplay parameters

This separation allows you to:

- Switch between different configuration profiles easily (e.g., for testing, different game modes)
- Keep configuration data separate from the loading mechanism
- Support convention-based fallbacks for quick prototyping

Global Settings

Automatic Setup

The package automatically creates the Global Settings asset on first import or when the editor loads. You don't need to do anything manually.

What happens automatically:

1. The `AstraHealthGlobalSettings.asset` is created in `Assets/Resources/`
2. If a default configuration exists (e.g., from imported samples), it's automatically assigned
3. The system is immediately ready to use

> **NOTE**
>
> Default location: `Assets/Resources/AstraHealthGlobalSettings.asset`

Project Settings

You can manage the package configuration through Unity's Project Settings window:

1. Open **Edit** → **Project Settings**
2. Navigate to **Astra Health**
3. Assign your desired **Active Config Profile**

Status Indicators:

- ✅ **Gray "Using Explicit Configuration"** - A configuration is explicitly assigned
- ⚠️ **Yellow "Using Fallback"** - No explicit configuration; using convention-based fallback
- 🛑 **Red "No Configuration Found"** - Critical: No configuration available

Quick Actions:

- **Create New Config Asset** - Opens a save dialog to create a new configuration

Manual Configuration (alternative to Project Settings)

You can also manage the package configuration manually by directly editing the `AstraHealthGlobalSettings` asset in the `Assets/Resources` folder. This approach allows you to assign a specific `AstraHealthConfigSO` asset without using the Project Settings window.

Convention Over Configuration

The package follows a **convention-over-configuration** philosophy to reduce setup friction:

Fallback Resolution

If no explicit configuration is assigned in Project Settings, the system automatically searches for a configuration named:

> `Astra Health Config`

located in any `Resources` folder in your project.

Search Order

The configuration provider uses a **three-step loading strategy**:

1. **Explicit Configuration** (Project Settings)
    - Loads `AstraHealthGlobalSettings` from `Resources/AstraHealthGlobalSettings`
    - If it has an `ActiveConfig` assigned, use it
2. **Convention-Based Fallback**

- Searches for `Astra Health Config` in any `Resources` folder
- Logs a warning indicating fallback usage

3. **Error State**
   - If neither is found, logs an error with instructions
   - System will not function until a configuration is provided

> **TIP**
> For production projects, always use **explicit configuration** via Project Settings for clarity and control.

Configuration Loading Strategy

The health configuration is loaded lazily on first access and cached for performance:

```
// Automatically loads configuration on first access
var config = AstraHealthConfigProvider.Instance;

// Pre-load during initialization to avoid runtime overhead
AstraHealthConfigProvider.WarmUp();

// Force reload (useful for testing)
AstraHealthConfigProvider.Reset();
```

When is the configuration loaded?

- Automatically before the first scene loads (via `RuntimeInitializeOnLoadMethod`)
- Lazily when first accessed via `AstraHealthConfigProvider.Instance`
- Explicitly when calling `WarmUp()`

Creating Configuration Assets

If for any reason you need to create a new `AstraHealthConfigSO` asset, you can do so via two methods:

Via Project Settings

1. Open **Edit** → **Project Settings** → **Astra Health**
2. Unassign any existing configuration, if any
3. Click **Create New Config Asset**
4. Choose a save location

5. The new configuration is automatically assigned

Via Asset Menu

1. Right-click in the **Project Window**
2. Select **Create** → **Astra Health** → **Config** → **Astra Health Config**
3. Name your configuration
4. Assign it in Project Settings or in the Global Settings asset

---

The red asterisks (*) indicate fields that are required for the system to function properly. Make sure to assign them before entering Play Mode to avoid runtime errors.

Health Configuration Reference

> **NOTE**
>
> In the `AstraHealthConfigSO` asset, you can hover over each field to see a tooltip with a brief description.

The `AstraHealthConfigSO` asset contains all gameplay parameters for the health system. Below is a detailed explanation of each field.

Health

Health Attributes Scaling

Type: `AttributesScalingComponent`
Required: No
Description: Defines how entities' maximum health scales based on character attributes (e.g., Vitality, Endurance).

See also: Astra RPG Framework Scaling documentation

Generic Flat Heal Amount Modifier Stat

Type: `Stat`
Required: No
Description: The stat that modifies **all** healing received by an entity as a flat amount.

Stacking Behavior:

- Combines **additively** with source-based flat amount modifications

How it works:

- The stat value represents a **flat amount modifier**
- Positive values increase healing, negative values decrease it
- Example: A value of `25` means +25 extra HP healing received

Example:

- Base Heal: 100 HP
- Generic Heal Amount Modifier: 25 (means +25 HP healing received)
- Final Heal: 100 + 25 = **125 HP**

Generic Percentage Heal Amount Modifier Stat

Type: `Stat` **Required:** No **Description:** The stat that modifies **all** healing received by an entity as a percentage. It is applied on top of the flat heal amount modifiers.

Stacking Behavior:

- Combines **additively** with source-based percentage modifications

How it works:

- The stat value represents a **percentage modifier**
- Positive values increase healing, negative values decrease it
- Example: A value of `20` means +20% extra healing received

Example:

- Base Heal: 100 HP
- Generic Flat Heal Amount Modifier: 20 (means +20 extra HP healing received)
- Generic Percentage Heal Amount Modifier: 20 (means +20% healing received)
- Final Heal: (100 + 20) × 1.20 = **144 HP**

> **WARNING**
> Remember to remove the default lower bound of 0 from your flat and percentage heal modifiers if you want to allow negative values for healing reduction effects. By default, the base framework sets a lower bound of 0 on all stats.

Damage

Default Damage Calculation Strategy

Type: `DamageCalculationStrategy`
Required: No
Description: The default strategy used to calculate net damage when an entity doesn't specify its own strategy.

The default strategy applied to entities that do not have a custom one assigned in their `EntityHealth` component. In most cases a single global default is sufficient; override it per-entity when a different calculation pipeline is required. See Damage Calculation Strategy for details on how strategies work.

Generic Flat Damage Modification Stat

Type: `Stat`
Required: No
Description: A universal flat damage modifier that applies to **all damage received**, regardless of type or source.

Applied by `ApplyFlatDmgModifiersStep` when that step is included in the active strategy.

Stacking Behavior:

- Combines **additively** with Type and Source **flat** modifications

Example:

- Incoming Damage: 150
- Generic Flat Damage Modifier: -20 (means -20 HP of damage taken)
- **Damage Mitigation:** -20 → Final Damage: **130**

Generic Percentage Damage Modification Stat

Type: `Stat` **Required:** No **Description:** A universal percentage damage modifier that applies to **all damage received**, regardless of type or source.

Applied by `ApplyPercentageDmgModifiersStep` when that step is included in the active strategy.

Stacking Behavior:

- Combines **additively** with Type and Source **percentage** modifications

Example:

- Incoming Damage: 150
- Generic Percentage Damage Modifier: -20 (means -20% damage taken)
- **Damage Mitigation:** -20% → Final Damage: 150 × 0.80 = **120**

> **NOTE**
>
> If the entity to be healed doesn't have the specified flat/percentage heal modification stats, they will be considered as having a value of 0 for those stats, and therefore no modification will be applied to the healing amount for that entity.

Rounding Settings

Type: `HealthRoundingSettings`
Required: No
Description: Controls how fractional intermediate results in health combat calculations are rounded to integer gameplay amounts. All five modes default to **Round** (midpoint-away-from-zero), preserving the behavior of versions that did not expose this setting.

> **NOTE**
>
> All rounding settings — including lifesteal — are configured here, under **Damage**, because they are part of the global combat configuration asset (`AstraRpgHealthConfigSO`).

`HealthRoundingSettings` exposes one `RoundingMode` field per mechanic:

Field	When it is applied
Percentage Damage Modifier Rounding Mode	After all percentage modifiers (generic, source-specific, type-specific) are combined and applied to the current damage amount in `ApplyPercentageDmgModifiersStep`.
Defense Penetration Rounding Mode	After the defense-penetration function reduces the target's effective defensive stat value inside `ApplyDefenseStep`.
Damage Mitigation Rounding Mode	After the damage-mitigation function converts the effective defense into a final reduced damage amount inside `ApplyDefenseStep`.
Critical Damage Multiplier Rounding Mode	After the critical-hit multiplier is applied to the current damage amount in `ApplyCriticalMultiplierStep`.
Lifesteal Rounding Mode	Applied independently to each lifesteal contribution (generic and damage-type) before amounts are summed or applied as heals.

`RoundingMode` has three values:

Value	Behaviour	Example
Round *(default)*	Nearest integer; midpoint rounds away from zero.	2.5 → 3, 2.4 → 2
Floor	Largest integer ≤ the value.	2.9 → 2, 2.1 → 2
Ceil	Smallest integer ≥ the value.	2.1 → 3, 2.9 → 3

> **TIP**
>
> **Floor** ensures defensive calculations never accidentally round up in the attacker's favour. **Ceil** guarantees that small offensive or heal amounts are never silently truncated to zero. **Round** (the default) provides neutral, balanced behaviour suitable for most games.

Health Regeneration

These settings are shared by both automatic and manual regeneration. `EntityPassiveHpRegeneration` consumes the passive-tick settings described below, while `EntityHealth.ManualHealthRegenerationTick()` uses the same regeneration `HealSourceSO` together with the dedicated manual-regeneration stat.

> **NOTE**
>
> Configuring the Health Regeneration section does not enable automatic regeneration on any entity by itself. To opt an entity into time-based regeneration, add the `EntityPassiveHpRegeneration` component alongside `EntityHealth`.

Health Regeneration Source

Type: `HealSourceSO`
Required: No
Description: The heal source used for passive regeneration effects and manual regeneration ticks.

Use cases:

- Allows tracking and modifying regeneration separately from active healing
- Can be used for effects like "Increase Regeneration by 50%"
- Tracking passive healing in analytics or combat logs

Passive Health Regeneration Stat (HP/10s)

Type: `Stat`
Required: No
Description: Determines the amount of health regenerated passively.

> **WARNING**
> The stat value represents health regenerated **per 10 seconds**.

Calculation:

```
Health Per Tick = (Stat Value / 10) * Interval In Seconds
```

Example:

- Stat Value: 50 HP/10s
- Interval: 1 second
- Health Per Tick: (50 / 10) * 1 = **5 HP per second**

Passive Health Regeneration Interval

Type: `float`
Default: 1.0 seconds
Required: Yes (must be > 0)
Description: The time (in seconds) between passive regeneration ticks.

Configuration Examples:

Interval	Stat Value	Result
1.0s	50 HP/10s	5 HP every 1 second
0.5s	50 HP/10s	2.5 HP every 0.5 seconds
2.0s	50 HP/10s	10 HP every 2 seconds

> **WARNING**
> Smaller intervals increase CPU overhead. Recommended range: 0.5s - 2.0s.

Suppress Passive Regeneration Events

Type: `bool`
Required: No
Description: If enabled, prevents triggering any heal events during passive regeneration ticks generated by `EntityPassiveHpRegeneration`. Keep it disabled if you need to track passive regeneration in a combat log or if you have effects that trigger on heal events and should also apply to passive regeneration. If your game doesn't require any of the above and you want to minimize overhead, you can enable this option and skip all heal-related logic during regeneration ticks. Useful if your game has a lot of entities with passive regeneration and you want to optimize performance. If you need both to keep sending regeneration events and to minimize overhead, I advise to increase the regeneration interval and to keep the "Suppress Passive Regeneration Events" option disabled. This way you can have less regeneration ticks and still trigger events for each tick.

Manual Health Regeneration Stat

Type: `Stat`
Required: No
Description: Determines health regenerated when triggering manual regeneration via API. The amount of health regenerated is equal to the value of this stat.

Use Cases:

- **Turn-based systems:** Regenerate health at the end of each turn
- **Rest mechanics:** Trigger regeneration when resting at campfires
- **Time-skip systems:** Apply regeneration for elapsed time

API Usage:

```
// Trigger manual regeneration
entityHealth.ManualHealthRegenerationTick();
```

Lifesteal

Lifesteal has **two configuration layers**:

- a **generic** layer configured in `AstraHealthConfigSO`
- an optional **type-specific** layer configured directly on each `DamageTypeSO`

Both layers use `LifestealStatConfig` and can stack on the same hit.

Generic Lifesteal

Type: `LifestealStatConfig`
Required: No
Description: Generic lifesteal configuration applied to all damage dealt by an entity, regardless of `DamageTypeSO`.

Typical Settings:

- Lifesteal percentage stat
- Heal source for generic lifesteal
- Amount selector (initial, step, or final damage)

If the embedded **Lifesteal Stat** is not assigned, generic lifesteal is effectively disabled. This generic contribution stacks with any lifesteal configured on the specific `DamageTypeSO` of the hit.

Suppress Lifesteal Events

Type: `bool`
Required: No
Description: If enabled, prevents triggering heal events for lifesteal heals. Useful when many entities can trigger lifesteal frequently and you do not need listeners, combat-log entries, or UI feedback for those heals.

Unify Lifesteal Heals

Type: `bool`
Required: No
Description: Controls how **Generic Lifesteal** and **Damage-Type Lifesteal** are applied when both contribute to the same hit. It is enabled by default.

Behavior:

- **Enabled** *(default)*: the two amounts are summed into a single heal; the damage type's `HealSourceSO` takes precedence, with fallback to the generic one
- **Disabled**: generic and type-specific lifesteal are applied as two separate heals, each with its own `HealSourceSO`

See also: Lifesteal

Experience

Default Exp Collection Strategy

Type: `ExpCollectionStrategySO`
Required: No

Description: The fallback strategy used by any `ExpCollector` that does not have a custom strategy assigned on the component itself.

Setting this once is usually all you need: every collector in the project that relies on the default will automatically use it without requiring per-entity configuration. Override it on individual `ExpCollector` components only when a specific entity needs different award logic.

See also: Experience Collection

Death

Default On Death Game Action *

Type: `GameAction`
Required: Yes
Description: The game action executed when an entity dies (if the entity doesn't have its own on-death game action). Use a composite game action to chain multiple effects.

Common on-death Game Action Ideas:

- **Destroy GameObject** - Removes the entity from the scene
- **Ragdoll** - Enables ragdoll physics
- **Respawn** - Respawns the entity after a delay
- **Loot Drop** - Spawns loot and destroys the entity

Example:

```
Death → Execute composite on-death Game Action → [Spawn Death VFX → Drop Loot →
Destroy GameObject]
```

Default On Resurrection Game Action

Type: `GameAction`
Required: No
Description: The game action executed when an entity is resurrected (if the entity doesn't have its own on-resurrection game action). Use a composite game action to chain multiple effects.

Common on-resurrection Game Action Ideas:

- **Simple Resurrection** - Restores health and enables the entity
- **Resurrection VFX** - Plays visual effects during resurrection
- **Stat Penalties** - Applies temporary debuffs after resurrection

Default Resurrection Source *

Type: `HealSourceSO`
Required: Yes
Description: The heal source used when an entity is resurrected via convenience overload `Resurrect` methods, or via the built-in Resurrect Game Action.

Use cases:

- Categorizes resurrection healing separately from normal healing
- Allows effects like "Increase Resurrection Healing by 50%"
- Used for analytics and gameplay feedback

Global Events

Global Events are `GameEvent` ScriptableObjects that broadcast health-related information to the **entire game**. They are assigned once in the configuration asset and shared by all entities, ensuring a single authoritative source for framework-level communication.

> **WARNING**
> Three events are **required** — the framework logs an assertion error at startup if they are missing. Assign them before entering Play Mode.

Global Pre Damage Info Event *

Type: `PreDamageGameEvent`
Required: Yes
Description: Raised before any entity takes damage and before the damage calculation pipeline runs. Carries the full `PreDamageContext` (raw amount, type, source, dealer, critical state). Subscribe to this event to implement effects that inspect or modify incoming damage, such as damage-amplifying debuffs or conditional immunity logic.

Per-entity extra events API:

```
entityHealth.Subscribe<PreDamageGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<PreDamageGameEvent>(myEntitySpecificEvent);
```

Global Damage Resolution Event *

Type: `DamageResolutionGameEvent`
Required: Yes
Description: Raised after any entity takes or ignores damage. Carries the full `DamageResolutionContext`

including the outcome (`Applied` or `Prevented`), net amount, and prevention reasons. Also used internally by the framework for lifesteal resolution.

Per-entity extra events API:

```
entityHealth.Subscribe<DamageResolutionGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<DamageResolutionGameEvent>(myEntitySpecificEvent);
```

Global Entity Died Event *

Type: `EntityDiedGameEvent`
Required: Yes
Description: Raised when an entity's HP reaches the death threshold. Carries `EntityDiedContext` with the entity reference and the damage context that caused the death. Used by the Experience Collection system.

Per-entity extra events API:

```
entityHealth.Subscribe<EntityDiedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<EntityDiedGameEvent>(myEntitySpecificEvent);
```

Global Max Health Changed Event

Type: `EntityMaxHealthChangedGameEvent`
Required: No
Description: Raised when any entity's total max HP changes (both increases and decreases). Carries `EntityMaxHealthChangedContext`. Useful for updating UI, recalculating derived stats, or triggering game-wide reactions to stat changes.

Per-entity extra events API:

```
entityHealth.Subscribe<EntityMaxHealthChangedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<EntityMaxHealthChangedGameEvent>(myEntitySpecificEvent);
```

Global Health Changed Event

Type: `EntityHealthChangedGameEvent`
Required: No
Description: Raised whenever any entity's current HP changes, whether increasing or decreasing. Carries `EntityHealthChangedContext`. This is the global event resolved by the corresponding Health Changed Event Channel on each `EntityHealth`, and replaces the previous split between gained-health and lost-health global events.

Per-entity extra events API:

```
entityHealth.Subscribe<EntityHealthChangedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<EntityHealthChangedGameEvent>(myEntitySpecificEvent);
```

Global Pre Heal Event

Type: `PreHealGameEvent`
Required: No
Description: Raised before any entity is healed and before healing modifiers are applied. Carries `PreHealContext` (base amount, source, healer, target). Subscribe to modify heal amounts or trigger pre-heal effects.

Per-entity extra events API:

```
entityHealth.Subscribe<PreHealGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<PreHealGameEvent>(myEntitySpecificEvent);
```

Global Entity Healed Event

Type: `EntityHealedGameEvent`
Required: No
Description: Raised after any entity is healed. Carries `ReceivedHealContext` with the final net heal amount after all modifiers.

Per-entity extra events API:

```
entityHealth.Subscribe<EntityHealedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<EntityHealedGameEvent>(myEntitySpecificEvent);
```

Global Entity Resurrected Event

Type: `EntityResurrectedGameEvent`
Required: No
Description: Raised after any entity is resurrected. Carries `ResurrectionContext` with HP before and after resurrection.

Per-entity extra events API:

```
entityHealth.Subscribe<EntityResurrectedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<EntityResurrectedGameEvent>(myEntitySpecificEvent);
```

Global Health Ratio Changed Event

Type: `HealthRatioChangedGameEvent`
Required: No
Description: Raised whenever the HP/MaxHP ratio changes — either because current HP changed (`AddHealth`/`RemoveHealth`) or because MaxHP changed (`RecalculateMaxHp`). Carries `HealthRatioChangedContext` with raw HP and MaxHP values before and after the change, plus `PreviousValue` and `NewValue` as integer HP percentages (0–100). Use this event to implement HP-ratio-based reactions such as entering a low-health state or triggering threshold-gated abilities.

Per-entity extra events API:

```
entityHealth.Subscribe<HealthRatioChangedGameEvent>(myEntitySpecificEvent);
entityHealth.Unsubscribe<HealthRatioChangedGameEvent>(myEntitySpecificEvent);
```

Troubleshooting

"No Configuration found!" error

Cause: No configuration is assigned and no fallback exists.

Solution:

1. Check **Project Settings** → **Astra Health**
2. Assign a configuration or create a new one
3. Alternatively, create a config named `Astra Health Config` in a `Resources` folder

"Using Fallback" warning

Cause: No explicit configuration assigned in Project Settings.

Solution:

1. Open **Project Settings** → **Astra Health**
2. Assign the fallback configuration explicitly
3. This warning is just informational and won't break functionality

Configuration not updating in Play Mode

Cause: Configuration is cached on first access.

Solution:

```
// Force reload
AstraHealthConfigProvider.Reset();
```

Missing Resources folder

Cause: The `Assets/Resources/` folder doesn't exist.

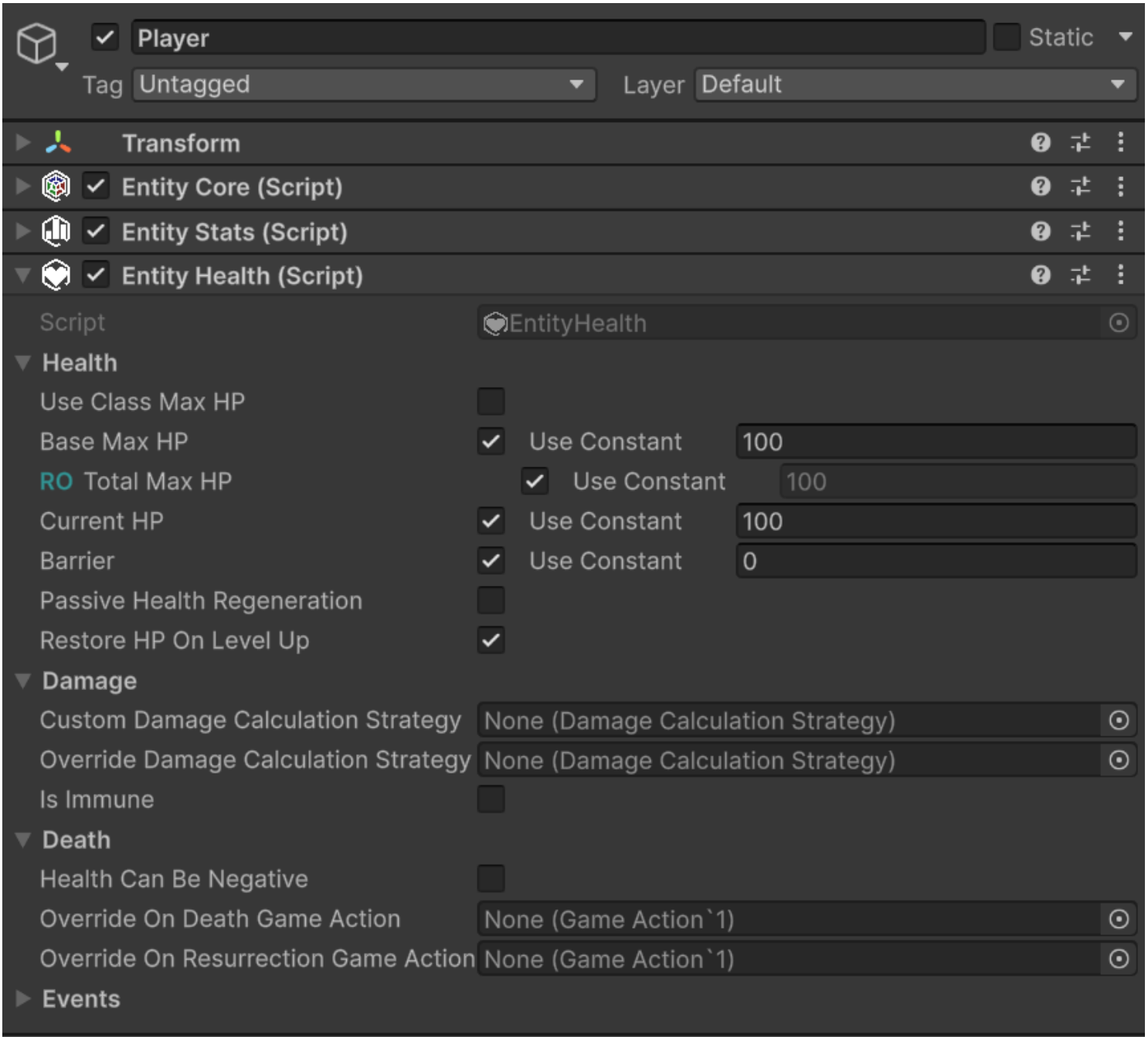
Solution: The package creates it automatically. If it's missing, create it manually:

1. Right-click `Assets`
2. Create → Folder → Name it `Resources`
3. Restart Unity to trigger the bootstrapper

`EntityHealth` Component

With Astra Health, the new `EntityHealth` MonoBehaviour allows you to add health points to an entity. In this way, the entity can take damage and, consequently, die.

Here is an example of the `EntityHealth` component:



The events section is collapsed by default since there are many events, and it is more practical to expand it only when necessary.

> **NOTE**
>
> Automatic time-based regeneration is handled by the dedicated `EntityPassiveHpRegeneration` MonoBehaviour, which you attach alongside `EntityHealth` on entities that should regenerate over time. See Passive Health Regeneration for setup details.

Let's proceed in order and analyze every property of the `EntityHealth` component.

Health

- **Use Class Max HP**: Boolean indicating whether to use the max health points defined in the entity's class as the base max health. If disabled, you can manually specify the base max health for the entity.
- **Base Max HP**: LongRef representing the **base** max health of the entity. If "Use Class Max HP" is enabled, this value is marked with teal **RO** (Read Only). Read Only, for LongRef fields, makes `const` values non-editable, and suggests not manually modifying values contained by the associated LongVar variable, if a `const` value is not used.
- **Total Max HP**: LongRef representing the **total** max health of the entity, calculated based on the base max health and any modifiers. This field is always read-only (RO) and is automatically updated when base max health or modifiers change.
- **Current HP**: LongRef representing the entity's current health points. Editable field that is automatically updated when the entity takes damage or is healed. This field is also updated if `Base Max HP` is modified from the inspector.

> **NOTE**
>
> If `Current HP` drops to 0, modifying `Base Max HP` will not update `Current HP`. This is intended behavior, as if an entity is dead, it makes no sense for its health points to be modified by a change in max health. If instead the entity is alive, modifying `Base Max HP` will update `Current HP` accordingly even if `Current HP` is currently 0. This can happen if, for example, you change from 10 base max HP to 25 by deleting the text box with backspace: 10 (backspace) -> 1 (backspace) -> 0 (2 pressed) -> 2 (5 pressed) -> 25. Note that by deleting all numbers and leaving the box empty, `Base Max HP` becomes 0, and consequently `Current HP` also becomes 0. At this point, the entity is considered dead.

- **Barrier**: LongRef representing any barrier points (or temporary HP) of the entity. The barrier absorbs damage before it affects the entity's health points.
- **Restore HP On Level Up**: Boolean indicating whether the entity is fully healed when leveling up.

Damage

For a better understanding of the first two properties of this section, I recommend taking a look at the Damage Calculation Strategy documentation. Simply put, a Damage Calculation Strategy defines how the damage an entity is about to take is calculated (e.g., applying damage mitigation for the defensive stat first, or damage absorption by the barrier first, when to apply the critical multiplier, etc.). Also recall that a default strategy can be assigned via configuration. See Default Damage Calculation Strategy.

- **Custom Damage Calculation Strategy**: Field of type `DamageCalculationStrategy`. If the entity should use a custom damage calculation strategy, you can specify it here. This, if defined, takes precedence over the default one defined via configuration. A common use case could be, for example, a boss that cannot take more than 10% of its max health in damage at a time.
- **Override Damage Calculation Strategy**: Field of type `DamageCalculationStrategy`. If defined, it takes precedence over all other damage calculation strategies. Designed to be assigned at runtime to implement special effects or for testing/debug. For example, an entity is affected by a debuff that turns all physical damage into guaranteed critical hits. This debuff could therefore be implemented through a custom strategy assigned to the entity in this field.
- **Is Immune**: Boolean indicating whether the entity is immune to all damage or not. If enabled, the entity will take no damage, regardless of the damage calculation strategy used.

Death

- **Health Can Be Negative**: Boolean indicating whether the entity's health points can drop below 0 before dying. If disabled, the entity's health points will never drop below 0, and the entity will die as soon as its health points reach 0. If enabled, the entity's health points can drop below 0, and the entity will die only when its health points reach a specified negative value (Death Threshold).
- **Death Threshold**: LongRef representing the entity's death threshold. Visible only if "Health Can Be Negative" is enabled. If the entity's health points reach this threshold, the entity dies.
- **Override On Death Game Action**: Game Action that is automatically executed when the entity dies. If defined, this takes precedence over the one defined in the configuration. Useful for implementing special behaviors upon an entity's death (for example, entities that explode upon death dealing area damage).
- **Override On Resurrection Game Action**: Game Action that is automatically executed when the entity is resurrected. If defined, this takes precedence over the one defined in the configuration. Useful for implementing special behaviors upon an entity's resurrection.

Damage vs RemoveHealth and Heal vs AddHealth

`EntityHealth` provides a couple of public methods to increase current HP and two to decrease them:

- `Heal` and `AddHealth` to increase current HP.
- `TakeDamage` and `RemoveHealth` to decrease current HP.

`TakeDamage` and `Heal` are extensively documented in the Dealing Damage to an Entity and Healing an Entity sections respectively. However, this sounds like a good moment to introduce the difference between these two pairs of methods, and when to use one or the other.

It is important to clarify why two different methods exist to increase and decrease current HP, and when to use one or the other.

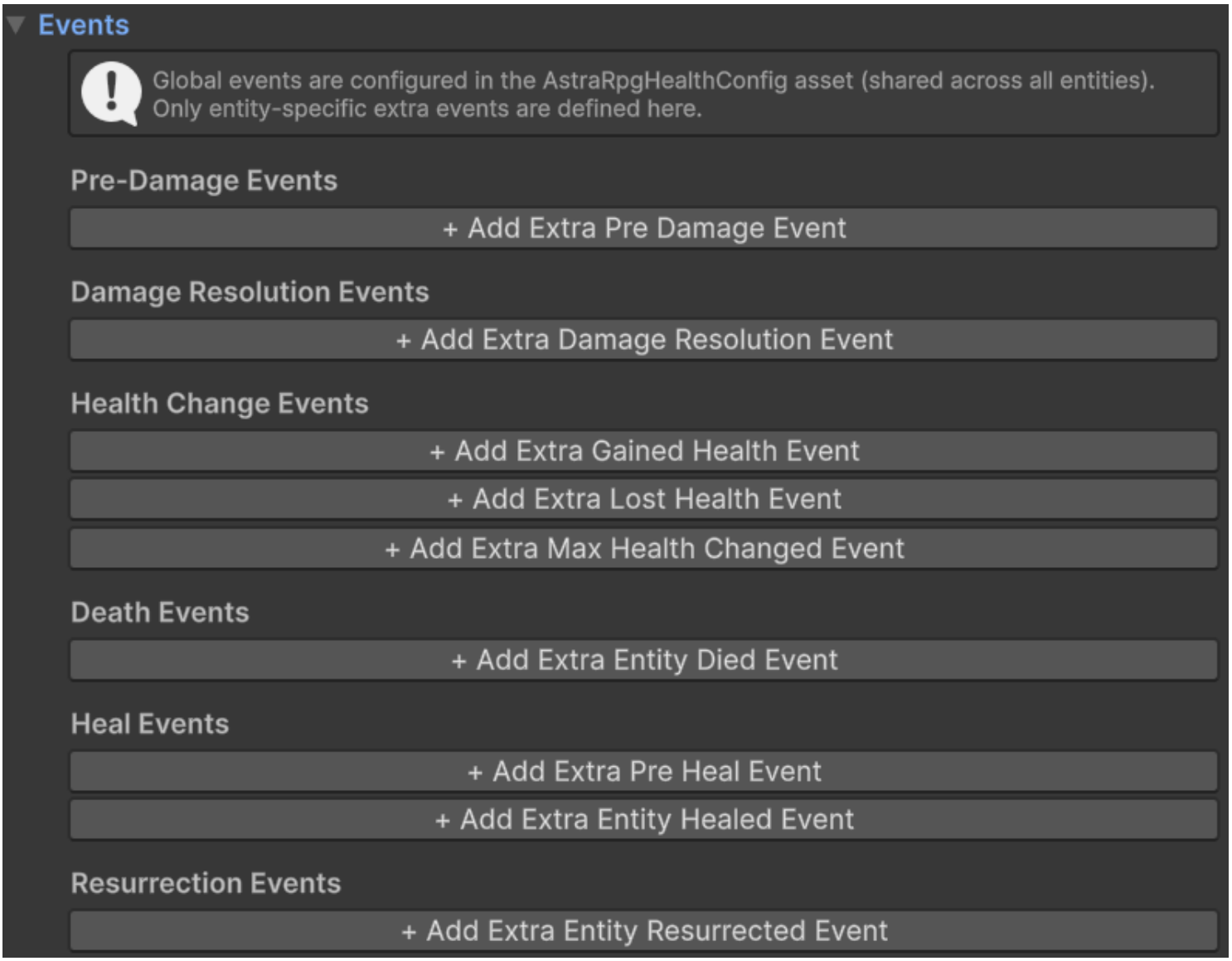
`AddHealth` and `RemoveHealth` operate at a lower level of abstraction than `Heal` and `Damage`, as they directly modify the entity's current HP by a specified `long` value, without passing through the damage calculation pipeline or without taking into account any healing or damage modifiers. They are very predictable and direct methods, and are mainly used internally by the framework. These methods can be useful in specific situations where the gain of HP is not due to healing, or the loss of HP is not due to damage, but rather to particular mechanics that require a direct modification of current HP. For example, suppose that following a cutscene we want to force the player's HP to 1. In this case, we could use `RemoveHealth` to remove all HP except 1, without having to worry about any damage modifiers or the damage calculation strategy that would alter the damage taken based on the player's stats and equipment, as well as raising damage events that could trigger game logic we don't want to activate in this specific case.

`Heal` and `TakeDamage`, on the other hand, are more complex methods that take into account various factors such as the damage calculation strategy, damage immunity, healing modifiers, etc. These methods are intended to be used primarily by game developers to apply damage and healing to entities, as they guarantee that all mechanics and rules of the health system are respected. For example, if you want to inflict damage on an entity, it is advisable to use the `TakeDamage` method, so that the damage is calculated correctly based on the configured damage calculation strategy, and that any immunities or modifiers are taken into consideration. Unsurprisingly, `Heal` and `TakeDamage` internally use `AddHealth` and `RemoveHealth` to effectively modify the entity's current HP after calculating the net HP gain or loss to apply.

In 99% of cases, you will use `Heal` and `TakeDamage` to apply healing and damage to entities. `AddHealth` and `RemoveHealth` are available to cover rarer and more particular use cases.

Events

Events are, by default, collapsed as there are many of them and they would expand excessively in the inspector. By opening them, we should see something like this:



In the image, you don't see any assigned event. Therefore, this is the view of the inspector you should get when you freshly add the `EntityHealth` component to an entity.

Through `EntityHealth` we can configure entity-scoped *Event Channels*. For each signal (e.g., Pre Damage Info Event, Entity Died Event, etc.), the corresponding Event Channel is the per-entity mechanism that orchestrates the raise flow.

Global Events

Global Events broadcast health-related information to the entire game and are configured once at the project level in the AstraHealthConfigSO asset — not per entity on `EntityHealth`. This guarantees a single authoritative source for framework-level communication and eliminates the risk of misconfiguration caused by different entities referencing different event assets.

The framework uses several global events internally: for example, the *Damage Resolution* event powers lifesteal, and the *Entity Died* event is what experience collectors listen to for XP awards. See Global Events

in Package Configuration for the full reference, including which events are required and which framework features depend on each.

Event Channels

Event Channels are assigned directly on the `EntityHealth` component. They are designed to bridge the package-level global communication with optional per-entity communication.

Each Event Channel contains:

- a resolver that retrieves the corresponding Global Event from `AstraHealthConfigSO`
- a list of *Extra Events* that you can assign directly on that specific entity

When the relevant action occurs (e.g., the entity takes damage, the entity dies, etc.), `EntityHealth` raises the corresponding Event Channel. The channel then raises the resolved Global Event configured at package level, if any, and, in the same flow, all Extra Events assigned to that channel.

Extra Events are therefore not an alternative to Event Channels: they are the entity-specific events stored inside them. A practical example is the communication between the player's `EntityHealth` and the HUD displaying their HP: only the player possesses a dedicated HUD, so it is useful to assign an exclusive extra event on the appropriate channel for this interaction. In this way, the HUD does not receive events from all entities, but only those relevant to the player, while the global event is still available for game-wide systems.

`EntityHealth` implements `IEventRegistrar`, which provides `Subscribe<TEvent>` and `Unsubscribe<TEvent>` for adding and removing extra events at runtime:

```
// Register a per-entity listener at runtime
entityHealth.Subscribe<EntityDiedGameEvent>(myEntitySpecificDeathEvent);

// Unregister it
entityHealth.Unsubscribe<EntityDiedGameEvent>(myEntitySpecificDeathEvent);
```

Replace `EntityDiedGameEvent` with the concrete event type for the channel you want to target (e.g., `PreDamageGameEvent`, `DamageResolutionGameEvent`, `EntityHealthChangedGameEvent`, `HealthRatioChangedGameEvent`, etc.).

Events Breakdown

Here is a detailed description of each event. Each signal has a corresponding Event Channel on `EntityHealth`; that channel raises the global event configured in `AstraHealthConfigSO` and any extra events assigned to the entity, so it is sufficient to describe each signal once.

Damage Related Events

To better understand the first two events of this section, I recommend taking a look at the Damage documentation to get a better understanding of damage in Astra Health.

- **Pre Damage Info Event**: Event raised before the entity takes damage, and before the damage calculation pipeline calculates the net damage. This event transmits information about the damage the entity is about to take, such as the damage type, the damage source (an entity, `null` if not applicable), the damage source type (e.g., environmental, skill, trap, etc.), the raw damage you intend to inflict, and other relevant information. For more details regarding the context parameter and its fields, refer to the API documentation PreDamageContext. This event can be useful for implementing passive abilities or, in general, advanced mechanics that trigger effects in response to specific conditions related to the damage an entity is about to take. For example, a debuff or status modifier that amplifies critical damage taken by 50% when the damage type is "Fire", or a passive ability that negates all instances of damage currently being taken if they had raw damage less than 50.
- **Damage Resolution Event**: Event raised after the entity has taken or ignored damage. Similarly to the context parameter of the previous event, this event transmits detailed information about the damage just taken or ignored. For more details regarding the context parameter and its fields, refer to the API documentation DamageResolutionContext.

In general, the thumb rule for deciding whether to use the Pre Damage Info Event or the Damage Resolution Event is the following: if you want to manipulate the damage an entity is about to take, it is better to subscribe to Pre Damage Info Event, and modify the context as desired; if instead you want to react to the damage an entity has just taken, and trigger effects in response to it, it is better to subscribe to Damage Resolution Event, and react based on the information transmitted by its context.

Health Related Events

- **Health Changed Event**: Event raised whenever the entity's current HP changes, whether the value increases or decreases. This includes healing, damage, resurrection, and other mechanics that directly modify current HP such as `AddHealth`, `RemoveHealth`, or max-HP-driven HP adjustments. Refer to the EntityHealthChangedContext API for more details on the context parameter.

> **NOTE**
>
> Because `TakeDamage` and `Heal` internally invoke `RemoveHealth` and `AddHealth`, calling these methods will also raise the `EntityHealthChanged` event.

- **Max Health Changed Event**: Event raised when an entity's total max health points change. This event is raised both when total max hp increase and when they decrease. See EntityMaxHealthChangedContext API for more details on the context parameter.

- **Health Ratio Changed Event**: Event raised whenever the HP/MaxHP ratio changes — whether because current HP changed (via `AddHealth`/`RemoveHealth`) or because MaxHP changed (via `RecalculateMaxHp`). Carries `HealthRatioChangedContext` with raw `PreviousHp`, `NewHp`, `PreviousMaxHp`, and `NewMaxHp` values, plus `PreviousValue` and `NewValue` as integer HP percentages (0–100). Useful for HP-ratio-based conditions — for example, triggering an ability when an entity drops below 25% HP. See HealthRatioChangedContext API for more details on the context parameter.
- **Entity Died Event**: Event raised when the entity dies. See EntityDiedContext API for more details on the context parameter.

Healing Related Events

- **Pre Heal Event**: Event raised before the entity is healed. This event transmits information about the healing the entity is about to receive, such as the amount of HP you intend to heal, the entity that provided the healing (`null` if not applicable), the heal source (e.g., skill, item, potion, etc.) and other relevant information. For more details regarding the context parameter and its fields, refer to the API documentation PreHealContext. The amount of health points intended to heal, transmitted by this event, is the value before applying any healing modifiers. This event can be useful for implementing passive abilities or, in general, advanced mechanics that trigger effects in response to specific conditions related to the healing an entity is about to receive. For example, a buff that amplifies healing derived from potions (heal source) by 30%.
- **Entity Healed Event**: Event raised when the entity has been healed. This event transmits information about the healing just received, such as the amount of HP the entity actually gained after the application of any healing modifiers. For more on the context parameter see the Received HealContext API.

Here too, as for damage events, the thumb rule for deciding whether to use the Pre Heal Event or the Entity Healed Event is the following: if you want to manipulate the healing an entity is about to receive, it is better to subscribe to Pre Heal Event, and modify the context as desired; if instead you want to react to the healing an entity has just received, and trigger effects in response to it, it is better to subscribe to Entity Healed Event, and react based on the information transmitted by its context.

Resurrection Related Events

- **Entity Resurrected Event**: Event raised when the entity is resurrected. See ResurrectionContext API for more details on the context parameter.

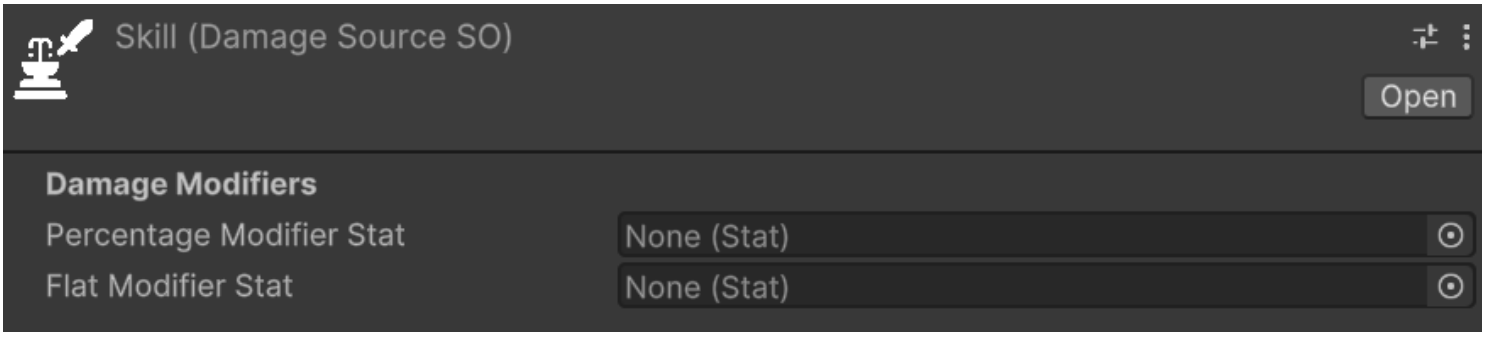
Damage

Damage Sources

Relative path: `Damage Source`

A `DamageSourceSO` represents the source of the damage. Some examples of `DamageSourceSO` could be: skill, base attack, fall damage, trap, environmental, etc. The `DamageSourceSO` is used to categorize the damage and can be used in various mechanics, such as damage modifiers that only apply to specific damage sources, for triggering specific effects when taking damage from a certain source, or for tracking damage statistics based on the source.

An instance of `DamageSourceSO`, in the inspector, should look like this:



Damage Sources - Modifiers

There are two properties to set in a `DamageSourceSO`:

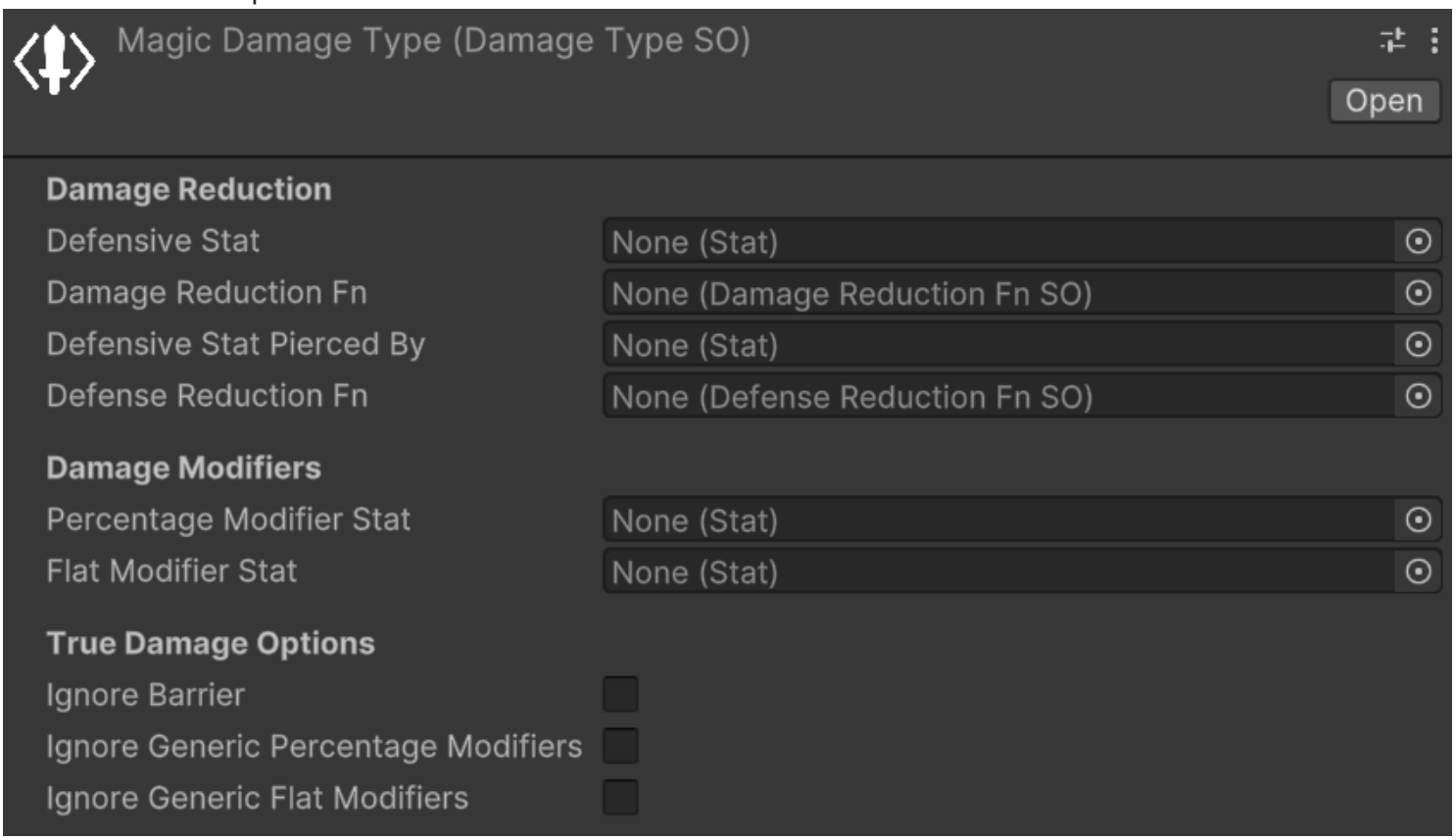
- **Percentage Modifier Stat**: the statistic to consider in an entity to apply percentage, specific damage modifiers for this damage source. Positive values of this statistic increase the damage received from this source, while negative values decrease it.
- **Flat Modifier Stat**: the statistic to consider in an entity to apply flat, specific damage modifiers for this damage source. Positive values of this statistic increase the damage received from this source, while negative values decrease it.

> **WARNING**
> If the entity lacks a percentage or flat damage source modifier statistic, an error will be logged when applying damage from that source. Ensure all entities with an `EntityHealth` component have the statistics referenced in your game's `DamageSourceSO`s.

A possible way to simplify the management of all the `DamageSourceSO` modifier stats is to create a `StatSet` specifically for this purpose, and include it as a *Included Stat Set* in the various entities' `StatSet`s of your game. This way, you centralize all the `DamageSourceSO` modifier stats in a single `StatSet`, and you can easily keep track of them and ensure that they are included in all the relevant entities.

Damage Types

A `DamageTypeSO` represents the type of damage—such as physical, fire, ice, lightning, or damage-over-time (DoT) effects like bleeding. In the following image you cane see an example of a `DamageTypeSO` instance in the inspector:



You can notice that the parameters are divided in four sections:

1. **Damage Mitigation**: parameters related to the damage and defense penetration mechanics for this damage type.
2. **Damage Modifiers**: parameters related to flat and percentage damage modifiers for this damage type.
3. **True Damage Options**: parameters related to the true damage options for this damage type.
4. **Lifesteal**: parameters related to the type-specific lifesteal contribution for this damage type.

> **NOTE**
>
> I recall the Damage Modifiers vs. Stat-Based Damage Mitigation section of the introduction documentation, where I explained the difference between damage mitigation and damage modifiers.

We will now see each of these sections in detail.

Damage Type's Damage Mitigation

The primary use case for `DamageTypeSO` is implementing entities with varying resistances to specific damage types. This is primarily achieved via **Defensive Stats**. For each `DamageTypeSO`, you can define a defensive statistic that reduces incoming damage of that type. For example, an `Armor` stat might reduce `Physical` damage, while a `Magic Resistance` stat reduces `Magic` damage.
The value of the defensive stat is fed into the associated **Damage Mitigation Fn** (function) and used to calculate the actual damage mitigation. The package provides some built-in damage mitigation functions, such as:

- **Flat Dmg Mitigation**: Reduces damage by a flat amount equal to the defensive stat value multiplied by a constant.
- **Percent Dmg Mitigation**: Reduces damage by a percentage equal to the defensive stat value.
- **Log Dmg Mitigation**: Reduces damage in a logarithmic way based on the defensive stat value, providing diminishing returns as the stat increases.

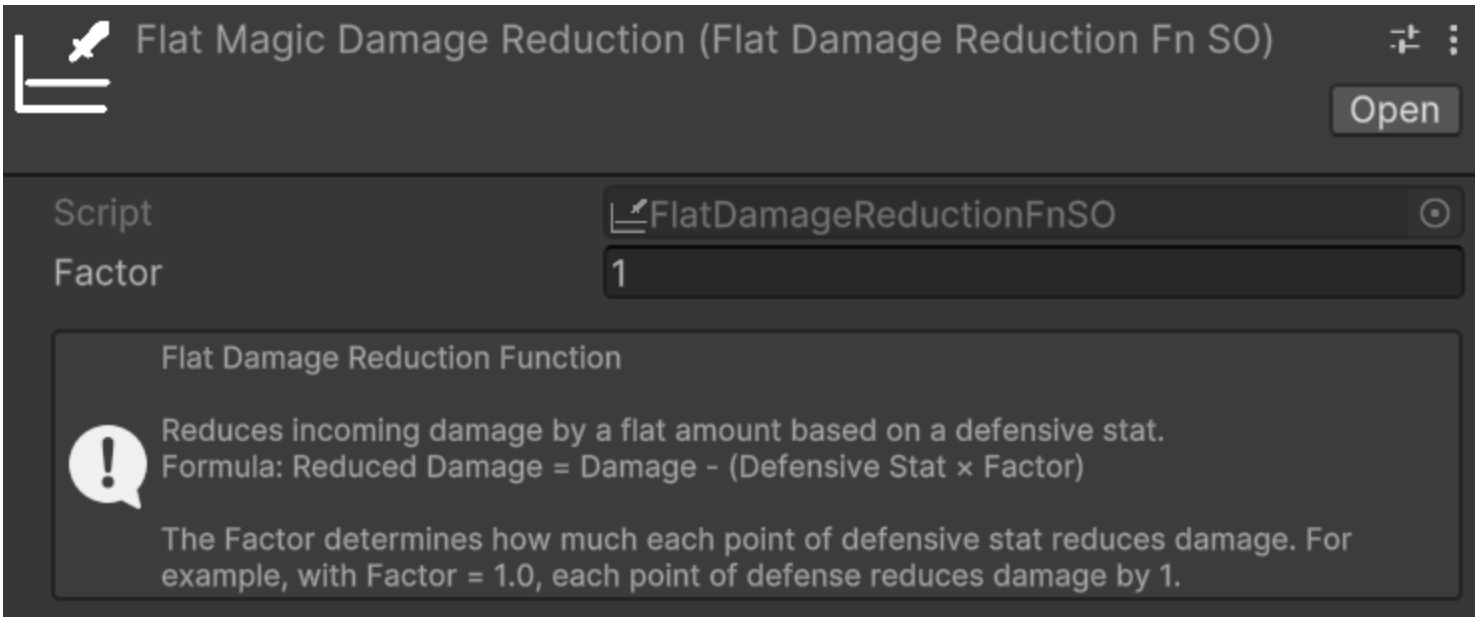
> **NOTE**
>
> **Defensive Stat** and **Damage Mitigation Fn** are optional. However, they must always be configured together: if one is set, the other must be set as well. If only one of them is assigned, a warning will be logged at runtime and the damage mitigation step will be skipped for that `DamageTypeSO`.

> **WARNING**
>
> If the target entity lacks the statistic referenced by **Defensive Stat**, an error will be logged when applying damage of that type. Ensure that all entities with an `EntityHealth` component have the defensive statistic referenced by the `DamageTypeSO`s used in your game.

Let's see all of them in detail.

Damage Mitigation Functions - Flat Dmg Mitigation

Relative path: `Dmg Mitigation Functions -> Flat Dmg Mitigation`



The flat damage mitigation function is the most simple and straightforward one. It reduces damage by a flat amount equal to the defensive stat value multiplied by the **Factor** specified via the Inspector.

For example:

- **Damage Type**: Magic damage.
- **Defensive Stat for the Magic Damage Type**: Magic Resistance.
- **Damage Mitigation Function for the Magic Damage Type**: Flat Dmg Mitigation with a scaling factor of 2.

In this case, if an entity has Magic Resistance equal to 10, and is about to take 50 Magic damage, the damage mitigation will be equal to 10 (Magic Resistance) * 2 (scaling factor) = 20. So the final damage taken by the entity will be 50 (initial damage) - 20 (damage mitigation) = 30 Magic damage. Clearly, this example assumes that there are no other damage modifications (e.g., neutral damage modifiers).

Use Cases: This function is best suited for games where offensive and defensive stat values are low — close to single digits or tens at most. In these scenarios, the linear relationship between the defensive stat and the damage mitigation makes it trivial to ensure that no entity can completely negate incoming damage, as long as the stat values remain within a controlled range. Percentage or logarithmic reduction functions may yield imprecise or unintuitive results at such coarse-grained stat scales.

Pros:

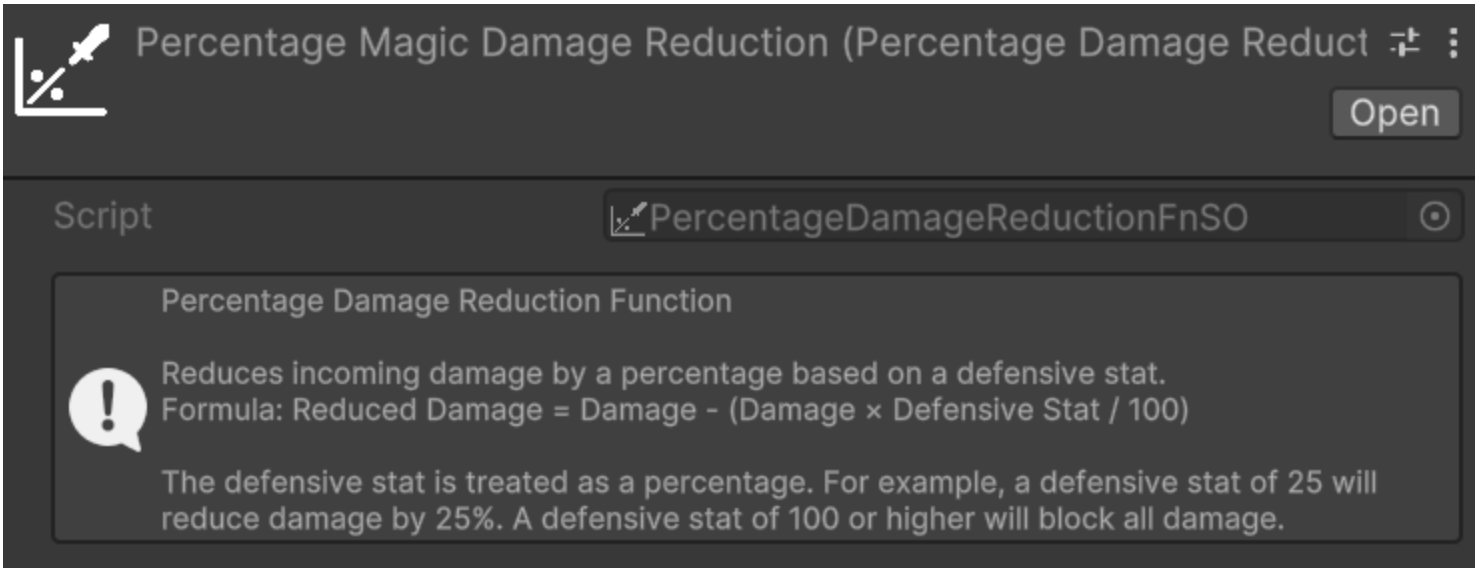
- Damage mitigation is highly predictable: knowing the defensive stat value and the factor, anyone can instantly calculate the resulting reduction.
- Simple to debug: the math is entirely transparent, making it straightforward to verify that the damage pipeline is working as expected at every step.
- Easy to balance: the linear relationship between the stat and the reduction makes it trivial to tune the factor to achieve the desired game feel.

Cons:

- Simplistic system: this function may not suit games that require nuanced or complex defensive mechanics.
- Risk of complete damage negation: if defensive stat values grow too high relative to typical damage values, entities can become entirely immune to certain damage types. This can happen, for example, if the level difference between attacker and defender is too high.

Damage Mitigation Functions - Percent Dmg Mitigation

Relative path: `Dmg Mitigation Functions -> Percentage Dmg Mitigation`



The percentage damage mitigation function reduces incoming damage by a percentage equal to the defensive stat value. For example, if an entity has a defensive stat of 30, it will receive 30% less damage of the associated type.

Use Cases:

- Games where players can face enemies with notable level or power differences. Because the reduction is percentage-based, even a low-level entity with a small but non-zero defensive stat will always receive a proportional reduction, preventing extreme damage scenarios that would arise from large stat disparities between the attacker and the defender.
- Systems that need to be less sensitive than the Flat Dmg Mitigation to the exact magnitude of offensive and defensive stats, while still remaining predictable and easy to balance.

Pros:

- Predictable and straightforward to reason about: a stat value of X directly translates to X% less damage.
- Remains effective regardless of the magnitude of incoming damage: even against a significantly stronger attacker, the same proportional reduction applies, guaranteeing that defense always has a

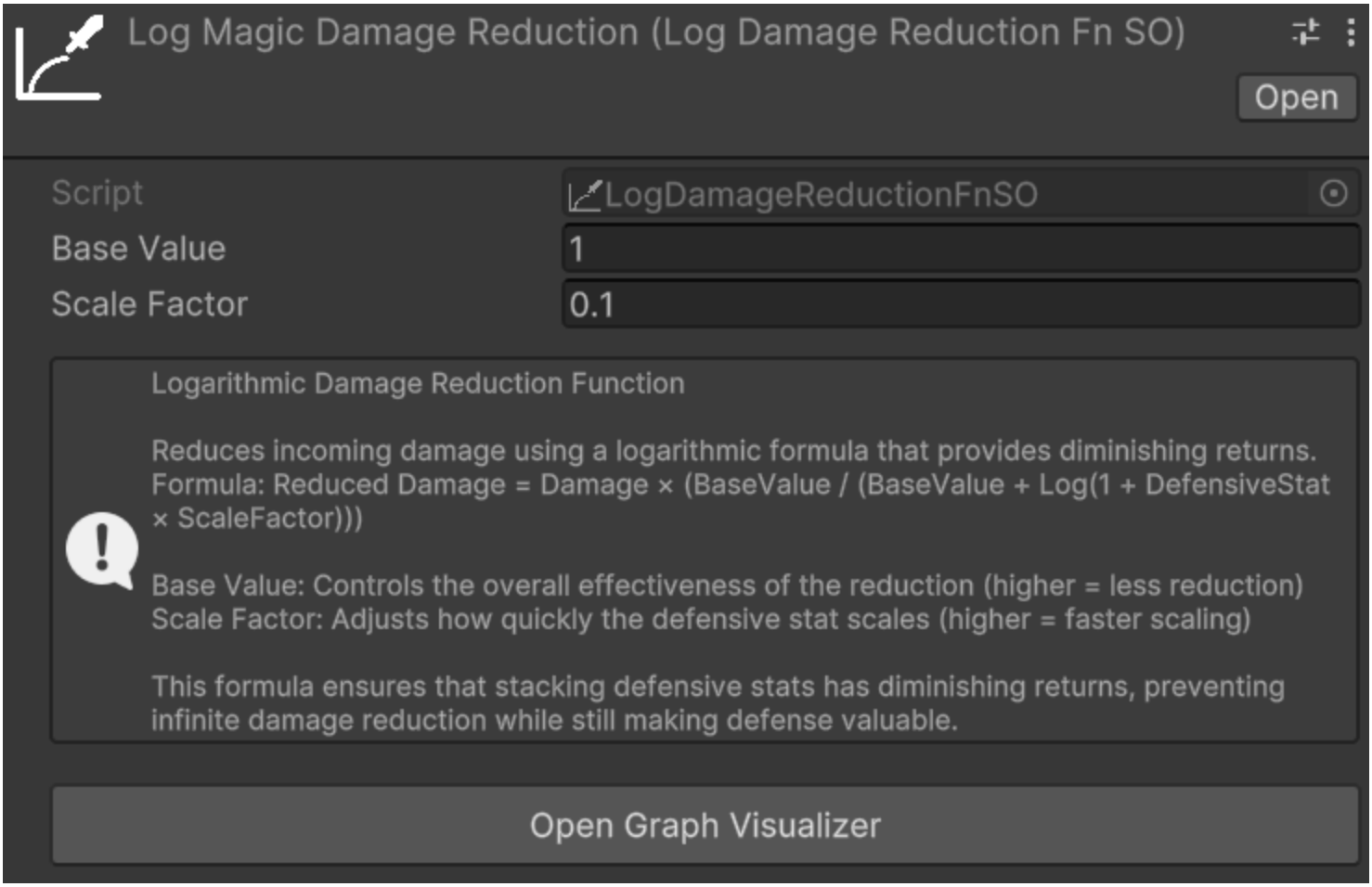
meaningful impact.

Cons:

- High risk of damage immunity at extreme values: once the defensive stat reaches 100, the entity becomes completely immune to that damage type. This can be especially problematic for tank-oriented entities or builds designed to stack defensive stats.
- Can make late-game balancing challenging if stat values are not tightly bounded.

Damage Mitigation Functions - Log Dmg Mitigation

Relative path: `Dmg Mitigation Functions -> Log Dmg Mitigation`



The logarithmic damage mitigation function reduces damage using a logarithmic curve, providing diminishing returns as the defensive stat value increases. A small initial investment in the defensive stat yields a substantial reduction, while further investment produces progressively smaller gains. This makes it theoretically impossible to reach 100% reduction regardless of how high the stat grows.

The Formula

The logarithmic damage mitigation function applies the following formula to compute the final damage taken:

```
Reduced Damage = Damage × (BaseValue / (BaseValue + Log(1 + DefensiveStat × ScaleFactor)))
```

Two parameters, configurable directly on the `Log Dmg Mitigation` asset, control the shape of the curve:

- **Base Value**: the constant in the denominator of the reduction multiplier. A larger Base Value reduces the relative weight of the logarithmic term, making the reduction curve less aggressive — the same defensive stat value will produce less damage mitigation. Conversely, a smaller Base Value amplifies the effect of the logarithm, yielding stronger reductions even at lower stat values.
- **Scale Factor**: the multiplier applied to the defensive stat value before computing the logarithm. A larger Scale Factor causes the curve to climb more steeply at low stat values, reaching substantial reductions earlier. A smaller Scale Factor stretches the curve, requiring higher stat values to achieve the same reduction.

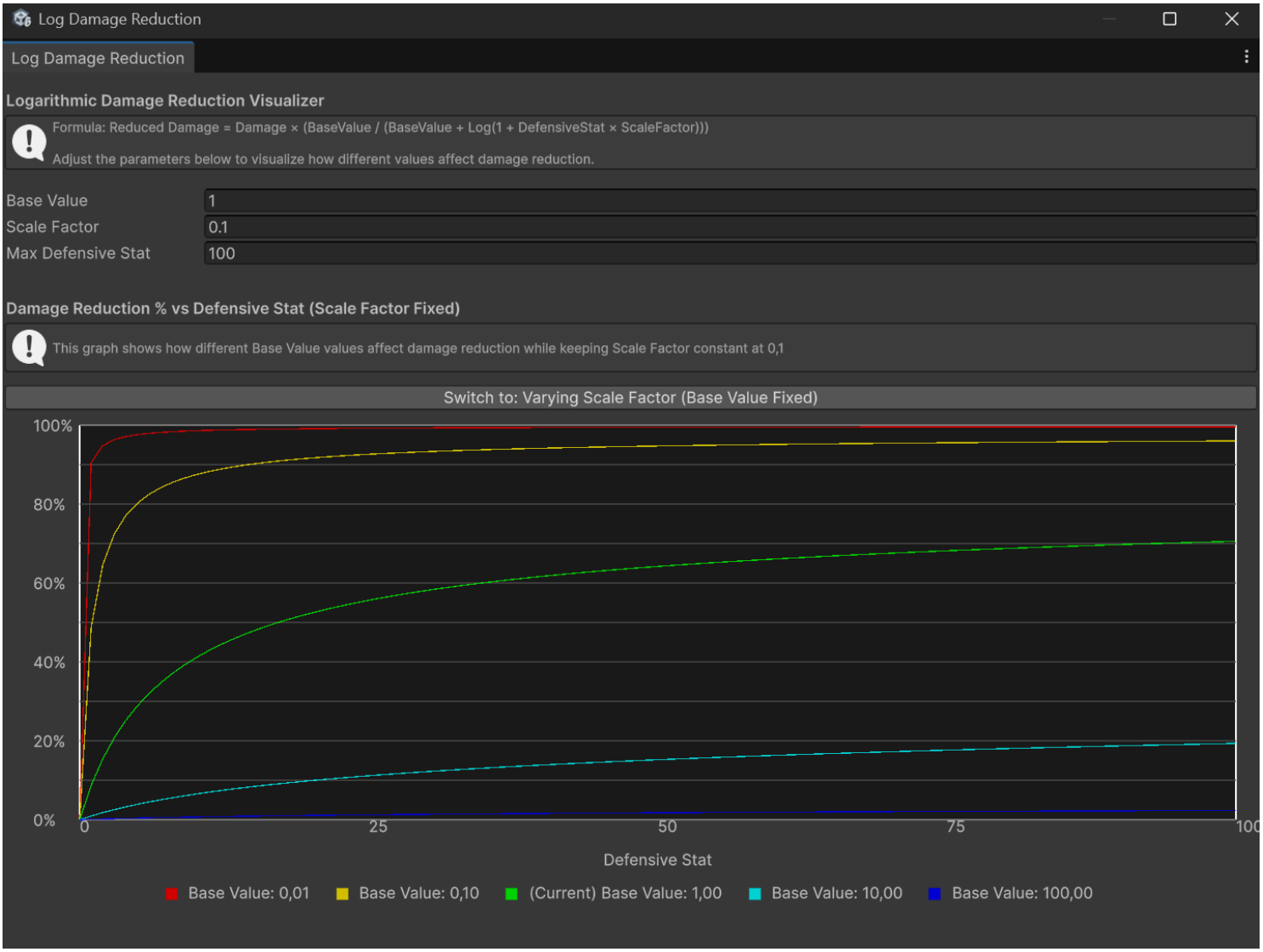
Both parameters must be set to strictly positive values.

Log Damage Mitigation Graph

Because the non-linear nature of the formula makes it difficult to reason about the effect of Base Value and Scale Factor at a glance, the package includes a dedicated visualization window. You can open it in two ways:

- Clicking the **Open Graph Visualizer** button in the inspector of any `Log Dmg Mitigation` asset.
- From the Unity menu: `Window → Astra Health → Log Damage Mitigation Graph`.

The graph visualizer window should look like this:



Once the window is open, three fields let you configure the visualization:

- **Base Value**: the reference Base Value to analyze, matching the value you have set on your asset.
- **Scale Factor**: the reference Scale Factor to analyze, matching the value you have set on your asset.
- **Max Defensive Stat**: the upper bound of the X-axis, i.e., the maximum defensive stat value to plot.

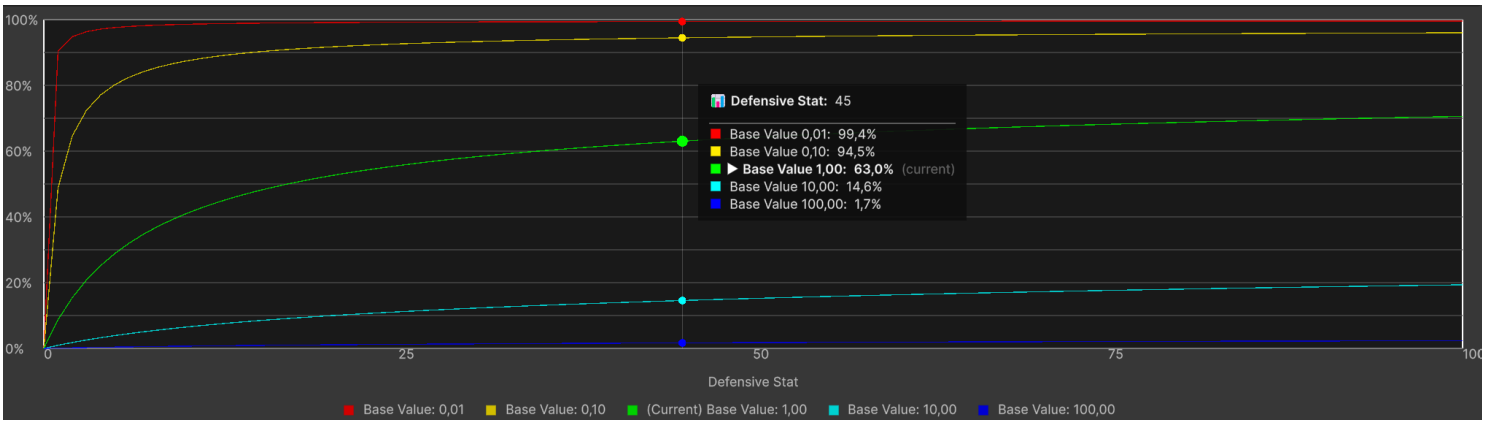
The window displays one graph at a time, plotting **Damage Mitigation %** on the Y-axis and the **Defensive Stat value** on the X-axis. A toggle button lets you switch between two views:

- **Varying Base Value (Scale Factor Fixed)** — shows five curves corresponding to different Base Values centered around the reference value you configured, while Scale Factor is held constant. This lets you compare how increasing or decreasing Base Value shifts the reduction curve, making it easier to find a value that achieves the desired behavior for your game's stat range. Click **Switch to: Varying Scale Factor (Base Value Fixed)** to move to the other view.

- **Varying Scale Factor (Base Value Fixed)** — shows five curves corresponding to different Scale Factors centered around the reference value, while Base Value is held constant. This lets you

compare how Scale Factor affects the steepness of the initial climb of the curve. Click **Switch to: Varying Base Value (Scale Factor Fixed)** to return to the previous view.

The legend labels each curve with its exact parameter value, and the middle curve (marked *Current*) corresponds to the reference value you entered. Hovering the mouse over the graph shows a tooltip with the exact damage mitigation percentage produced by each curve for the defensive stat value under the cursor, like this:



Use Cases:

- RPGs with wide stat ranges and long progression curves — for example, games with levels 1 through 100 or beyond — where both offensive and defensive stats grow substantially over time. The diminishing returns ensure that no entity can become completely immune to a damage type simply by stacking the defensive stat.
- Games where investing in defense should always be viable, but never dominant: players are rewarded for defensive investment, yet the diminishing returns naturally discourage over-specialization and keep combat meaningful at all stages.
- Projects that need a self-capping damage mitigation formula without enforcing a hard maximum: the logarithmic curve naturally prevents extreme values from causing damage immunity, reducing the need for manual clamping or caps in the game design.

Pros:

- Inherently prevents complete damage immunity: the logarithmic curve approaches but never actually reaches 100% reduction, no matter how high the defensive stat grows.
- Scales gracefully across wide stat ranges, remaining meaningful at every stage of the game without requiring constant rebalancing.
- Discourages over-specialization in defense: the diminishing returns act as a natural soft cap, making it progressively less efficient to stack defensive stats beyond a certain point.

Cons:

- Less intuitive than flat or percentage reduction: players and designers cannot easily predict the exact damage mitigation for a given stat value at a glance — the log graph window tool is typically

needed.
- More complex to debug and tune: the non-linear relationship between the stat and the reduction requires more careful analysis and testing during development.

Damage Mitigation Functions - Custom Dmg Mitigation Functions

If you want to provide your own custom damage mitigation function, you can create a new class that inherits from DamageMitigationFnSO and implement the `CalculateReducedDamage` method. Remember to use the `CreateAssetMenu` attribute (or the `MenuItem` attribute) to make it creatable from the Unity editor. You can take a look at the existing damage mitigation functions implementations for reference.

Defense Penetration

Defense penetration allows the damage dealer to partially bypass the target's defensive stat before damage mitigation is calculated. This mechanism is useful for implementing mechanics such as armor penetration or magic penetration, where an attacker can reduce the effective defenses of the target.

Two optional parameters of the `DamageTypeSO` control this behavior:

- **Defensive Stat Pierced By**: the statistic on the **damage dealer** that pierces the target's defensive stat. For example, an `Armor Penetration` stat might pierce the `Armor` stat of the target.
- **Defense Penetration Fn**: the function that computes how the piercing stat lowers the target's defensive stat. The resulting reduced defensive stat value is then passed to the **Damage Mitigation Fn** in place of the original value.

> **NOTE**
>
> **Defensive Stat Pierced By** and **Defense Penetration Fn** are optional. However, they must always be configured together: if one is set, the other must be set as well. If only one of them is assigned, a warning will be logged at runtime and the defense penetration step will be skipped for that `DamageTypeSO`.

> **WARNING**
>
> If the damage dealer entity lacks the statistic referenced by **Defensive Stat Pierced By**, an error will be logged when applying damage of that type. Ensure that all entities capable of dealing damage of this type have the relevant piercing statistic.

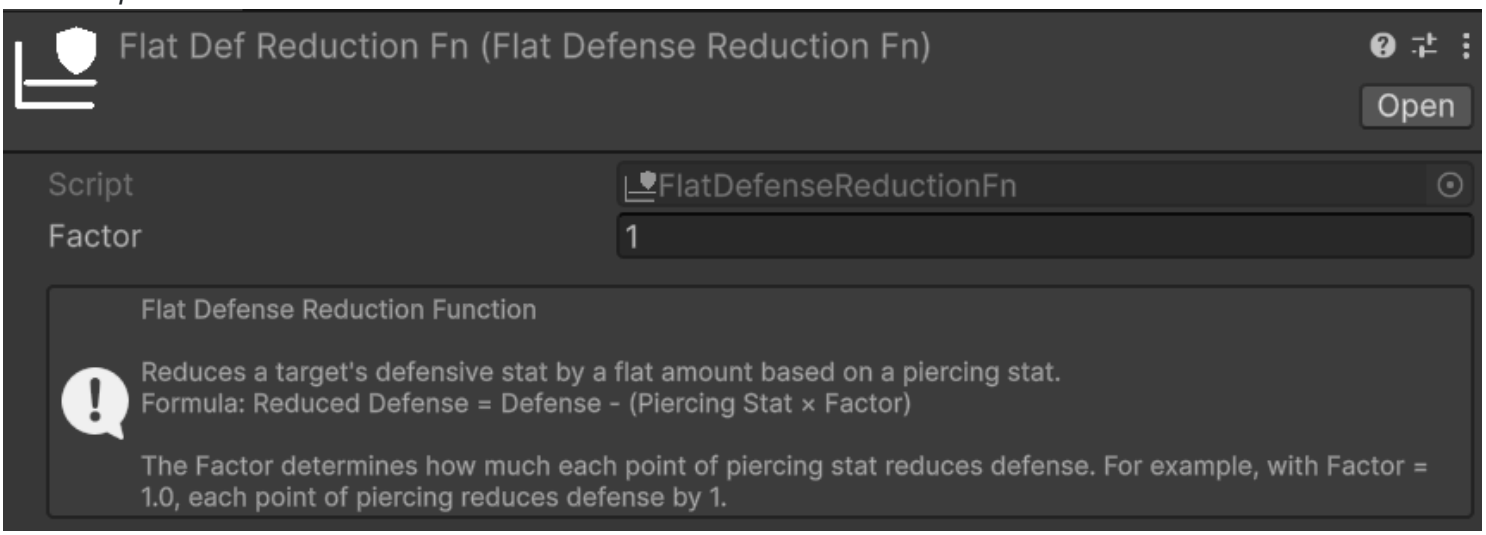
To illustrate how defense penetration interacts with the rest of the damage pipeline, consider the following example:

- **Damage Type**: Physical damage.
- **Defensive Stat for the Physical Damage Type**: Armor.
- **Damage Mitigation Function for the Physical Damage Type**: Flat Dmg Mitigation with a factor of 1.
- **Defensive Stat Pierced By**: Armor Penetration.
- **Defense Penetration Fn**: Flat Def Penetration with a factor of 1.

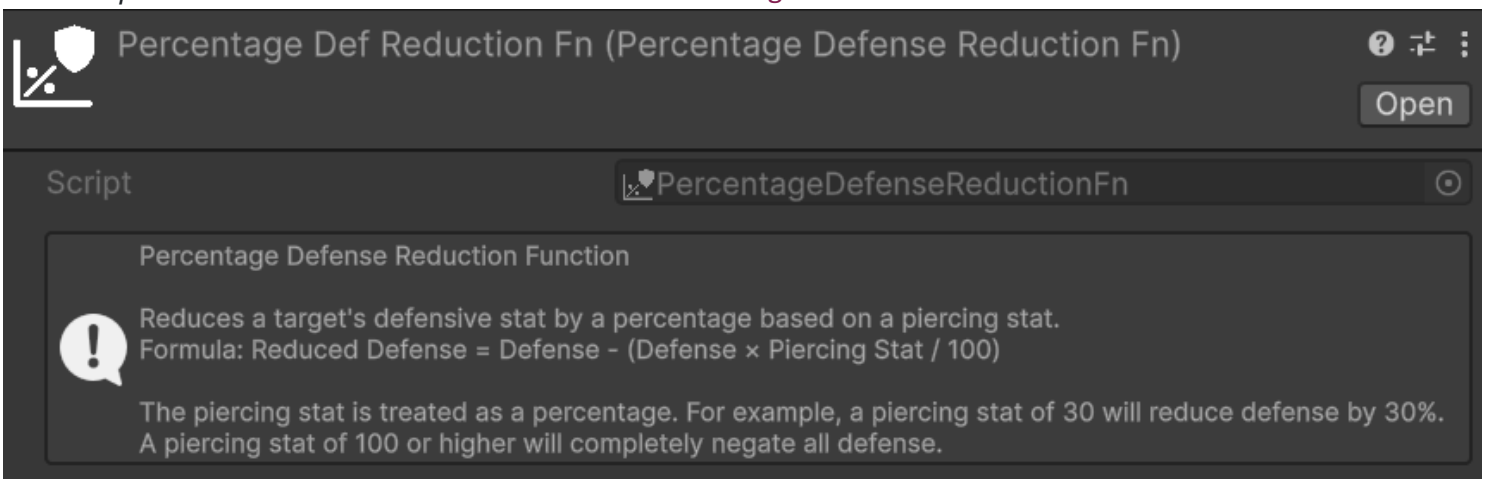
In this case, if the target has Armor equal to 30 and the attacker has Armor Penetration equal to 10, the effective Armor value fed into the Damage Mitigation Fn will be 30 (Armor) − 10 (Armor Penetration) × 1 (factor) = 20. So, if the incoming damage is 80, the final damage taken will be 80 − 20 (effective Armor) = 60 Physical damage. This example assumes no other damage modifications are active.

The package provides three built-in Defense Penetration Functions — Flat, Percentage, and Logarithmic — that work in a fully analogous way to their counterparts described in the Damage Mitigation section above. The parameters, trade-offs, and use cases of each variant are the same; the only difference is that these functions operate on the **defensive stat value** rather than directly on the damage amount. For a detailed description of each, refer to the Damage Mitigation section.

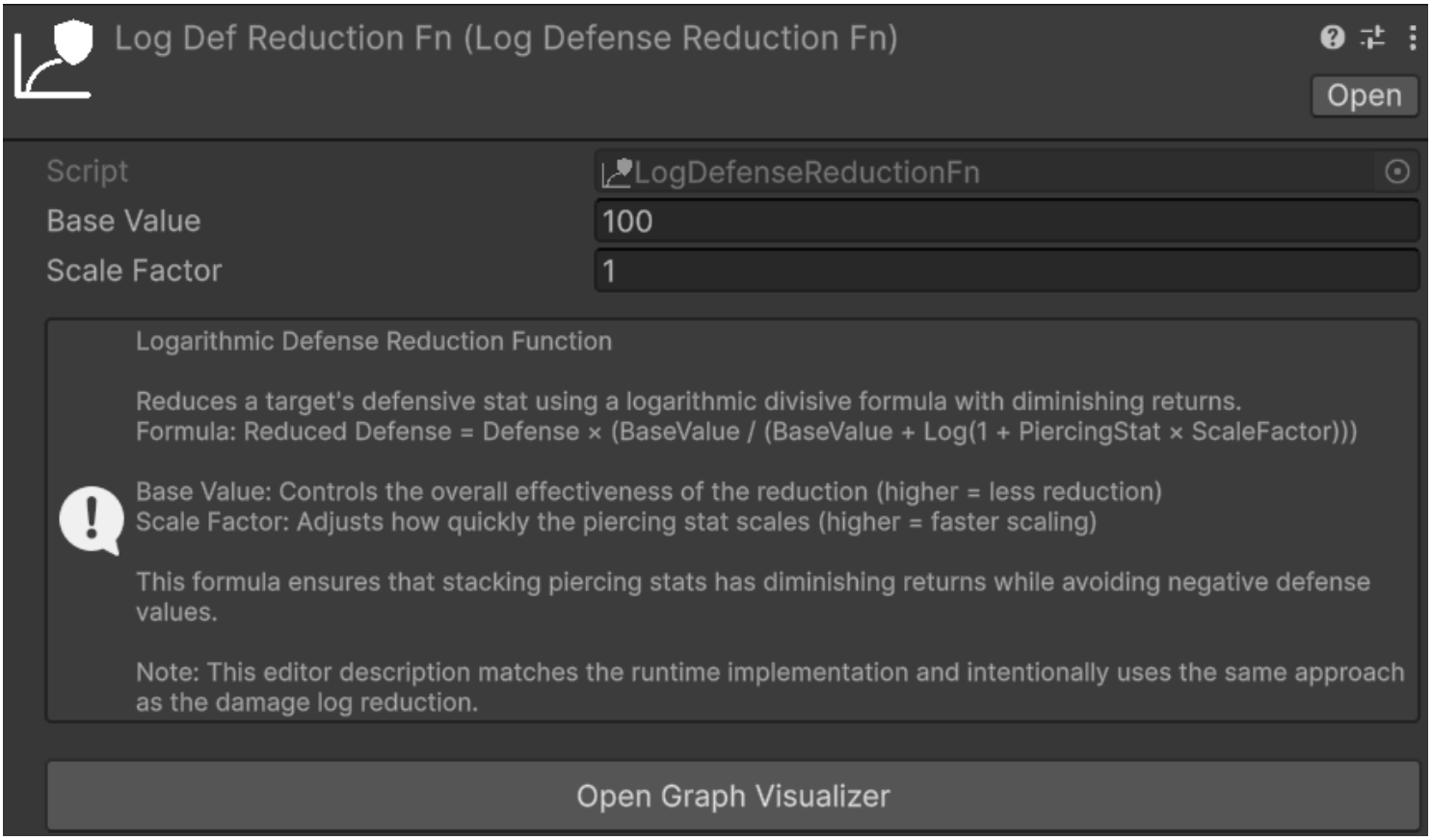
Relative path: `Def Penetration Functions -> Flat Def Penetration`



Relative path: `Def Penetration Functions -> Percentage Def Penetration`



Relative path: `Def Penetration Functions -> Log Def Penetration`



If none of the built-in Defense Penetration Functions suits your needs, you can implement a custom one by creating a class that inherits from DefensePenetrationFnSO and implementing the `CalculateReducedDefense` method. As with the Damage Mitigation Functions, remember to use the `CreateAssetMenu` attribute (or the `MenuItem` attribute) to make it creatable from the Unity editor.

Damage Type's Damage Modifiers

The **Percentage Modifier Stat** and **Flat Modifier Stat** fields in this section let you assign stat-based damage modifiers that apply specifically when an entity receives damage of this `DamageTypeSO`. Assigning a positive value to either stat increases the damage received from this type; a negative value decreases it. For a full explanation of how all damage modifier categories work and how they stack together, see the Damage Modifiers section.

A possible way to simplify the management of all the `DamageTypeSO` modifier stats is to create a `StatSet` specifically for this purpose, and include it as a *Included Stat Set* in the various entities' `StatSet`s of your game. This way, you centralize all the `DamageTypeSO` modifier stats in a single `StatSet`, and you can easily keep track of them and ensure that they are included in all the relevant entities.

Damage Type's True Damage Options

The **True Damage Options** section of a `DamageTypeSO` exposes three boolean flags that allow selective bypasses for individual stages of the damage pipeline. They are the primary tool for implementing "true

damage" mechanics — damage that partially or entirely skips specific mitigation layers — without requiring a custom `DamageCalculationStrategy`.

- **Ignore Barrier**: when enabled, the `ApplyBarrierStep` is skipped entirely for this `DamageTypeSO`. Damage bypasses the target's barrier and is applied directly to its health pool. Use this for damage types that are meant to be unavoidable through shielding — for example, a pure true damage type, fall damage, or damage-over-time effects that should not interact with temporary shields.
- **Ignore Generic Percentage Modifiers**: when enabled, the `ApplyPercentageDmgModifiersStep` skips the **Generic Percentage Damage Modification Stat** configured in `AstraHealthConfigSO`. Percentage modifiers from the `DamageSourceSO` or from this `DamageTypeSO` itself still apply normally.
- **Ignore Generic Flat Modifiers**: when enabled, the `ApplyFlatDmgModifiersStep` skips the **Generic Flat Damage Modification Stat** configured in `AstraHealthConfigSO`. Flat modifiers from the `DamageSourceSO` or from this `DamageTypeSO` itself still apply normally.

> **NOTE**
>
> **Ignore Generic Percentage Modifiers** and **Ignore Generic Flat Modifiers** bypass only the *generic* modifier layer — the global stats configured in `AstraHealthConfigSO`. Source-specific and type-specific modifier stats are never bypassed by these flags and always participate in the pipeline normally.
>
> However, if a `DamageSourceSO` or this `DamageTypeSO` has no modifier stats assigned, those layers contribute nothing by construction — which effectively produces the same result as a bypass. Enabling all three flags while also leaving all modifier stats unset on the source and type produces a fully modifier-free damage type.

> **TIP**
>
> To create a damage type that is also unaffected by any defensive stat, simply leave **Defensive Stat** and **Damage Mitigation Fn** empty. The `ApplyDefenseStep` has nothing to compute and is skipped. Combined with the three True Damage Option flags and no modifier stats, this defines a fully unmitigated damage type.

Damage Type's Lifesteal

Each `DamageTypeSO` exposes a **Lifesteal** field containing a `LifestealStatConfig`.

When its **Lifesteal Stat** is configured and the damage dealer has that stat in its `StatSet`, successful hits of this damage type heal the attacker for a percentage of the selected damage amount. The same three parameters are available as for generic lifesteal:

- **Lifesteal Stat**
- **Lifesteal Source**
- **Amount Selector**

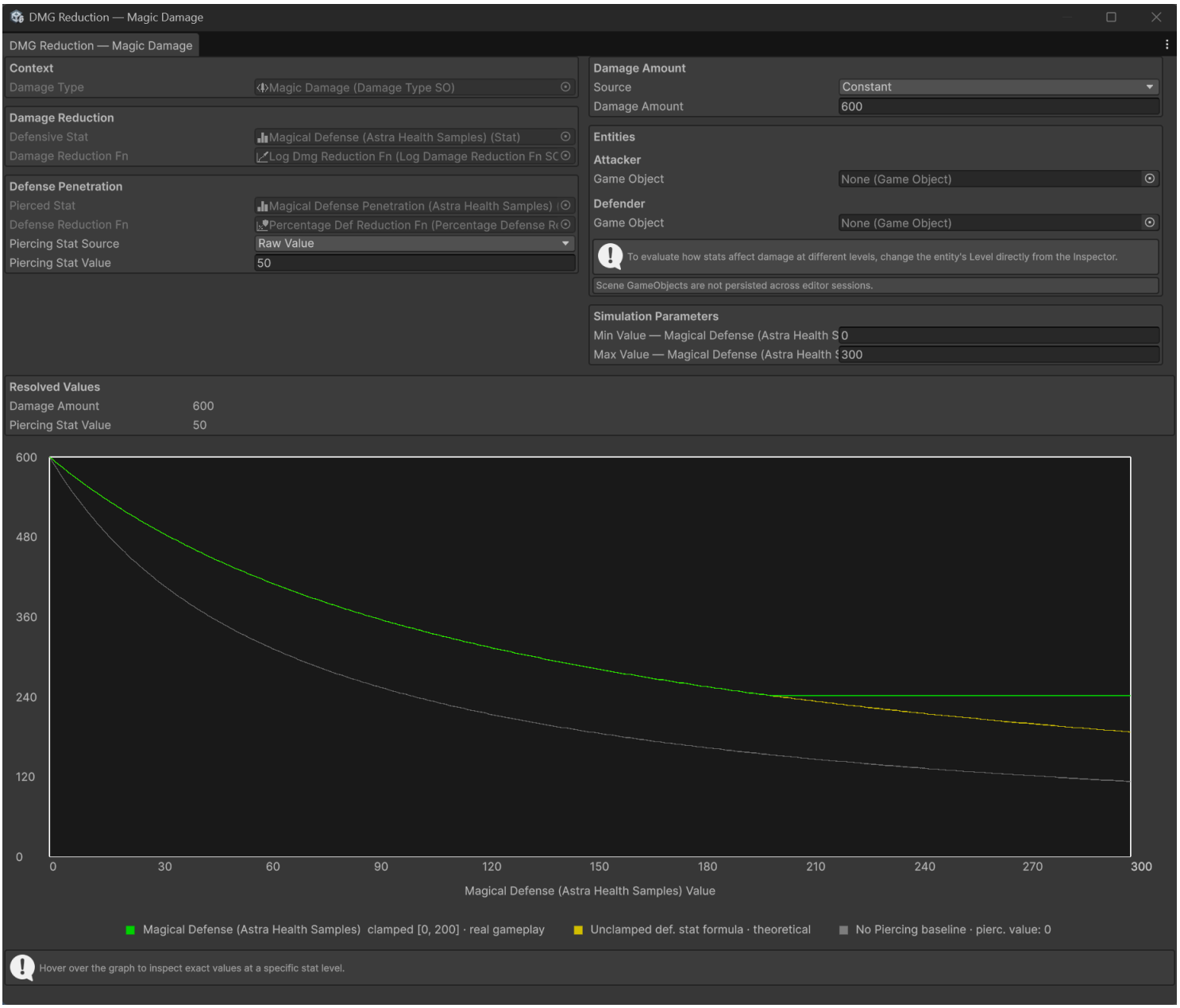
This contribution applies only to hits whose resolved `DamageTypeSO` is this asset. It stacks with the **Generic Lifesteal** configured in `AstraHealthConfigSO`.

For the full behavior — including how this stacks with generic lifesteal and how the `Unify Lifesteal Heals` flag affects the resulting healing events — see Lifesteal.

Damage Mitigation Graph

The **Damage Mitigation Graph** is an Editor window that plots how damage received scales as a function of a defender's defensive stat value. Use it to visually verify the behavior of your damage mitigation and defense penetration configuration, preview the impact of armor penetration at different stat values, and compare clamped (defensive stat) vs. unclamped (defensive stat) curves.

To open it, click the **Open Damage Mitigation Graph** button in the inspector of any `DamageTypeSO`. Once opened, it should look something like this:



Obviously the plot and the values will vary based on the specific configuration of the `DamageTypeSO` you have.

All configuration fields — **Defensive Stat**, **Damage Mitigation Fn**, **Pierced Stat**, and **Defense Penetration Fn** — are read-only inside the window and reflect the asset directly. To change any of them, edit the `DamageTypeSO`.

> **NOTE**
>
> The Damage Mitigation Graph is a **simulation tool only**. No asset or scene object is modified when you change values inside the window.

Control Panel

The window is divided into two equal columns that scale with the window width. Below them are the **Resolved Values** panel, an optional **Defender Breakdown** panel (shown only when a Defender is assigned in the Entities section), and the graph itself.

Context (read-only)
Displays the `DamageTypeSO` that opened the window as a disabled field. It cannot be reassigned here — to analyze a different asset, open its graph from its own inspector.

Damage Mitigation

Field	Description
Defensive Stat	The stat whose value sweeps the X axis of the graph. Read-only — sourced directly from the `DamageTypeSO`. To change it, edit the asset.
Damage Mitigation Fn	The `DamageMitigationFnSO` that converts `(damage, effectiveDefense)` into `finalDamage`. Read-only — sourced directly from the `DamageTypeSO`. To change it, edit the asset.

If either field is left empty, a warning is shown and the graph is not drawn.

Defense Penetration

Field	Description
Pierced Stat	The stat on the attacker that provides armor penetration, read-only — sourced from `DamageTypeSO.DefensiveStatPiercedBy`. To change it, edit the asset.
Defense Penetration Fn	Optional `DefensePenetrationFnSO` that reduces the defender's stat before damage mitigation is applied. Read-only — sourced directly from the `DamageTypeSO`. To change it, edit the asset. When assigned, a **Piercing Stat Source** selector appears.

The **Piercing Stat Source** controls where the piercing stat value comes from:

Source	Description
Raw Value	A plain number field. Type the piercing stat value directly.
Growth Formula	A `GrowthFormula` asset plus a Level field clamped to the formula's valid range. The resolved value is displayed inline.

Source	Description
Class at Level	A `Class` asset with either a constant level integer or a runtime `IntVar`. The piercing stat is read from `DamageTypeSO.DefensiveStatPiercedBy` for that class at the given level. Requires **Defensive Stat Pierced By** to be set on both the `Class` and `DamageTypeSO` assets.
From Attacker	Reads `DamageTypeSO.DefensiveStatPiercedBy` directly from the Attacker entity set in the Entities section. Requires **Defensive Stat Pierced By** to be set on the `DamageTypeSO` asset and an Attacker to be assigned. The Attacker must also have the **Defensive Stat Pierced By** stat in its `EntityStats` component.

> **NOTE**
> If no **Defense Penetration Fn** is assigned, the Piercing Stat Source controls are hidden and the grey baseline line is not drawn on the graph.

Damage Amount

Source	Description
Constant	A plain number field. Defaults to 100.
Scaling Formula	A `ScalingFormula` asset evaluated against the Attacker and/or Defender entities configured in the Entities section. Info panels in the window list which entity components the formula requires.

Entities

Field	Description
Attacker	An in-scene `GameObject` with `EntityCore`. Required when Damage Amount source is **Scaling Formula** or Piercing Stat Source is **From Attacker**.
Defender	An in-scene `GameObject` with `EntityCore` and `EntityStats`. When assigned, the **Defender Breakdown** panel appears below the Resolved Values panel.

> **TIP**
> To simulate how the graph changes at different entity levels, modify the entity's **Level** field directly in the Inspector while the window is open. The graph updates in real time. Level controls are intentionally absent from the window itself to avoid triggering `OnEntityLevelUp` / `OnEntityLevelDown` events while interacting with the graph window.

Simulation Parameters

Field	Description
Min Value — {stat name}	Left bound of the X axis. Defaults to 0. Always editable, regardless of whether the Defensive Stat has a Min Value defined.
Max Value — {stat name}	Right bound of the X axis. Always editable.

Reading the Graph

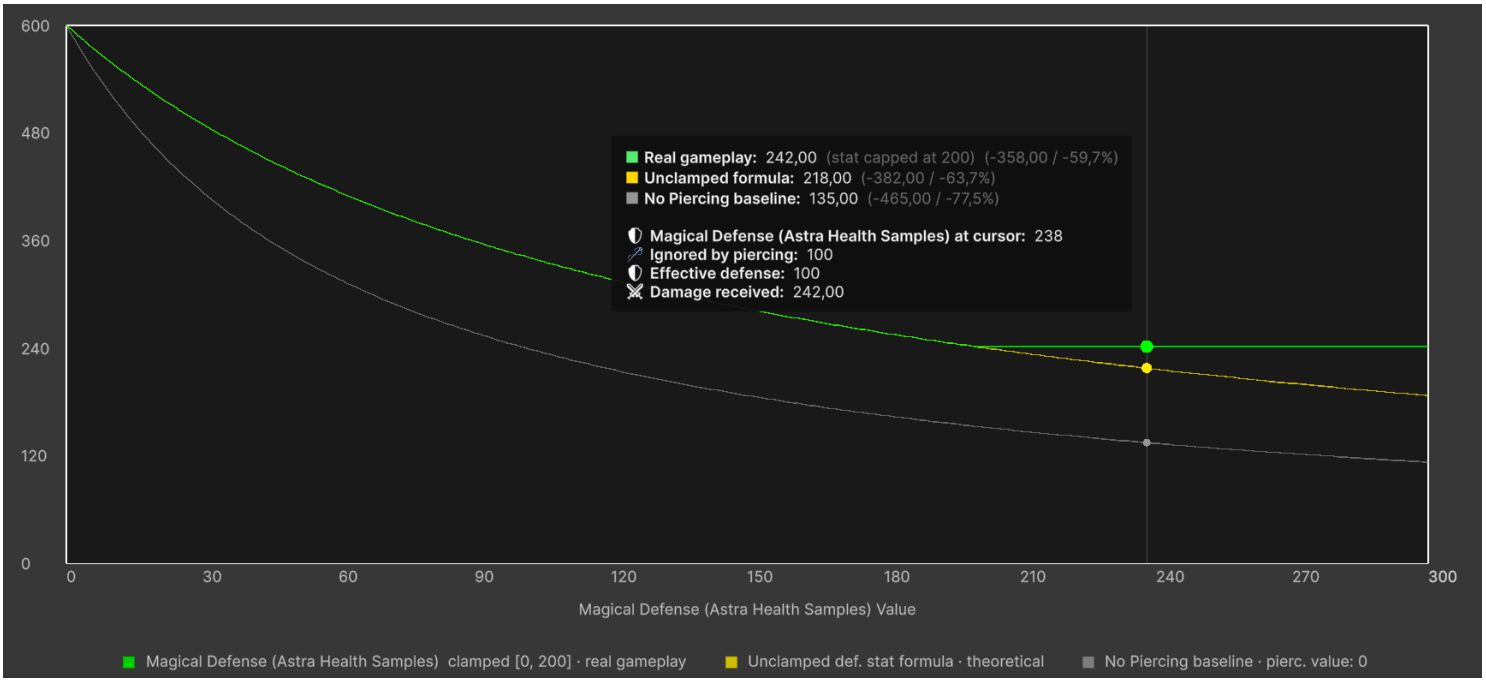
The graph plots **Damage Received** on the Y axis and the **Defensive Stat Value** on the X axis. Up to three lines are drawn simultaneously:

Line	Color	Condition	What it represents
Real gameplay	Green	**Always**	Damage received as the stat grows, with the stat clamped to its configured bounds (if any). This is the primary reference curve — it reflects the exact behavior any entity can exhibit in-game. When the stat has no bounds configured, this line coincides with the unclamped formula and is the only curve shown.
Unclamped formula	Yellow	Only when the Defensive Stat has a **Min Value** and/or **Max Value**	Same formula but with the stat **not** clamped to its bounds — useful for understanding the theoretical shape of the reduction function and how much the configured bounds constrain it. Only shown when it meaningfully differs from the green line. It may happen that you still see only the green line if the bounds do not affect the outcome (aka. the two lines coincide).

Line	Color	Condition	What it represents
No-piercing baseline	Grey	Only when **Defense Penetration Fn** is assigned and effective piercing ≠ 0	The damage curve with piercing forced to zero. Lets you immediately see how much the configured penetration shifts the outcome relative to a no-penetration scenario.

The legend below the graph labels each active line with its color and — for the green line — the exact bound(s) at which the stat is clamped.

Hover tooltip — moving the mouse over the graph shows a tooltip with, for each active line, the damage received and the absolute and percentage reduction at the hovered stat value. The detail rows also show the raw defensive stat value at the cursor and, when a `DefensePenetrationFnSO` is set, the portion of the stat ignored by piercing and the resulting effective defense.
It should look like this:



Resolved Values and Defender Breakdown

The **Resolved Values** panel, always visible below the control columns, shows the values actually fed into the graph — the resolved damage amount and the resolved piercing stat value. If either resolution fails (e.g., a required entity is missing or a formula throws an exception), an error message is shown here and the graph is suppressed.

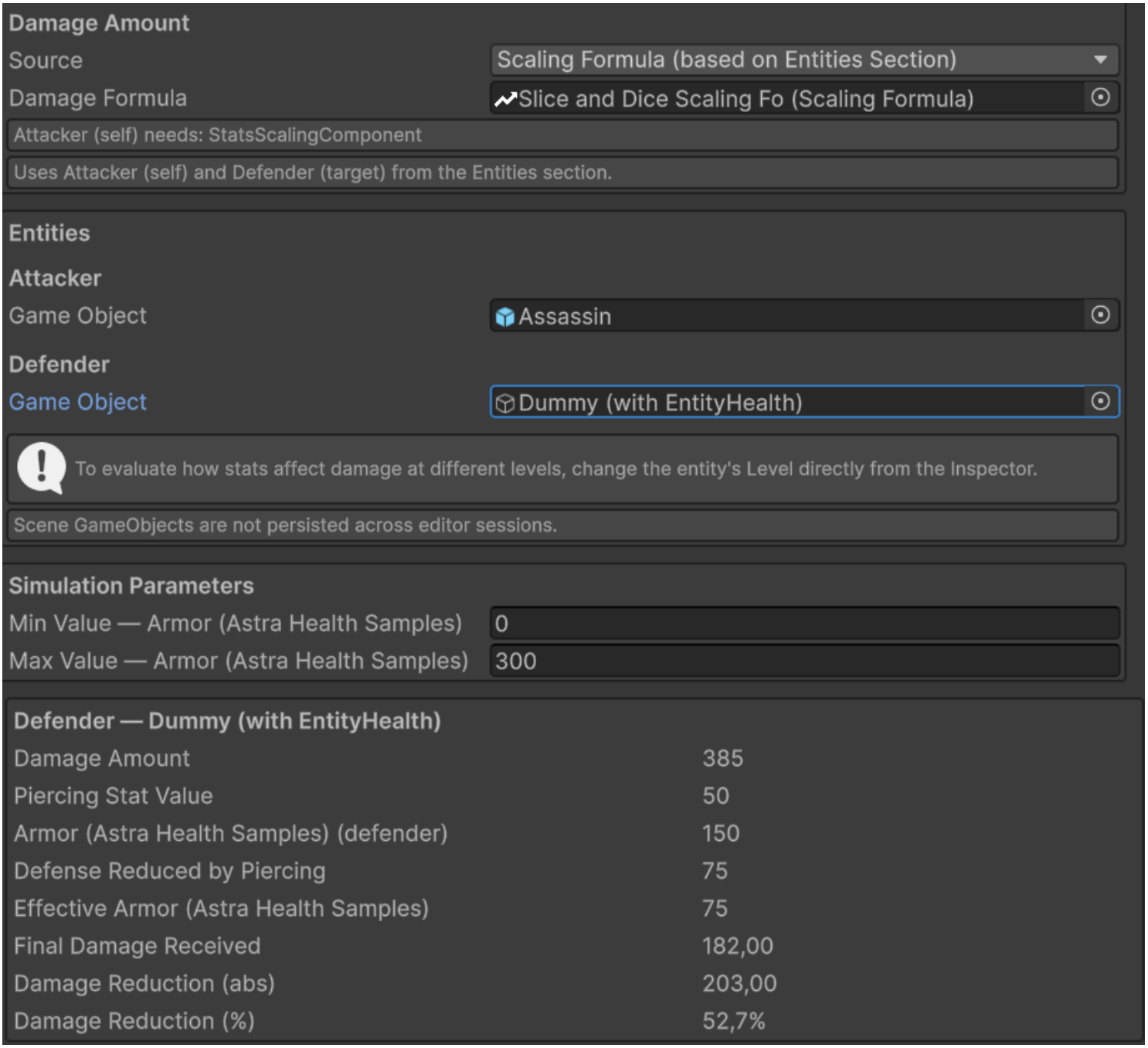
When a **Defender** is assigned with `EntityCore` and `EntityStats`, the **Defender Breakdown** panel appears alongside the Resolved Values panel. It provides a point-in-time snapshot of the exact damage

calculation for the defender at its current in-scene stat values:

Row	Description
Damage Amount	The resolved incoming damage
Piercing Stat Value	The resolved piercing value
`{stat.name}` (defender)	The defender's current defensive stat value
Defense Reduced by Piercing	How much of the defensive stat is bypassed by the piercing value (only when **Defense Penetration Fn** is set)
Effective `{stat.name}`	The final defensive stat fed into the damage mitigation function (only when **Defense Penetration Fn** is set)
Final Damage Received	The final damage the defender would receive at its current level and stats
Damage Mitigation (abs / %)	The absolute and percentage difference between raw and final damage

The breakdown reflects the Defender's **current** in-scene stat values at its current level. To see how the numbers change at a different level, modify the entity's Level field directly in the Inspector as described above.

Here is a screenshot of the Defender Breakdown panel in action:



Notice that a `ScalingFormula` is used for the damage amount, the *Dummy* entity set as Defender in the Entities section, and the Defender Breakdown panel on the bottom showing the exact damage calculation for the Defender based on its current stats and level.

Dealing Damage to an Entity

The API method you will use the most with this package is certainly `TakeDamage`, whose responsibility is to apply damage to the entity, taking into account modifiers, immunity, the damage calculation strategy, and other relevant mechanics. This method takes a `PreDamageContext` as input.

The recommended way via code to inflict damage on an entity is as follows:

1. Construct an instance of `PreDamageContext` with all relevant information about the damage you intend to inflict through its fluent builder.
2. Call `TakeDamage` passing the newly constructed context.

Suppose we are implementing a skill that deals 50 fire damage to the target. The code to apply damage to the target could be the following:

```
// Assuming that:
// - dmgType is a DamageType representing fire damage
// - dmgSource is a DamageSource representing the damage coming from a skill
// - target is the EntityCore that we want to damage
// - skillCaster is the EntityCore that casts the skill

// first we ensure that the target has an EntityHealth component
if (target.TryGetComponent(out EntityHealth targetHealth)) {
    // then we build the PreDamageContext with all the relevant information
    var preDamageContext = PreDamageContext.Builder
            .WithAmount(50)
            .WithType(dmgType)
            .WithSource(dmgSource)
            .WithTarget(target)
            .WithDealer(skillCaster)   // optional
            .Build();

    // finally, we call TakeDamage to apply the damage to the target
    targetHealth.TakeDamage(preDamageContext);
}
```

Thanks to the `PreDamageContext` fluent builder, the IDE will automatically suggest the fields to fill in one at a time. As long as it presents them one at a time, it means they are required fields. If instead it presents more than one at a time, it means those fields are optional, and you can decide whether to fill them in or build the context without them. Optional fields include the damage dealer (`WithDealer`), the critical hit flag, the critical multiplier, the instigator (`WithInstigator`), and the reactability flag (`WithIsReactable`). In the example, for simplicity, I did not fill in these fields.

Now, we know that hardcoding the damage value directly in the code is not a good practice. Let's see how to use a `ScalingFormula` to dynamically calculate the amount of damage to inflict. The step builder creation would become the following:

```
// Assuming that scalingFormula is a ScalingFormula that calculates the damage amount based
on the skill caster's stats...

// ...we calculate the damage amount by evaluating the scaling formula
```

```
long damageAmount = scalingFormula.CalculateValue(skillCaster);
var preDamageContext = PreDamageContext.Builder
        .WithAmount(damageAmount)
        .WithType(dmgType)
        .WithSource(dmgSource)
        .WithTarget(target)
        .WithDealer(skillCaster)   // optional
        .Build();
```

Damage Modifiers

Damage modifiers are a flexible tool for implementing mechanics such as resistances, weaknesses, buffs, and debuffs that affect how much damage an entity receives. Unlike Defensive Stat-Based Damage Mitigation, damage modifiers are off by default — their stats default to 0 — and can both increase and decrease the damage amount.

Three categories of damage modifiers exist, and they all stack additively with one another:

- **Generic modifiers**: apply to all damage received by an entity, regardless of damage type or source.
- **DamageSource modifiers**: apply only when the damage originates from a specific `DamageSourceSO`.
- **DamageType modifiers**: apply only when the damage is of a specific `DamageTypeSO`.

Generic Damage Modifiers

Generic modifiers are configured in the `AstraHealthConfigSO` asset and apply universally to every instance of damage received by an entity. The two relevant fields are **Generic Flat Damage Modification Stat** and **Generic Percentage Damage Modification Stat**, described in detail in the Package Configuration page.

As a quick recap:

- **Generic Flat Damage Modification Stat**: a `Stat` whose value is added to (or subtracted from) the incoming damage amount as a flat quantity. A positive value increases the damage received; a negative value decreases it.
- **Generic Percentage Damage Modification Stat**: a `Stat` whose value is applied as a percentage modification to the incoming damage. A value of 20 means +20% more damage received; a value of −20 means −20% less damage received.

DamageSource Modifiers

As already introduced in the Damage Sources section, each `DamageSourceSO` asset exposes a **Percentage Modifier Stat** and a **Flat Modifier Stat**. These work identically to the generic modifiers described above, but are applied only when the damage originates from that specific `DamageSourceSO`.

DamageType Modifiers

Similarly, each `DamageTypeSO` asset exposes a **Percentage Modifier Stat** and a **Flat Modifier Stat** in its **Damage Modifiers** inspector section. These work in the same way as the source-specific modifiers, but are applied only when the damage is of that specific `DamageTypeSO`. A typical use case is implementing elemental weaknesses and resistances: for example, a `Fire Weakness` stat could be assigned as the **Percentage Modifier Stat** of a Fire `DamageTypeSO`, so that entities with a positive value of that stat take proportionally more Fire damage.

> **WARNING**
>
> As with `DamageSourceSO` modifiers, if the target entity lacks any statistic referenced by the **Percentage Modifier Stat** or **Flat Modifier Stat** fields on a `DamageTypeSO`, an error will be logged when applying damage of that type. Ensure that all entities with an `EntityHealth` component have the damage modifier statistics referenced by the `DamageTypeSO`s used in your game. A practical approach is the same suggested for `DamageSourceSO` modifiers: centralize all modifier stats in a dedicated `StatSet` and include it in the relevant entities' stat sets.

Stacking Behavior

When the `ApplyFlatDmgModifiersStep` and `ApplyPercentageDmgModifiersStep` steps are included in the active `DamageCalculationStrategy` (that we will see soon in the Damage Calculation Strategy section), all applicable modifiers of the same kind are **summed additively** into a single net value, which is then applied to the current damage amount in one operation.

For percentage modifiers, the system also performs individual immunity checks before summing: if the generic percentage modifier alone reaches −100 or below, the damage is fully prevented and the entity is flagged as immune to all damage for that hit; if the source- or type-specific modifier alone reaches −100 or below, the damage is prevented and the entity is flagged as immune to that specific source or type respectively.

Damage Calculation Pipeline

When you call `TakeDamage` on an `EntityHealth`, the raw damage amount you specify does not reach the entity's health pool directly. Instead, it passes through the **Damage Calculation Pipeline**: an ordered sequence of processing steps, each responsible for one aspect of the calculation — barriers, critical multipliers, modifiers, and defenses. Think of it as a production line: raw damage enters at the first station and is transformed by each step in turn; what exits the last step is what the entity actually loses from its health.

The strength of this design is its configurability. Which steps are active, and in what order, is defined entirely by a `DamageCalculationStrategy` asset that you set up in the Inspector — no scripting required.

Different entities can use different strategies: a boss might have its own pipeline featuring an ApplyDamageCapStep as its final step — limiting incoming damage to at most 10% of the boss's max HP — while regular enemies fall back to the project-wide default. Strategies can also be swapped at runtime, enabling mechanics such as temporary invincibility phases or damage rules that change mid-encounter.

Internally, the pipeline runs its steps sequentially on a shared state object and short-circuits the moment damage is marked as prevented: if a barrier fully absorbs the hit, or a modifier grants immunity, all remaining steps are skipped and the result is returned immediately.

Pipeline Data Types

The pipeline uses a small set of dedicated types to keep concerns cleanly separated — there is one type for what you request, one for what flows through the steps, and one for what you receive back. As a designer integrating `TakeDamage` into your game code, you will interact with two of these directly: `PreDamageContext` to describe the damage you want to apply, and `DamageResolutionContext` to inspect the outcome. The others are managed internally by the pipeline and are primarily relevant if you are implementing custom steps or hooking into damage events for analytics or reactions.

`PreDamageContext` is the descriptor you build before calling `TakeDamage`. Its mandatory fields are enforced by a fluent step-builder that requires them in order: amount → type → damage source → target. The dealer entity (`WithDealer`) is optional and can be omitted when no specific entity is responsible for the damage (e.g. traps, environmental hazards, or system-initiated damage). Other optional fields include `IsCritical`, `CriticalMultiplier`, `Ignore`, `Instigator`, and `IsReactable`. Setting `Ignore = true` causes the pipeline to be bypassed entirely; the damage is flagged as prevented with `PrePhaseIgnored` before any calculation begins. `IsReactable` defaults to `true` and signals whether reactive systems (such as counter-damage actions) may respond to this event — set it to `false` when building a secondary effect to prevent chain reactions. See Preventing Infinite Cycles for details.

`DamageInfo` is the mutable state object that flows through the pipeline. It is constructed from a `PreDamageContext` at the start of `TakeDamage` and holds:

- **`Amounts`** — a `DamageAmountContext` tracking the current damage value and step-by-step history.
- **`Type`**, **`DamageSourceSO`**, **`Target`**, **`Performer`**, **`IsCritical`**, **`CriticalMultiplier`** — metadata forwarded from the `PreDamageContext`.
- **`Reasons`** — a `DamagePreventionReason` flags enum accumulating all reasons why damage was prevented.
- **`IsPrevented`** — returns `true` when `Reasons` is not `None`; used to short-circuit the pipeline.

`DamageAmountContext` tracks the numerical amount throughout the pipeline:

- **`InitialAmount`** — the original raw value; never modified after construction.
- **`Current`** — the damage value as modified by each step; steps read and write this property.

- **`Records`** — a read-only list of `StepAmountRecord` entries, each capturing the step type and the before/after `Current` values for that step. Useful for debugging and diagnostics.

`DamageResolutionContext` is the value returned by `TakeDamage`. It contains:

- **`Outcome`** — `DamageOutcome.Applied` or `DamageOutcome.Prevented`.
- **`Reasons`** — the accumulated `DamagePreventionReason` flags when prevented; `None` when applied.
- **`TerminationStepType`** — the `Type` of the step that caused early termination, if any.
- **`FinalDamageInfo`** — the `DamageInfo` at the end of the pipeline; may be `null` when damage was prevented in the pre-phase.
- **`PreDamageContext`** — the original input that initiated this damage attempt.
- **`IsReactable`** — propagated from `PreDamageContext.IsReactable`. When `false`, reactive systems listening to damage resolution events should not respond to this outcome.

`DamageOutcome` is a two-value enum: `Applied` (damage was applied to the entity's health or barrier) and `Prevented` (damage was stopped before affecting the entity).

`DamagePreventionReason` is a flags enum that accumulates one or more reasons why damage was prevented:

Value	When set
`None`	No prevention; damage can proceed.
`EntityImmune`	Target has global immunity (`IsImmune = true` on `EntityHealth`).
`AllDamageImmune`	Generic percentage damage mitigation stat alone reached ≤ −100%.
`DamageTypeImmune`	Type-specific percentage modifier alone reached ≤ −100%.
`DamageSourceImmune`	Source-specific percentage modifier alone reached ≤ −100%.
`BarrierAbsorbed`	Barrier fully absorbed the incoming damage.
`DefenseAbsorbed`	Defensive stat computation reduced damage to zero.
`PrePhaseIgnored`	`PreDamageContext.Ignore` was `true` before the pipeline started.
`PrePhaseZeroAmount`	`PreDamageContext.Amount` was zero or negative.
`PipelineReducedToZero`	A step reduced damage to zero without a more specific absorption reason.
`EntityDead`	Target was already dead when `TakeDamage` was called.

> **NOTE**
>
> `DamagePreventionReason` is a flags enum: multiple values can be set simultaneously on the same damage attempt. Inspect `DamageResolutionContext.Reasons` to read all accumulated reasons after a call to `TakeDamage`.

Damage Step

Each station in the pipeline is a `DamageStep` — an independent, composable unit of work that applies one focused transformation to the damage in flight. The seven built-in steps cover the most common scenarios out of the box. If your game needs specialised behaviour — for example, a step that triggers a status effect when damage exceeds a certain threshold — you can implement a custom step by creating a class that inherits from `DamageStep` and implements `ProcessStep()`.

Step order matters. Each step receives the output of the one before it, so a different arrangement of the same steps produces a different result. Placing a critical multiplier step before defensive steps amplifies the pre-mitigation damage; placing it after scales a lower, post-mitigation value. We will see how to compose and reorder steps from the Inspector in the Damage Calculation Strategy section.

`DamageStep` is the abstract base class for all pipeline steps. Each step performs one focused transformation on `DamageInfo`. The pipeline runner calls `Process()` on each step in order; concrete implementations override `ProcessStep()`.

`Process()` enforces two preconditions automatically before delegating to `ProcessStep()`:

- If `DamageInfo.IsPrevented` is `true`, the step is skipped.
- If `DamageInfo.Amounts.Current` is ≤ 0, the step is skipped.

After `ProcessStep()` returns, `Process()` appends a `StepAmountRecord` to `DamageAmountContext.Records` with the before and after amounts. If the step brought a positive amount to zero without setting a more specific prevention reason, `DamagePreventionReason.PipelineReducedToZero` is set automatically.

The package includes seven built-in `DamageStep` implementations:

ApplyCriticalMultiplierStep

Critical hits deal amplified damage. This step multiplies the current damage amount by the critical hit multiplier when the incoming damage has been flagged as a critical strike. Its position in the pipeline is a design decision: placed before defensive steps, the multiplier scales raw (or modifier-adjusted) damage; placed after, it scales the already-mitigated result. Usually critical multipliers are applied early or as first step.

Scales the current damage by the critical hit multiplier when the damage is flagged as a critical hit. The step reads `DamageInfo.IsCritical` and `DamageInfo.CriticalMultiplier`. It is a no-op if `IsCritical` is `false`, or if the multiplier is ≤ 0 or exactly 1.0.

Both the critical flag and the multiplier are set in the `PreDamageContext` when building the damage request. A multiplier of 2.0 doubles the `Current` amount at the point where this step executes.

> **NOTE**
> The position of `ApplyCriticalMultiplierStep` in the pipeline determines what value the multiplier is applied to. Placing it before `ApplyDefenseStep` amplifies damage prior to defensive mitigation; placing it after scales post-defense damage. Design your strategy order accordingly.

> **NOTE**
> The multiplied amount is rounded to a `long` using the **Critical Damage Multiplier Rounding Mode** from `HealthRoundingSettings` (default: **Round**).

ApplyDefenseStep

This step applies the target entity's defensive stat for this damage type — the primary stat-based mitigation layer in a typical RPG setup. For example, for a Physical damage type with Armor as its **Defensive Stat**, this step reads the target's Armor value, optionally reduces it by any armor penetration, then feeds the result into the **Damage Mitigation Fn** to compute the mitigated damage. This is usually the step that eliminates most of the incoming damage for well-armored targets.

Applies defensive stat-based damage mitigation as configured in the `DamageTypeSO`. The step reads the **Defensive Stat** and **Damage Mitigation Fn**, and optionally the **Defense Penetration Stat** and **Defense Penetration Fn** (see Defense Penetration).

The step is a no-op when both **Defensive Stat** and **Damage Mitigation Fn** are unset — which is the intended configuration for a damage type that should never be mitigated by defenses. If the configuration is inconsistent (one field set, the other null), a warning is logged and the step is skipped. The effective defensive value is computed after applying any penetration reduction, then fed into the **Damage Mitigation Fn** to yield the final reduced amount. If the result is ≤ 0, `DamagePreventionReason.DefenseAbsorbed` is set.

> **NOTE**
>
> Two rounding modes from `HealthRoundingSettings` are applied within this step:
>
> - **Defense Penetration Rounding Mode** — rounds the effective defense value after the defense-penetration function reduces it (only when defense penetration is configured).
> - **Damage Mitigation Rounding Mode** — rounds the final damage amount returned by the damage-mitigation function.

ApplyBarrierStep

Barriers act as a temporary shield layer in front of an entity's health pool: incoming damage hits the barrier first, and only what the barrier cannot absorb continues on to the next step. This step is the mechanism behind shielding abilities and any temporary protective layer you implement with the barrier system. Its position in the pipeline determines how much barrier is actually consumed: placing it after `ApplyDefenseStep` means the barrier only ever sees damage that already survived defensive mitigation, so the barrier is drained more slowly. Placing it before defensive steps causes the barrier to absorb the full, unreduced hit — making it a purer shield but one that depletes faster.

Consumes the target entity's barrier (temporary shield) to reduce incoming damage before it reaches health. If the `DamageTypeSO` has **Ignore Barrier** enabled, this step is skipped entirely. If the target has no `EntityHealth` component or its barrier is zero, the step is also a no-op.

The step subtracts from `DamageInfo.Amounts.Current` the portion absorbed by the barrier, and reduces the entity's barrier by the same amount. If the barrier fully absorbs the hit, `DamagePreventionReason.BarrierAbsorbed` is set and the pipeline terminates. Damage that exceeds the barrier continues through subsequent steps as the updated `Current` value.

> **NOTE**
>
> `ApplyBarrierStep` does not reduce health directly. Any remaining damage after barrier absorption continues through the pipeline and is applied to health at the end of `TakeDamage`.

ApplyPercentageDmgModifiersStep

Percentage modifiers scale the incoming damage by a fraction. The three modifier layers (generic, source-specific, type-specific) each correspond to a stat whose value the target entity holds at runtime. A negative value reduces damage, a positive value amplifies it. When a single modifier layer reaches −100% or below on its own, the entity becomes immune to that entire category of damage for this hit.

Applies percentage-based damage modifiers from up to three layers, in order:

1. **Generic** — reads `AstraHealthConfigSO.GenericPercentageDamageModificationStat` from the target's stats. Skipped if the `DamageTypeSO` has **Ignore Generic Percentage Modifiers** enabled.
2. **DamageSource-specific** — reads `DamageSourceSO.PercentageDamageModificationStat` from the target's stats.
3. **DamageType-specific** — reads `DamageTypeSO.PercentageDamageModificationStat` from the target's stats.

Each layer is evaluated individually for full immunity before contributions are summed. If the generic layer alone reaches ≤ −100%, `DamagePreventionReason.AllDamageImmune` is set and the step exits immediately. If the source-specific layer alone reaches ≤ −100%, `DamagePreventionReason.DamageSourceImmune` is set. If the type-specific layer alone reaches ≤ −100%, `DamagePreventionReason.DamageTypeImmune` is set. If no single layer triggers immunity, the three contributions are summed additively into a single net percentage applied to `Current` in one operation.

> **NOTE**
>
> If the cumulative percentage is ≤ −100%, `DamageInfo.Amounts.Current` clamps to a minimum of 0: percentage modifiers cannot bring damage below zero. In such case, the damage is prevented with `DamagePreventionReason.PipelineReducedToZero`.

> **NOTE**
>
> The scaled amount is rounded to a `long` using the **Percentage Damage Modifier Rounding Mode** from `HealthRoundingSettings` (default: **Round**).

ApplyFlatDmgModifiersStep

Where percentage modifiers scale damage proportionally, flat modifiers add or subtract a fixed amount regardless of the hit's magnitude. A −15 flat absorb always removes 15 damage, whether the incoming hit was 20 or 2000. Flat modifiers are typically placed after percentage modifiers so they adjust the already-scaled value — but you are free to order them as your game design requires.

Applies flat damage modifiers from the same three layers as `ApplyPercentageDmgModifiersStep`:

1. **Generic** — reads `AstraHealthConfigSO.GenericFlatDamageModificationStat` from the target's stats. Skipped if the `DamageTypeSO` has **Ignore Generic Flat Modifiers** enabled.
2. **DamageSource-specific** — reads `DamageSourceSO.FlatDamageModificationStat` from the target's stats.
3. **DamageType-specific** — reads `DamageTypeSO.FlatDamageModificationStat` from the target's stats.

All three contributions are summed additively. The net value is added to or subtracted from `DamageInfo.Amounts.Current`.

> **NOTE**
>
> `DamageInfo.Amounts.Current` clamps to a minimum of 0: flat modifiers cannot bring damage below zero. In such case, the damage is prevented with `DamagePreventionReason.PipelineReducedToZero`.

> **NOTE**
>
> The conventional order is to run percentage modifiers before flat modifiers, so that flat additions or reductions are applied to the already-scaled value. This ordering is not enforced by the system; you can arrange steps freely.

ApplyDamageCapStep

Enforces an upper bound on the damage an entity can receive from a single hit. Place this step wherever your design requires — typically at the end of the pipeline — to prevent any single attack from dealing more damage than the configured limit, regardless of multipliers, modifiers, or critical hits that may have amplified the amount.

The step looks for a component implementing `IDamageCap` on the target entity (see Damage Bound Components). If no such component is present, the step is a no-op. When a provider is found, the step clamps `DamageAmountContext.Current` to at most the value returned by `IDamageCap.GetDamageCap()`. A cap value of 0 or below is treated as a misconfiguration and ignored.

> **NOTE**
>
> If the cap reduces `Current` to exactly 0, `DamagePreventionReason.PipelineReducedToZero` is set automatically by the base class.

ApplyDamageFloorStep

Guarantees a minimum damage an entity receives from a single hit, provided the damage has not already been fully prevented by an earlier step. Use this to implement mechanics such as "this hit always deals at least N damage" — ensuring that heavy mitigation from armor, barriers, or modifiers never reduces the effective hit below a design-defined baseline.

The step looks for a component implementing `IDamageFloor` on the target entity (see Damage Bound Components). If no such component is present, the step is a no-op. When a provider is found and `Current` is below the floor, the step raises `Current` to that floor value. A floor value of 0 or below is treated as a misconfiguration and ignored.

> **IMPORTANT**
>
> `DamageStep.Process()` skips any step when `Current` is already ≤ 0 or the damage is already prevented. `ApplyDamageFloorStep` therefore cannot restore damage that was fully absorbed by an earlier step (for example, by a barrier that brought `Current` to zero). To guarantee a minimum amount even after heavy mitigation, place this step **before** the reducing steps in the pipeline.

Damage Bound Components

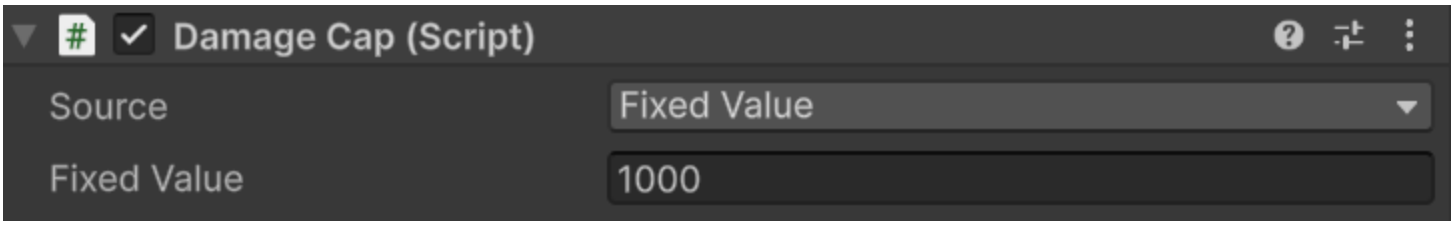
`ApplyDamageCapStep` and `ApplyDamageFloorStep` are pure logic steps — they do not hold the bound value themselves. The value is provided at runtime by a component attached to the target entity that implements one of two interfaces: `IDamageCap` (upper bound) or `IDamageFloor` (lower bound). The package ships a concrete `MonoBehaviour` implementation for each, but the interfaces are also the extension point if you want to provide a custom bound from your own component.

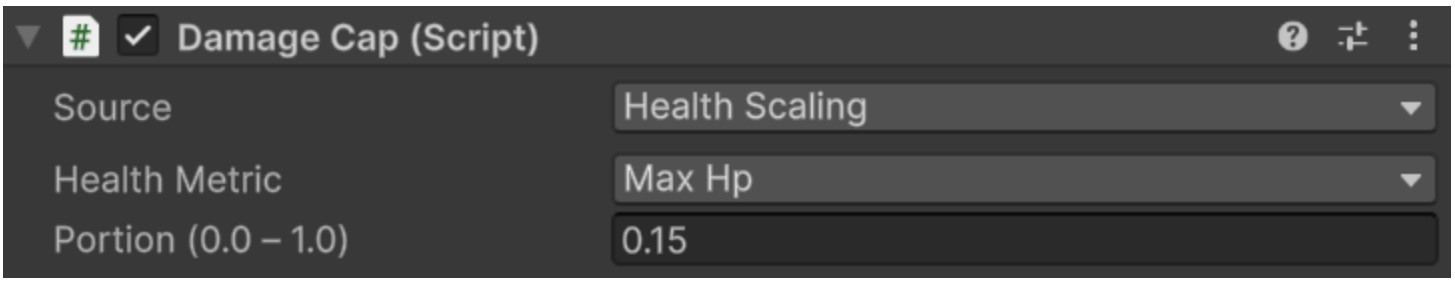
- **`DamageCap`** — implements `IDamageCap`. Attach it alongside `EntityHealth` on an entity to activate an upper bound on incoming damage. `ApplyDamageCapStep` queries it via `GetDamageCap()`.
- **`DamageFloor`** — implements `IDamageFloor`. Attach it alongside `EntityHealth` on an entity to activate a lower bound on incoming damage. `ApplyDamageFloorStep` queries it via `GetDamageFloor()`.

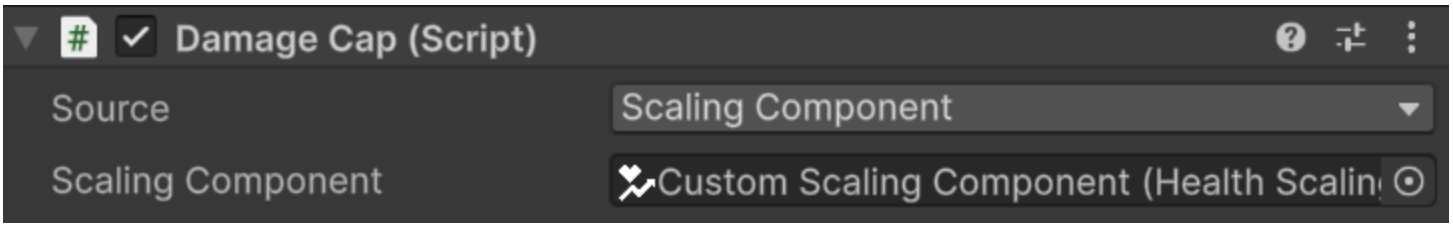
Both components share the same configuration structure. The **Source** field (a `DamageBoundSource` enum) selects how the bound value is computed at runtime:

- **Fixed Value** — a constant `long` configured directly on the component (e.g., "never take more than 200 damage per hit").
- **Health Scaling** — a fraction of a chosen health metric at the moment the step executes. Two inline fields control the result: **Health Metric** (Current HP, Max HP, or Missing HP) and **Portion** (0.0 to 1.0). Setting Health Metric to **Max HP** and Portion to **0.1**, for instance, yields a cap or floor equal to 10% of the entity's max HP — evaluated dynamically each time a hit is processed. This mode effectively inlines the same idea exposed by `HealthScalingComponentSO`, but without requiring a separate asset.
- **Scaling Component** — delegates the calculation to any `ScalingComponent` ScriptableObject asset. Useful when the bound logic is already captured in a reusable scaling asset, or when it depends on values beyond the entity's health pool. Any `ScalingComponent` subtype is valid, including `HealthScalingComponentSO`.

The inspector adapts to the selected source, showing only the fields relevant to the current mode:







`DamageFloor` has an identical inspector layout; only the semantic meaning of the value changes.

> **WARNING**
>
> Both `DamageCap` and `DamageFloor` require an `EntityCore` component on the same `GameObject`. The **Health Scaling** source additionally requires an `EntityHealth` component on the same entity. If `EntityHealth` is missing when Health Scaling is selected, a warning is logged at runtime and the bound evaluates to 0, effectively disabling it for that hit.
>
> The **Scaling Component** source also silently becomes ineffective if no asset is assigned: the component returns 0, and the corresponding cap or floor is ignored for that hit. The custom editor surfaces this with a warning box in the Inspector.

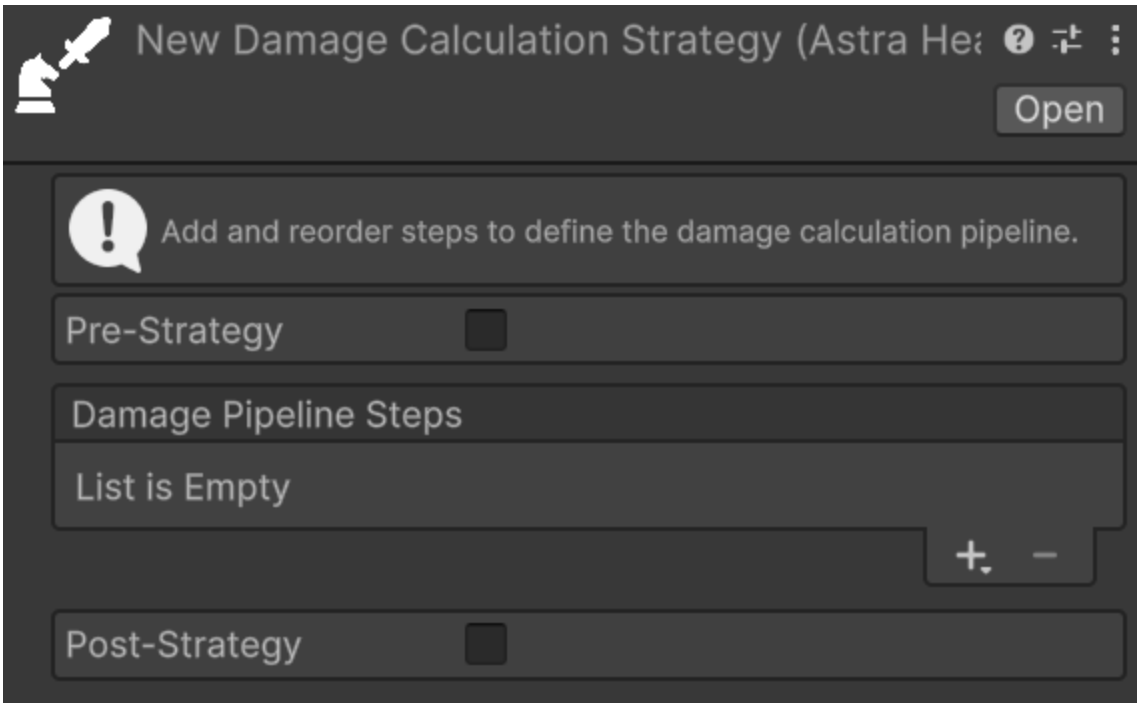
Damage Calculation Strategy

Relative path: `Damage Calculation Strategy`

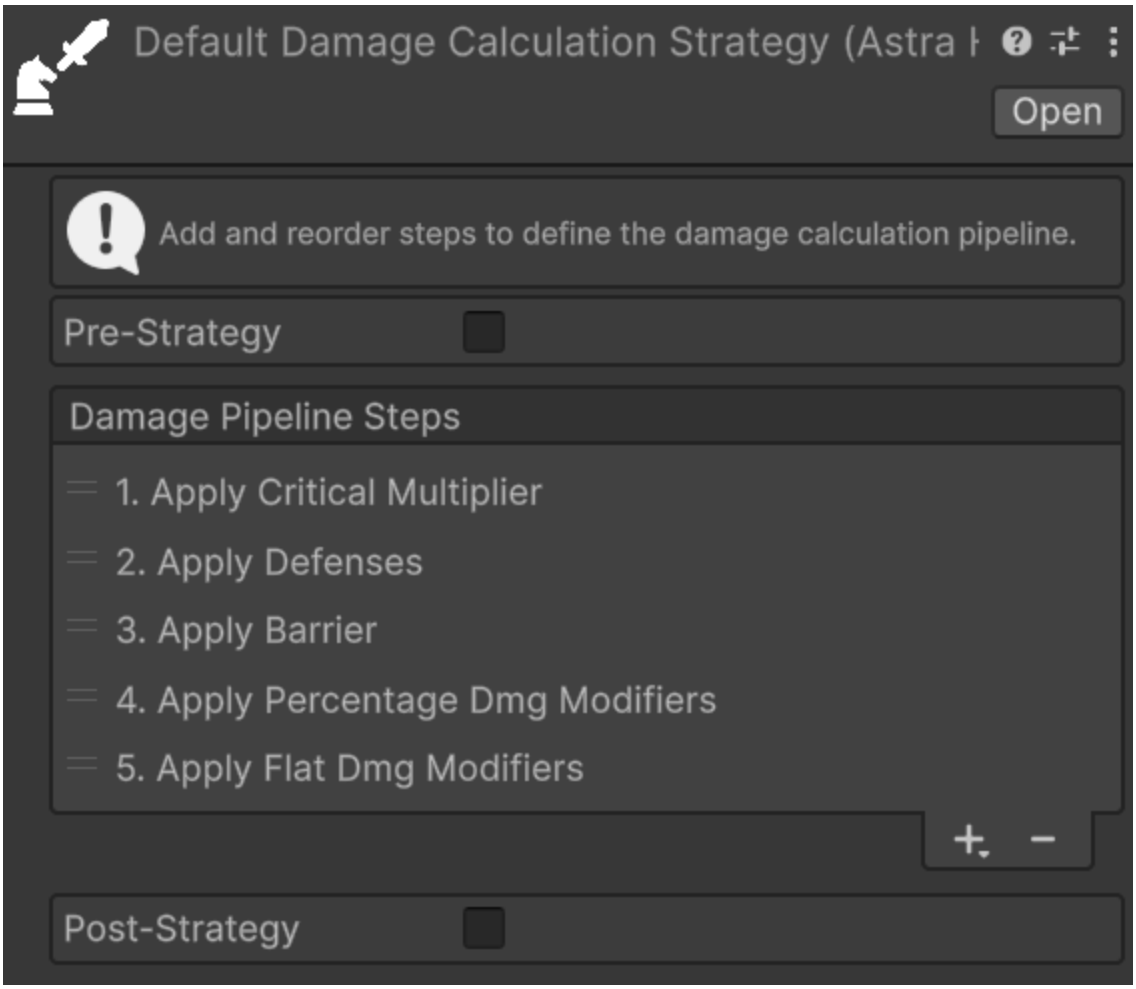
Rather than embedding damage calculation logic in scripts, Astra Health externalises the pipeline into a `DamageCalculationStrategy` asset — a `ScriptableObject` you control entirely from the Inspector. Each strategy holds an ordered list of steps; changing the list changes the pipeline. This separation lets you design, tune, and swap calculation behaviours without touching any code. A boss encounter can use a custom strategy with a specific subset of steps (e.g., an ApplyDamageCapStep as its final step that limits the incoming damage to a maximum of 10% of the Max HP); regular enemies fall back to the project-

wide default; a runtime debuff can temporarily replace an entity's strategy to alter how it takes damage during a special encounter phase (e.g., all lightning damage is doubled).

In the Inspector, the strategy exposes a reorderable list labelled **Damage Pipeline Steps**. Steps can be added via a type-selection dropdown (the "Step" suffix is trimmed from display names for readability), removed, and reordered via drag. A newly created, empty strategy looks like this:



Once steps are added and arranged, the Inspector shows them as a numbered list reflecting the execution order:



Every `EntityHealth` component resolves the active strategy at runtime through a **three-tier priority**:

1. **Override Damage Calculation Strategy** — takes precedence over all other settings. Intended for temporary runtime effects (e.g., a debuff that temporarily changes how an entity takes damage) or for testing when a custom strategy is already assigned.
2. **Custom Damage Calculation Strategy** — an entity-specific strategy that overrides the global default but can itself be overridden at runtime by the tier above.
3. **Default Damage Calculation Strategy** — configured in `AstraHealthConfigSO` and applied to every entity that defines neither a custom nor an override strategy.

> **IMPORTANT**
>
> If no strategy is resolved — neither custom nor override is set on the entity, and no default is configured in `AstraHealthConfigSO` — `TakeDamage` logs an error and the pipeline does not run.

Extending a Strategy Without Duplication

A common pattern is building a specialised pipeline by extending the project-wide default. For example, a boss entity might need all the same mitigation steps as regular enemies, plus one extra capping step.

Maintaining two separate assets with identical steps creates a maintenance burden: every change to the default pipeline must be manually replicated in every dependent asset.

To address this, each `DamageCalculationStrategySO` exposes a **Pre-Strategy** section above the step list and a **Post-Strategy** section below it. When enabled, each section executes an entire other strategy before or after the steps defined on the current asset, without duplicating any step.

Both sections share the same layout:

- **Enabled** toggle — when enabled, the strategy in this section will run before (for Pre) or after (for Post) the steps defined on this asset.
- **Use Default Pipeline** checkbox — when checked, the global default `DamageCalculationStrategy` configured in `AstraHealthConfigSO` is used. When unchecked, a **Custom Strategy** drag-and-drop field appears to assign any other `DamageCalculationStrategySO` asset explicitly.

A typical example: to create a boss pipeline that applies all standard mitigation and then limits incoming damage to 10% of the boss's max HP, configure the strategy asset as follows:

- **Pre-Strategy**: enabled → **Use Default Pipeline** checked (runs the project-wide default pipeline first).
- **Own steps**: `ApplyDamageCapStep` (with a `DamageCap` component on the boss, **Source** set to Health Scaling, Max HP, 0.1).
- **Post-Strategy**: disabled.

With this setup, any change to the default pipeline is automatically picked up by the boss strategy — no duplication required.

> **CAUTION**
>
> The system includes a recursion guard that prevents a strategy from executing itself directly or indirectly. If a cycle is detected at runtime — for example, Strategy A uses Strategy B as its post-strategy, and Strategy B uses Strategy A as its pre-strategy — a warning is logged and the repeated execution is skipped. Design strategy chains as directed acyclic graphs to avoid this.

Healing

An entity can be healed in 4 different ways:

- Direct healing through the `Heal` method.
- Health regeneration. This can be passive, through the dedicated `EntityPassiveHpRegeneration` component plus the Health Regeneration configuration, or it can be manually activated via the `ManualHealthRegenerationTick` method. Also for the latter, you must configure the statistic to consider for the calculation of healing through the configuration.
- Through lifesteal, that is healing the entity based on the damage dealt. However, lifesteal functioning is detailed in Lifesteal as it deserves a dedicated section.
- Via resurrection with the two `Resurrect` methods. One to resurrect the entity with a percentage of HP, and one to resurrect it with a fixed amount of health points. Applicable only if the entity is dead. Also resurrection is detailed in a dedicated section, Resurrection.

> **NOTE**
>
> Regeneration (both passive and manual), lifesteal, and resurrection all use, behind the scenes, the `Heal` method to effectively heal the entity.

We will first introduce some concepts that are common to all the four healing methods, and then we will analyze direct healing and health regeneration. As aforesaid, lifesteal and resurrection are analyzed in their respective sections, Lifesteal and Resurrection.

Heal Source

Relative path: `Heal Source`

A `HealSourceSO` represents the source of healing. Some examples of `HealSourceSO` could be: healing potions, skills, passive regeneration, lifesteal, system (e.g., the player is healed to max health during the tutorial by a support machine), and so on. The `HealSourceSO` is an important concept as it allows distinguishing between different sources of healing, and thus applying specific healing modifiers for each healing source, in addition to the generic healing modifiers that apply to all healing sources. Furthermore, `HealSources` are important because they are communicated in healing contexts, and therefore in the listeners of healing events, allowing the latter to react specifically based on the healing source that triggered the event. Thanks to this, you could, for example, implement a buff that increases the healing received from healing potions. Finally, `HealSources` pave the way for the implementation of a combat log that tracks not only the amount of healing received but also the healing source, allowing the player to have a deeper understanding of what healed them most effectively in a specific game situation.

An instance of `HealSourceSO` in the inspector looks like this:

You can notice that there are two properties to configure for each `HealSourceSO`:

- **Flat Heal Modification Stat**: the statistic to consider in an entity to apply flat, specific healing modifiers for this healing source. Positive values of this statistic increase the healing received from this source, while negative values decrease it. If the entity does not have this statistic, a value of 0 will be considered for this statistic, and therefore no specific flat healing modifier will be applied for this healing source.
- **Percentage Heal Modification Stat**: the statistic to consider in an entity to apply percentage, specific healing modifiers for this healing source. Positive values of this statistic increase the healing received from this source, while negative values decrease it. If the entity does not have this statistic, a value of 0 will be considered for this statistic, and therefore no specific percentage healing modifier will be applied for this healing source.

Heal Modifiers

Heal modifiers are statistics that influence the amount of healing received by an entity. They can influence it positively or negatively, depending on the value. There are two types of healing modifiers: generic heal modifiers, which apply to all healing sources, and source-specific heal modifiers, which apply only to a specific healing source. Furthermore, healing modifiers can be flat or percentage-based. Flat healing modifiers influence the amount of healing received by adding or subtracting a fixed amount of HP to the healing, while percentage healing modifiers influence the amount of healing received by multiplying the healing by a certain factor. It is important to note that flat healing modifiers are applied before percentage healing modifiers. Therefore, percentage modifications will also affect the impact of flat healing modifiers.
For percentage modifiers, a statistic with a value of 120 increases the healing by 120%, while a statistic with a value of -25 decreases the healing by 25%.

Generic healing modifiers are summed with the source-specific ones: flat modifiers are added together, as are the percentage ones.

Finally, if an entity does not possess the statistic associated with a healing modifier in its Stat Set, a value of 0 will be considered for that statistic, and therefore no healing modifier will be applied for that

statistic.

Clearly, to provide healing modifiers to entities, since they are statistics, you must proceed through the Stat Modifiers APIs, as described in the Understanding Stat Modifier Types section of the base framework documentation.

Example of Heal Modifiers

Suppose we want to calculate the healing received by an entity using a healing potion. Here are the example values:

Definition of the statistics used in the example:

- **Potion Heal Flat Modifier**: Source-specific statistic (the potion) that adds or subtracts a fixed value to the HP healed by the potion.
- **Potion Heal Percentage Modifier**: Source-specific statistic (the potion) that increases or decreases the potion's healing by a percentage.
- **Generic Flat Heal Modifier**: Generic statistic that applies to all healing sources, adding or subtracting a fixed value to the HP healed.
- **Generic Percentage Heal Modifier**: Generic statistic that applies to all healing sources, increasing or decreasing the healing by a percentage.

Phase	Value	Calculation	Result
Base healing	80 HP	-	80 HP
Flat modifiers	Potion Heal Flat Modifier: 50 Generic Flat Heal Modifier: -10	50 - 10 = 40 HP 80 + 40	120 HP
Percentage modifiers	Potion Heal Percentage Modifier: 20% Generic Percentage Heal Modifier: 10%	20% + 10% = 30% 120 × 1.3	156 HP

Final healing received: 156 HP

This table clearly shows how flat and percentage modifiers are summed, and how the final result is reached.

Direct Healing

The `Heal` method is the most straightforward way to heal an entity. It takes a `PreHealContext` as input, which contains all relevant information about the healing you intend to apply and operates as follows:

1. It checks if the entity is dead. If the entity is dead, will raise an exception, as you cannot heal a dead entity. If you want to resurrect a dead entity, check the Resurrection section.

2. It checks if the healing to be applied is marked as a critical hit and, if so it applies the critical multiplier to the healing amount.
3. It retrieves the generic and `HealSourceSO`-specific **flat** heal modifiers for the entity.
4. It retrieves the generic and `HealSourceSO`-specific **percentage** heal modifiers for the entity.
5. It retrieves the heal flat modifiers specific to the heal source, and sums them with the generic flat heal modifiers.
6. It retrieves the heal percentage modifiers specific to the heal source, and sums them with the generic percentage heal modifiers.
7. It applies the flat modifiers to the base healing amount, and then applies the percentage modifiers to the result, to calculate the final healing amount.
8. It increases the entity's current HP by the final healing amount.
9. It raises the *Entity Healed Event* with a `HealResolutionContext` containing all relevant information about the healing that was just applied.
10. It returns the `HealResolutionContext` created in the previous step so that the caller can use it for further processing if needed.

The `Heal` method, differently from the `TakeDamage` method, cannot be configured to reorder the steps of the healing pipeline. This choice was made as healing is generally more straightforward than damage, and there are usually no mechanics that require reordering the steps of the healing pipeline. Therefore, I evaluated simplicity and ease of use more important than flexibility for this method.

`Heal` Usage Example

Suppose we want to heal an entity for 20% of its total max HP. The code could be the following:

```
// Assuming that:
// - healSource is a HealSource representing the healing coming from a skill
// - skillCaster is the EntityCore that casts the healing skill
// - target is the EntityCore that we want to heal

if (target.TryGetComponent(out EntityHealth targetHealth)) {
    // in case the target has a EntityHealth component...
    long healAmount = target.GetMaxHpPortion(0.2d);

    PreHealContext preHealContext = PreHealContext.Builder
        .WithAmount(healAmount)
        .WithSource(healSource)
        .WithTarget(target.EntityCore)
        .WithHealer(skillCaster)       // optional
        .Build();

    targetHealth.Heal(preHealContext);
}
```

It is not a good practice to hardcode the healing amount directly in code (20% of max hp in the example). Hardcoded values make tuning and balancing difficult, prevent reuse across different skills or items, and force code changes and recompilation for values that designers or content creators should be able to tweak. Prefer one of these approaches instead:

- **Serialize the amount** as an inspector-editable field so designers can tweak values without touching code.
- **Use a `ScalingFormula`** (or equivalent) to compute the heal amount dynamically from the caster's or target's stats, allowing the effect to scale with progression and remain data-driven.

Example using a `ScalingFormula` to calculate the heal amount at runtime:

```
// Assuming that scalingFormula is a ScalingFormula that calculates the heal amount
// based on the skill caster's stats...
long healAmount = scalingFormula.CalculateValue(skillCaster);

PreHealContext preHealContext = PreHealContext.Builder
    .WithAmount(healAmount)
    .WithSource(healSource)
    .WithTarget(target.EntityCore)
    .WithHealer(skillCaster)       // optional
    .Build();

targetHealth.Heal(preHealContext);
```

Recommended pattern when healing programmatically:

1. Construct a `PreHealContext` with all relevant information using the fluent builder.
2. Call `Heal` passing the constructed context.

The `PreHealContext` fluent builder is helpful because it guides you through required inputs first — the IDE will typically present required builder steps one at a time. When the builder presents multiple fields together, those fields are optional and can be omitted. Optional fields include the healer entity (`WithHealer`), the critical hit flag, the critical multiplier, the instigator (`WithInstigator`), and the reactability flag (`WithIsReactable`). `IsReactable` defaults to `true`; set it to `false` when building a secondary heal that reacts to another event, to prevent chain reactions. See Preventing Infinite Cycles for details.

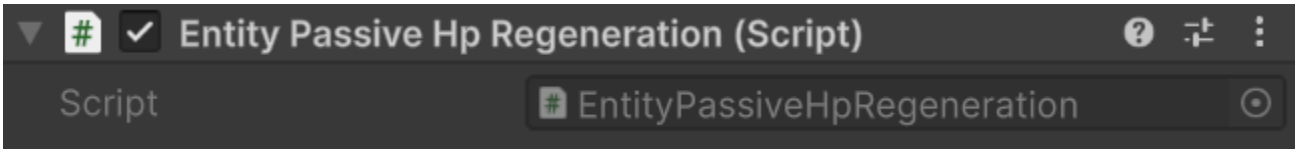
Health Regeneration

Health regeneration is a mechanism that allows an entity to passively regenerate its health over time. In games generally, this mechanic is implemented through discrete regeneration ticks. Ticks, depending on the game, can be marked by the passage of time, or by certain events, like the end of a turn in a turn-

based game.
This package supports both passive and manual regeneration.

Passive Health Regeneration

Passive regeneration is handled by the dedicated `EntityPassiveHpRegeneration` MonoBehaviour. Attach `EntityPassiveHpRegeneration` alongside `EntityHealth` on every entity that should regenerate automatically over time.



The component reads its automatic-tick settings from the Health Regeneration section of the package configuration. The following parameters should be configured there:

- **Health Regeneration Source**: the `HealSourceSO` to associate with regeneration (passive and manual). The package will use this `HealSourceSO` to create the healing contexts to pass to the `Heal` method when it's time to regenerate health.
- **Passive Health Regeneration Stat (HP/10s)**: the amount of HP the entity regenerates every 10 seconds. The package will take care of scaling this amount based on the time that passes between consecutive ticks, to ensure smooth and consistent regeneration.
- **Passive Health Regeneration Interval**: the time interval, in seconds, between one regeneration tick and the next.

Configuring these values alone is not enough: only entities that also have `EntityPassiveHpRegeneration` attached will receive automatic regeneration ticks.

To give an example, if we want an entity to regenerate 5 HP every 0.5 seconds, we must configure the "Passive Health Regeneration Interval" parameter to 0.5, ensure the entity has `EntityPassiveHpRegeneration` attached, and set the statistic associated with "Passive Health Regeneration Stat (HP/10s)" to a value of 100 (5 HP every 0.5 seconds correspond to 10 HP per second, which correspond to **100 HP every 10 seconds**).

Performance Considerations

Before making any considerations about performance, I want to remind you that premature optimization is the root of all evil. Before taking actions to optimize performance, a real performance problem should be identified through profiling. That said, since by default passive regeneration raises the `Entity Healed` Event Channel, which in turn raises the global `Entity Healed` event and any extra `Entity Healed` events assigned to that entity, it's important to consider the performance impact if you have a large number of entities with `EntityPassiveHpRegeneration` regenerating health simultaneously with a very short regeneration interval. If there were also several listeners subscribed to the global `Entity Healed` event,

you would get several function calls at each regeneration tick X each entity. If this becomes a performance issue for your game, you have several levers available to optimize performance:

- **Increase the regeneration interval**: by increasing the regeneration interval, the number of regeneration ticks per time unit is reduced, and therefore the number of Entity Healed Events raised.
- **Disable the raising of Entity Healed Events for passive regeneration**: if you don't need to react to passive regeneration through the listeners of Entity Healed Events, you can disable the raising of these events for passive regeneration through the configuration. This way, even if you have a large number of entities regenerating health simultaneously with a very short regeneration interval, you won't have a significant performance impact due to raised events and associated function calls.
- **Use Extra Entity Healed Events**: if you only need to react to the passive regeneration of one or some specific entities, you can configure these entities with a dedicated extra `Entity Healed` event on their `Entity Healed` Event Channel, and subscribe your listeners to this extra event instead of the global `Entity Healed` event. This way, the only function calls you'll have at each tick will be those of the listeners subscribed to the specific extra event of the entities configured to raise it, thus reducing the performance impact.

I would like to reiterate that, unless you have identified a real performance problem through profiling, it is not necessary to take any of these actions to optimize performance. You probably won't have any performance issues even with a thousand entities and dozens of listeners subscribed to the global Entity Healed Event.

Manual Health Regeneration

Manual regeneration is instead triggered manually through the `ManualHealthRegenerationTick()` API method.

This method, when called, triggers a regeneration tick for the entity. Note that the statistic considered for the health calculation does NOT coincide with the one configured for passive regeneration, but is a separate statistic, also set via the package configuration, specifically: Manual Health Regeneration Stat. The value of this statistic determines the amount of HP regenerated at each manual regeneration tick. Obviously, healing modifiers, both generic and those specific to the heal source configured for regeneration, will also affect manual regeneration, just like passive regeneration.

Passive vs Manual Health Regeneration: Which One Should I Use?

If you have doubts about which type of regeneration to use, here are some guidelines that might help you choose:

- **Real Time**: if your game is in real-time, passive regeneration is generally more suitable, as it integrates better with continuous gameplay flow.

- **Turn-Based**: if your game is turn-based, manual regeneration is generally more suitable, as you can trigger it at the end of each turn, ensuring more precise control over the game flow.
- **Regeneration in the base**: if your game involves a regeneration mechanic that only happens when the character is in a specific location (e.g., a base), you can use passive regeneration, but enable `EntityPassiveHpRegeneration` only when the character is in that location, and disable it when they leave. If, on the other hand, you want it to be active even when the character is outside the base, but still want it to be stronger when in the base, you can assign a specific Heal Modifier for the regeneration's Heal Source when the character is in the base, to increase the amount of HP regenerated when the character is at the base. Upon leaving the base, you can remove this Heal Modifier to reduce the amount of HP regenerated.
- **Regeneration out of combat**: if your game features regeneration that only happens when the character is out of combat, you can use passive regeneration, enabling `EntityPassiveHpRegeneration` when out of combat, and disabling it when entering combat. Or, if you want it to be active during combat as well, but stronger out of combat, you can (as with base regeneration) add a specific Heal Modifier for the regeneration's Heal Source when the character is out of combat.
- **Regeneration as a buff**: if the entity receives a buff providing HP regeneration according to a specific timing, you can use manual regeneration to trigger regeneration ticks perfectly aligned with the timing intended by the buff.

Clearly, these are just guidelines, not strict rules. For every problem, there are always several possible solutions, and choosing the best one always depends on the specific context of your game and the mechanics you want to implement. So, if you feel a solution different from the suggested ones fits your game better, feel free to use it.

Lifesteal

Lifesteal is a mechanic that lets an entity recover health proportional to the damage it deals. When an entity successfully damages a target, a percentage of the damage — determined by a configurable stat — is healed back to the attacker.

The system supports **two lifesteal layers**:

1. **Generic Lifesteal** — configured once in `AstraHealthConfigSO` and applied to all damage dealt by the entity.
2. **Damage-Type Lifesteal** — configured directly inside each `DamageTypeSO` and applied only when that specific damage type is dealt.

Both layers use the same `LifestealStatConfig` structure, so designers configure the same three concepts in both places: the lifesteal stat, the heal source, and the damage amount selector. If both layers are configured for the same hit, their heal amounts stack. Depending on the `Unify Lifesteal Heals` flag in `AstraHealthConfigSO`, those stacked amounts can be applied either as two separate heals or as a single unified heal.

LifestealStatConfig

`LifestealStatConfig` is the reusable data structure used by both **Generic Lifesteal** and the per-`DamageTypeSO` **Lifesteal** section.

Lifesteal Stat
The stat that drives the lifesteal percentage. At runtime, the attacker reads this stat value from its own `StatSet` and computes the heal as:

> heal amount = basis damage × (lifesteal stat value / 100)

The stat value is read as a `Percentage` type, meaning the raw `long` value stored in the stat is divided by 100 internally. A stat value of `15` therefore corresponds to 15% lifesteal. For lifesteal to activate, this stat must be present in the dealing entity's `StatSet`.

> **NOTE**
> The computed heal amount is rounded to a `long` using the **Lifesteal Rounding Mode** from `HealthRoundingSettings` (default: **Round**). This rounding is applied independently to each lifesteal contribution (generic and damage-type) before they are summed or applied as separate heals.

Lifesteal Source
The `HealSourceSO` used when applying the lifesteal heal. As with any heal in the framework, the heal source determines which heal modifiers — flat and percentage — are applied to the resulting heal, so lifesteal heals can be boosted or reduced by modifiers associated with the configured `HealSourceSO`. See Heal Source for details on `HealSourceSO` configuration.

Amount Selector
Determines which damage value is used as the basis for the lifesteal computation. See Amount Selector below.

Generic Lifesteal

`AstraHealthConfigSO` exposes a **Generic Lifesteal** field of type `LifestealStatConfig`.

If its **Lifesteal Stat** is configured and the damage dealer has that stat in its `StatSet`, every successful hit can generate lifesteal from this generic configuration regardless of the hit's `DamageTypeSO`.

Use this when you want a broad mechanic such as "all outgoing damage grants lifesteal", without repeating the same setup on every damage type.

See also: Lifesteal in Package Configuration.

Damage-Type Lifesteal

Each `DamageTypeSO` contains its own **Lifesteal** field, also of type `LifestealStatConfig`.

This contribution is evaluated only when the resolved hit uses that `DamageTypeSO`. It stacks with **Generic Lifesteal**, making it possible to define a baseline lifesteal that applies to all damage plus an additional bonus or alternate source for specific damage types.

Use this when lifesteal should depend on the nature of the damage — for example, only physical hits lifesteal, or fire damage lifesteal should use a dedicated `HealSourceSO`.

See also: Damage Type's Lifesteal.

Amount Selector

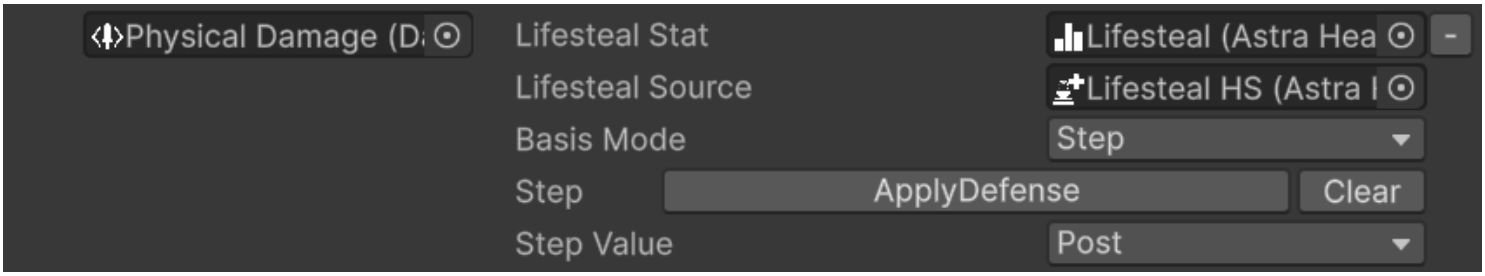
The Amount Selector controls which damage value is sampled as the lifesteal basis. Three modes are available:

- **Final** *(default)*: uses the damage value after all pipeline steps — the damage actually applied to the target. This is the most intuitive mode for a straightforward "heal for a percentage of damage dealt" mechanic.
- **Initial**: uses the raw damage value before any pipeline modifications. Useful when you want lifesteal to reflect the attacker's power output regardless of the target's defenses — for example, a vampire's

lifesteal that scales with their attack power rather than with how much the target's armor reduced.

- **Step**: samples the damage value at a specific pipeline step. You select a step and whether to use the value recorded before (`Pre`) or after (`Post`) that step executes, giving designers precise control over which phase of the calculation drives the heal. Refer to Damage Calculation Pipeline for an overview of the available pipeline steps.

The following image shows the inspector when Step mode is selected:



> **NOTE**
>
> If **Step** mode is selected but no step is configured, the system falls back to **Final** damage. The inspector displays a warning to indicate this condition.

Separate vs Unified Heals

When both **Generic Lifesteal** and **Damage-Type Lifesteal** contribute to the same hit, the framework can apply them in two different ways. By default, `Unify Lifesteal Heals` is enabled, so the two contributions are merged into a single heal.

Separate Heals

When `Unify Lifesteal Heals` is **disabled**, each contribution produces its own heal:

- the generic contribution uses the generic `Lifesteal Source`
- the damage-type contribution uses the damage type's `Lifesteal Source`

This is the right choice when those two heal sources should remain distinguishable for heal modifiers, event listeners, combat logs, analytics, or VFX/UI feedback.

Unified Heal

When `Unify Lifesteal Heals` is **enabled** *(default)*, the generic and damage-type lifesteal amounts are summed and applied as a **single heal**.

In this mode, the `HealSourceSO` is resolved as follows:

- use the damage type's `Lifesteal Source` if the damage-type contribution is active and that source is configured
- otherwise fall back to the generic `Lifesteal Source`

This is useful when generic lifesteal is meant to be an invisible baseline bonus and you want the final heal to behave as a single gameplay event.

Performance Considerations

> **NOTE**
>
> Before making any considerations about performance, keep in mind that premature optimization is the root of all evil. A real performance problem should be identified through profiling before taking any action.

By default, lifesteal heals raise the standard healing events (`Pre Heal` and `Entity Healed`). In games where many entities deal lifesteal damage frequently — for example, a horde of enemies all with a lifesteal stat, each landing hits several times per second — the volume of heal events generated can become significant, especially when the global `Entity Healed Event` has multiple listeners.

If profiling reveals this to be a bottleneck, the **Suppress Lifesteal Events** flag in `AstraHealthConfigSO` disables healing event emission for lifesteal heals entirely. The heal is still applied; only the events are suppressed. This option is appropriate when you do not need to react to lifesteal heals through event listeners — for example, when your UI or combat log does not track lifesteal healing specifically.

A similar consideration applies to passive health regeneration, which can also generate a high volume of heal events under certain conditions. See Performance Considerations in the Healing documentation for a broader discussion.

Conditions

NOTE

Lifesteal is evaluated after every damage application. The following conditions must all be met for it to trigger:

- The dealing entity is the source of the damage. Lifesteal does not trigger on damage dealt by others.
- The dealing entity is alive at the moment of damage resolution.
- At least one lifesteal layer is configured for the hit:
  - **Generic Lifesteal** in `AstraHealthConfigSO`, and/or
  - **Lifesteal** on the hit's `DamageTypeSO`.
- For each configured layer, the corresponding **Lifesteal Stat** is present in the dealing entity's `StatSet`.
- The computed heal amount for that layer is greater than zero. A lifesteal stat of 0% produces no heal contribution.

IMPORTANT

Lifesteal relies on the **Global Damage Resolution Event** being correctly configured on each entity's `EntityHealth`. This event is used by the framework to propagate damage outcomes across all listening entities — including the attacker, which uses it to detect its own successful hits and trigger lifesteal. Assigning a non-global event to this slot will break lifesteal and other framework features that depend on centralized damage communication. See Global Events for setup details.

Health Scaling Component

Relative path: `Scaling -> Health Scaling Component`
Base framework convenience relative path: `Astra RPG Framework -> Scaling -> Health Scaling Component`

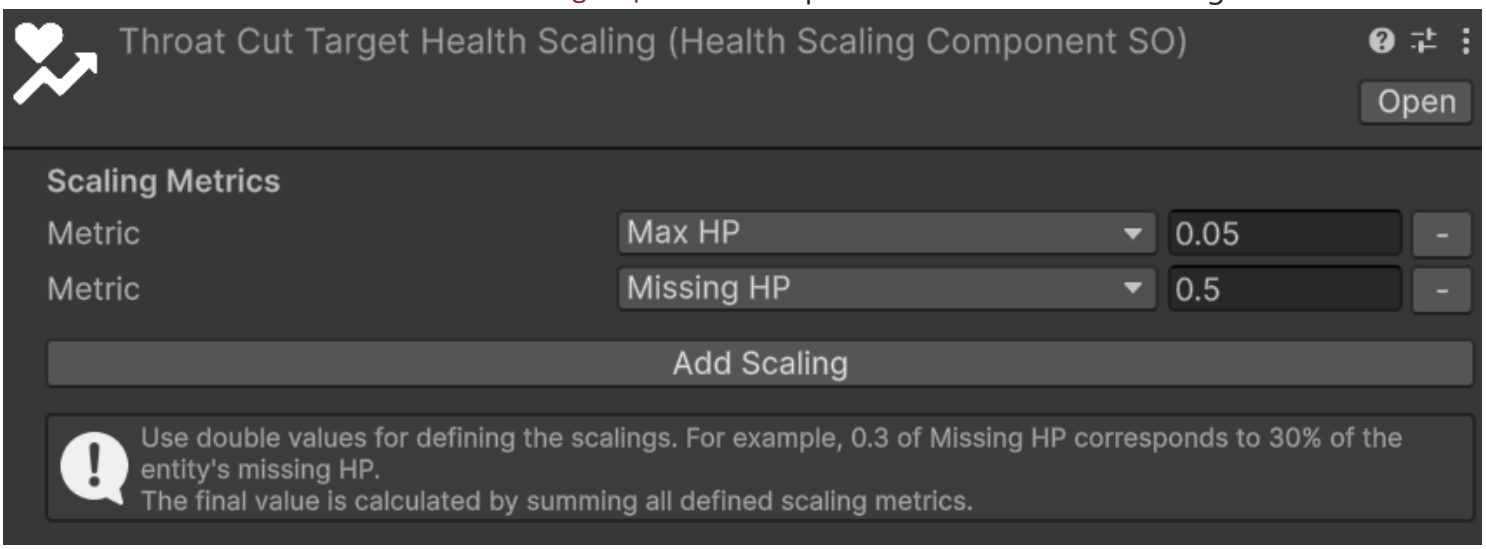
`HealthScalingComponentSO` is a ScalingComponent provided by Astra Health that calculates a scaled value from an entity's health metrics — current HP, maximum HP, or missing HP. It plugs into `ScalingFormula` assets as any other scaling component, enabling mechanics such as HP-based damage abilities and shields or heals that grow proportionally to an entity's health pool.

> **NOTE**
>
> `HealthScalingComponentSO` requires the target entity to have an `EntityHealth` component. If the component is absent, `CalculateValue` returns `0` and logs a warning. Refer to Entity Health Component for setup details.

Inspector

The inspector exposes a **Scaling Metrics** list, where each entry pairs one health metric with a `double` multiplier.

Here is a screenshot of the `HealthScalingComponentSO` inspector with two entries configured:



Each entry consists of:

- **Metric**: the health value to scale on. See Health Metrics for the available options.
- **Value** *(double)*: the multiplier applied to the chosen metric. `0.3` means 30%.

Use the **Add Scaling** button to append a new entry and the **−** button on the right of each row to remove it.

Health Metrics

Metric	Description
HP	The entity's current HP at the moment of calculation.
Max HP	The entity's current maximum HP.
Missing HP	The difference between maximum and current HP (`max - current`).

Combining Multiple Metrics

A single `HealthScalingComponentSO` can contain any number of entries. The final value is the integer sum of each individual contribution:

$$total = round(metric_1\_value \times multiplier_1) + round(metric_2\_value \times multiplier_2) + \dots$$

For example, a component configured with `Missing HP: 0.3` and `Max HP: 0.1` on an entity with 200 Max HP and 80 current HP (120 missing) yields:

```
round(120 × 0.3) + round(200 × 0.1) = 36 + 20 = 56
```

Usage in a Scaling Formula

`HealthScalingComponentSO` is a `ScalingComponent`, so it plugs directly into the **Self Scaling Components** or **Target Scaling Components** lists of a `ScalingFormula`. Refer to the Scaling Formulas section of the base framework documentation for the full workflow.

> **TIP**
>
> Assign the component to **Target Scaling Components** when the value should reflect the *target's* health state — for example, execute damage that scales with the target's missing HP. Use **Self Scaling Components** when the value reflects the *caster's* state — for example, a shield whose strength scales with the caster's max HP.

Experience Collection

Experience collection is the mechanism through which entities earn experience points when they contribute to enemy kills (or entities deaths in general in some cases). In Astra Health, the system revolves around two actors: the *collector*, the entity that receives experience, and the *victim*, the entity that dies and provides it.

When a victim dies, its `EntityHealth` raises the Global Entity Died event. Any `ExpCollector` component listening to that event evaluates its configured strategy to determine whether XP should be awarded — and if so, extracts the XP amount from the victim and forwards it to the collector's `EntityLevel`. The entire validation and collection logic lives inside a dedicated *Experience Collection Strategy* asset, making it straightforward to swap, extend, or compose strategies without touching any component code.

> **IMPORTANT**
>
> Experience collection relies on the **Global Entity Died** event configured in your `AstraHealthConfigSO`. Ensure this event is assigned before entering Play Mode — without it, `ExpCollector` will never be notified of kills. See Global Entity Died Event for details.

Prerequisites

Before setting up experience collection, ensure the following are in place.

On each **victim** entity:

- **The victim must expose experience through an `ExpSource` component**. The Astra Framework provides a concrete `ExpSource : MonoBehaviour` that stores the XP amount and a harvested flag. Attach this component to any entity that should provide XP when it dies, and configure the amount accordingly. Refer to the Framework documentation for details: https://electricdrill.github.io/AstraRpgFrameworkDocs/MD/workflows.html#exp-source

On the **collector** entity:

- **The collector must be an entity.** Therefore, it must have the `EntityCore` component.

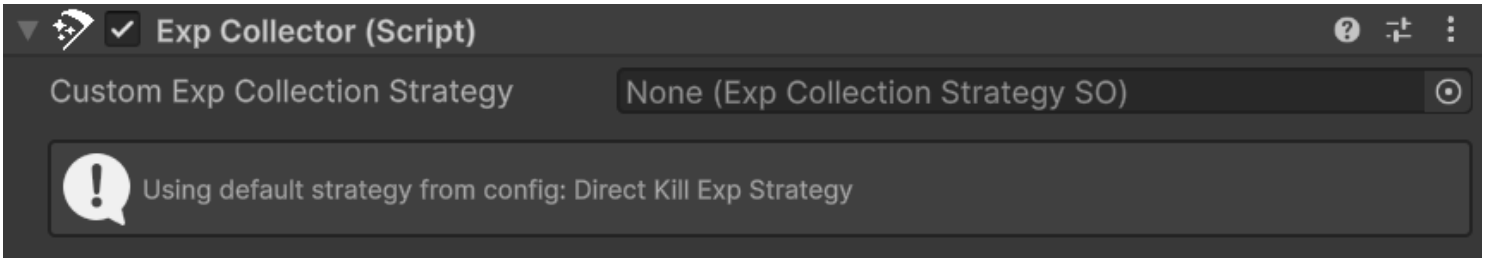
Setting Up the Exp Collector

`ExpCollector` is the component that bridges the global death event and the configured strategy. Add it to any entity that should receive XP from kills.

`ExpCollector` automatically subscribes to the Global Entity Died Event configured in `AstraHealthConfigSO` — no manual event wiring is required. Whenever any entity dies, the component evaluates its configured

strategy to determine whether XP should be awarded.

The following image shows the `ExpCollector` inspector:



Custom Exp Collection Strategy is the optional per-entity override. Assign a strategy here to give this specific collector its own behavior — for example, a boss that earns XP only from specific kill types, or a trap system with its own award logic. When left empty, the collector falls back to the project-wide default configured in `AstraHealthConfigSO` (see Default Strategy in Package Configuration).

Inspector Status Box

The custom `ExpCollector` inspector includes a status indicator that shows at a glance whether an active strategy is in place:

- **Info — "Using custom strategy: [name]"**: a **Custom Exp Collection Strategy** is assigned on this component.
- **Info — "Using default strategy from config: [name]"**: no custom strategy is assigned, but a default is found in the `AstraHealthConfigSO`.
- **Warning — "No AstraRpgHealthConfig found..."**: no package configuration asset is in scope. See Package Configuration for setup details.
- **Warning — "No default experience collection strategy configured in AstraRpgHealthConfig..."**: a configuration asset exists but no **Default Exp Collection Strategy** is set on it.

Default Strategy in Package Configuration

Collectors without a **Custom Exp Collection Strategy** fall back to the **Default Exp Collection Strategy** set in the `AstraHealthConfigSO` (see Package Configuration - Experience). Configuring this field once means every bare `ExpCollector` in the project inherits the same behavior without needing an explicit assignment on each one.

Strategy resolution at runtime follows this order:

1. **Custom strategy** assigned on the `ExpCollector` → used directly.
2. **No custom strategy** → default strategy from `AstraHealthConfigSO` → used.
3. **No custom strategy and no config or default** → a warning is logged and no XP is collected.

Built-in Strategies

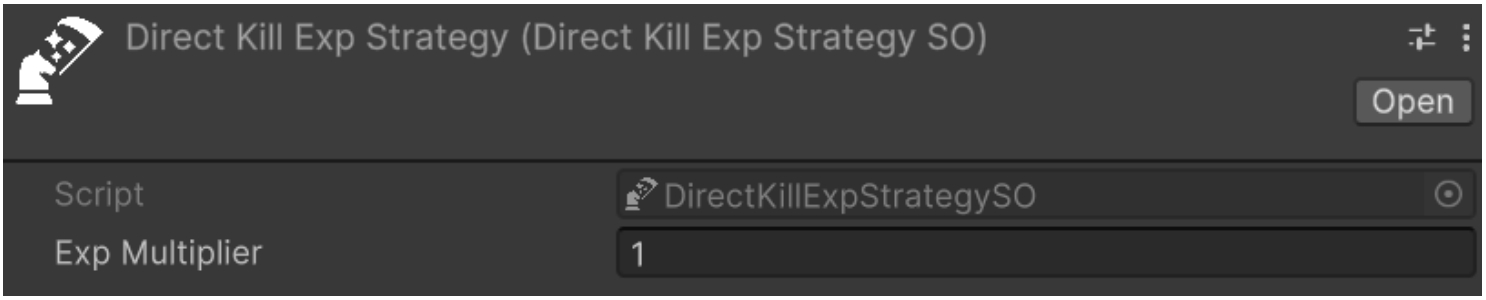
Three strategies are provided out of the box. Each is a `ScriptableObject` asset you create, configure, and assign to an `ExpCollector` or set as the project default in `AstraHealthConfigSO`.

Direct Kill

Relative path: `Astra Health -> Exp Collection Strategies -> Direct Kill`

`DirectKillExpStrategySO` is the most straightforward strategy: the collector receives XP only if it personally dealt the finishing blow. This is the natural choice for games where XP belongs exclusively to the entity that secured the kill.

The following image shows a `DirectKillExpStrategySO` in the inspector:



- **Exp Multiplier**: a float multiplier applied to the victim's base XP amount (default: `1.0`). Values greater than `1` grant a bonus — for example, `2.0` awards double XP for personal kills.

For XP to be awarded, all of the following must hold:

- The entity that dealt the final blow is the collector itself.
- The victim has an `ExpSource` component.
- The victim's `ExpSource` has not already been harvested by another collector.

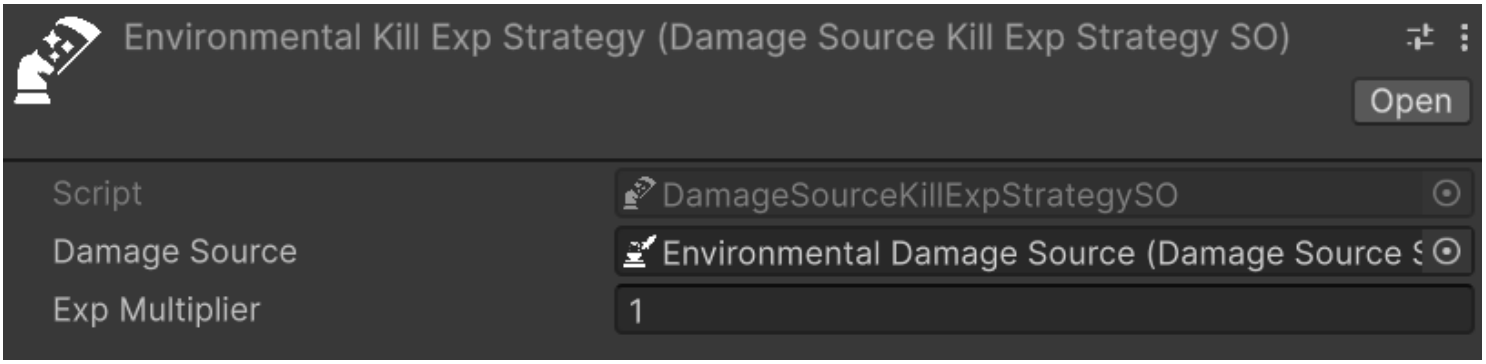
The harvested check ensures that a single kill cannot be claimed twice when multiple `ExpCollector` entities use non-harvested-dependent strategies.

Damage Source Kill

Relative path: `Astra Health -> Exp Collection Strategies -> Damage Source Kill`

`DamageSourceKillExpStrategySO` grants XP to the collector whenever a kill is delivered by a specific `DamageSourceSO`, regardless of which entity caused it. This is the right strategy for scenarios where the *cause* of death — rather than the attacker — determines who is rewarded.

The following image shows a `DamageSourceKillExpStrategySO` in the inspector:



- **Damage Source** *(required)*: the `DamageSourceSO` whose lethal hits trigger XP collection on this collector.
- **Exp Multiplier**: float multiplier on the victim's base XP (default: `1.0`).

Let's look at a classic example from the Souls-like genre. In those games, the player still earns souls when an enemy falls off a ledge and dies, even though the player did not deliver the final blow. To replicate this:

- Create a `DamageSourceSO` named "Environmental" and use it for all fall damage applied to enemies.
- Create a `DamageSourceKillExpStrategySO`, set **Damage Source** to "Environmental", and assign it to the player's `ExpCollector`.

Now whenever an enemy is killed by environmental damage, the player's collector fires and the player earns the XP.

> **NOTE**
>
> `DamageSourceKillExpStrategySO` does not check whether the victim's `ExpSource` has been harvested. If multiple `ExpCollector` entities use a strategy configured with the same `DamageSourceSO`, each of them receives XP independently when that source kills a victim. This is intentional for scenarios like the one above, but factor it into your design when multiple collectors may be active simultaneously.

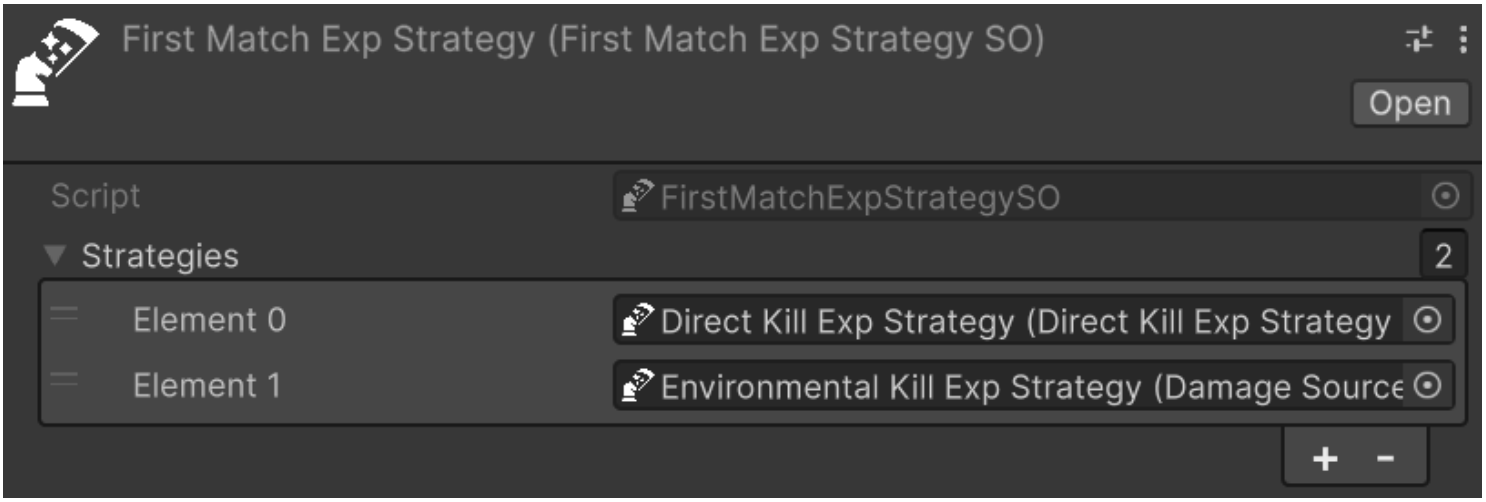
First Match Composite

Relative path: `Astra Health -> Exp Collection Strategies -> First Match`

With the previous strategies, you are bound to a single condition for XP collection: either the collector must deal the killing blow, or the kill must come from a specific damage source. What if you want a more complex setup where multiple conditions can grant XP, each with its own priority?

`FirstMatchExpStrategySO` is a composite strategy that holds an ordered list of sub-strategies. It tries each in sequence and awards XP through the first one that succeeds, leaving the rest unevaluated.

The following image shows a `FirstMatchExpStrategySO` in the inspector:



- **Strategies**: the ordered list of `ExpCollectionStrategySO` assets to evaluate.

This is useful when an entity should earn XP under different conditions, each with a clear priority. For example:

- Try `DirectKillExpStrategySO` first — reward if the entity delivered the finishing blow.
- Fall back to `DamageSourceKillExpStrategySO` — reward if the kill came from a specific environmental source.

> **NOTE**
> Only the first matching sub-strategy collects XP. Once a strategy succeeds, the remaining entries in the list are skipped.

Custom Strategies

To implement experience collection logic beyond what the built-in strategies offer, extend `ExpCollectionStrategySO`. The class uses the Template Method pattern: `TryCollectExp` orchestrates the entire flow, and the virtual and abstract methods are the designated extension points.

The table below lists the available override points:

Method	Type	Purpose
`Validate`	abstract	Determine whether XP should be collected for this event. Returns a `multiplier` out parameter alongside the bool result.
`CollectExp`	virtual	Perform the actual collection: calculate the amount, call `AddExp`, and mark the source as harvested. Override for behaviors like distributing

Method	Type	Purpose
		XP across a party.
`CalculateExpAmount`	virtual	Compute the final XP amount from the source and the multiplier. Override to change the formula.
`TryGetExpSource`	virtual	Extract an `IExpSource` from the victim. Override if experience sources are not standard `MonoBehaviour` components on the victim's GameObject.
`TryCollectExp`	virtual	The full template method. Override for complete control over the collection flow, as `FirstMatchExpStrategySO` does.

For most custom strategies, overriding `Validate` is sufficient: implement your conditions, set the multiplier, and return `true` or `false`. Override `CollectExp` only when the collection action itself must differ — for example, to split the XP reward across multiple entities simultaneously.

For more information about the API and extension points, refer to the `ExpCollectionStrategySO` API documentation.

Resurrection

Resurrection brings a dead entity back to life, restoring it to a playable state with a configurable amount of HP. Unlike healing — which throws an exception when applied to a dead entity — resurrection is exclusively for entities that have already crossed their death threshold.

The requested HP amount is a **base** value before modifiers. Internally, resurrection routes through the same `Heal` pipeline used for direct healing, so all generic and `HealSourceSO`-specific heal modifiers apply. This means resurrection integrates naturally with any modifier system already in place: a buff that amplifies healing received will equally benefit resurrections, without any additional configuration.

Package Configuration

Before resurrection can be triggered, two fields in the `AstraHealthConfigSO` need attention:

Default Resurrection Source

Type: `HealSourceSO` — **Required:** Yes

The `HealSourceSO` used automatically by the convenience `Resurrect` overloads and by the built-in Resurrect Game Action when no explicit source is provided. It categorises resurrection healing as a distinct type, enabling `HealSourceSO`-specific modifiers to apply. Without this field assigned, the convenience overloads fail at runtime with an assertion error.

See Heal Source for guidance on configuring `HealSourceSO` assets and Default Resurrection Source in the Package Configuration reference.

Default On Resurrection Game Action

Type: `GameAction<IHasEntity>` — **Required:** No

The Game Action executed automatically whenever any entity is resurrected, unless the entity defines its own Override On Resurrection Game Action. Use this to implement a common post-resurrection behavior shared across all entities — playing a revival visual effect, re-enabling a disabled GameObject, or resetting specific state. Leave it empty if no shared behavior is needed.

See Default On Resurrection Game Action in the Package Configuration reference.

Resurrecting an Entity

`EntityHealth` implements `IResurrectable`, which exposes three overloads.

`Resurrect(PreHealContext)` gives full control over the resurrection: amount, source, and healer entity. It expects a `PreHealContext` built with the same fluent builder used in direct healing:

```
// Assuming:
// - resurrectionSource is the HealSourceSO categorising this resurrection
// - healer is the EntityCore of the character performing the resurrection (null for a
system action)
// - target is the EntityCore of the dead entity to resurrect

if (target.TryGetComponent(out EntityHealth targetHealth)) {
    PreHealContext context = PreHealContext.Builder
        .WithAmount(targetHealth.MaxHp / 2)   // 50% of max HP as base amount,
before modifiers
        .WithSource(resurrectionSource)
        .WithTarget(target)
        .WithHealer(healer)                    // optional
        .Build();

    targetHealth.Resurrect(context);
}
```

Refer to Direct Healing for a detailed explanation of `PreHealContext` and its fluent builder.

`Resurrect(long)` and **`Resurrect(Percentage)`** are convenience overloads that build the context automatically using the **Default Resurrection Source** from the configuration and a `null` healer (system-initiated resurrection). Use them when no specific caster is involved and the global default source is appropriate:

```
// Resurrect with a flat HP amount
targetHealth.Resurrect(100L);

// Resurrect with 30% of max HP
Percentage thirtyPercent = 30L;
targetHealth.Resurrect(thirtyPercent);
```

In all cases, the HP amount is the **base** value before heal modifiers are applied. Two constraints are always checked: the base amount must be strictly greater than the entity's death threshold, and must not exceed its maximum HP.

Heal Modifiers at Resurrection

Because resurrection calls `Heal` internally, the full modifier stack applies:

- **Generic flat heal modifiers** — add or subtract a flat HP amount from the base
- **Generic percentage heal modifiers** — scale the base HP up or down by a percentage

- `HealSourceSO`**-specific modifiers** — flat and percentage modifiers associated with the configured resurrection source

The order of application is the same as described in the Direct Healing section: first the flat modifiers (generic and `HealSourceSO`-specific) are summed and applied to the base amount, then all percentage modifiers (generic and `HealSourceSO`-specific) are summed and applied to the result.

The entity's HP after resurrection may therefore differ from the base amount requested. `ResurrectionContext.NewValue` always reflects the **actual HP after all modifiers** have been applied.

> **NOTE**
> If active negative heal modifiers are strong enough to reduce the effective HP to or below the entity's death threshold, the system applies a fallback and forces HP to `death threshold + 1`, guaranteeing the entity is always resurrected alive. `ResurrectionContext.NewValue` reflects this clamped value.

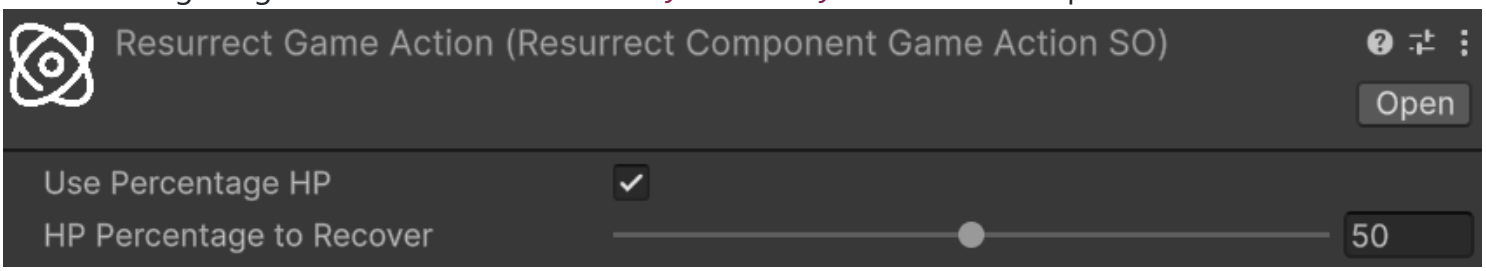
The Resurrect Game Action

Relative path: `Astra RPG Framework/Game Actions/Context: Entity/Resurrect`

`ResurrectEntityIHasEntityGameActionSO` is a ready-made Game Action that can be dropped into any Game Action slot in the inspector to resurrect an entity. It always uses the **Default Resurrection Source** from the configuration; no source field is exposed in the inspector.

Because its context type is `IHasEntity`, it connects directly to any `GameEventListener` whose payload implements `IHasEntity` — which includes all health event listeners such as `EntityDiedGameEventListener`, `DamageResolutionGameEventListener`, and `EntityResurrectedGameEventListener` — without requiring a Component reference.

The following image shows the `ResurrectEntityIHasEntityGameActionSO` inspector:



The configurable fields are:

- **Use Percentage HP**: if enabled, the entity is resurrected with a percentage of its maximum HP; if disabled, a flat HP amount is used instead

- **Percentage HP to Recover**: the percentage of max HP to restore, in the range 0–100. Active only when **Use Percentage HP** is enabled
- **Flat HP to Recover**: the flat HP amount to restore. Active only when **Use Percentage HP** is disabled

> **NOTE**
> If **Percentage HP to Recover** or **Flat HP to Recover** is set to 0, the game action falls back to resurrecting the entity with exactly 1 HP, guaranteeing the resurrection is always valid.

Common use case — automatic resurrection after a delay: combine `ResurrectEntityIHasEntityGameActionSO` with a Delayed Game Action inside the entity's **Override On Death Game Action** (or the global **Default On Death Game Action**). When the entity dies, the death game action fires a delayed sequence that triggers the `ResurrectEntityIHasEntityGameActionSO` after a configured number of seconds — implementing an automatic respawn without a single line of code. See the Game Actions documentation for details on composing and chaining game actions.

Per-Entity Customization

Each `EntityHealth` component exposes an **Override On Resurrection Game Action** field in its inspector that takes precedence over the **Default On Resurrection Game Action** defined in the configuration. Use it to give specific entities a unique post-resurrection behavior while keeping a sensible global default for everything else.

See Death in the Entity Health reference for the full list of per-entity death and resurrection inspector fields.

Resurrection Events

Resurrecting an entity raises multiple events. Because resurrection routes through `Heal`, the healing events fire first:

- **Pre Heal Event** and **Entity Healed Event** — raised by the internal `Heal` call, carrying the standard healing context before and after modifiers. Subscribe to these if your logic needs to react to the healing component of a resurrection; for example, to trigger a heal-triggered ability even during resurrection
- **Entity Health Changed Event** — raised when the entity's HP increases during the `Heal` call. Useful for general HP-change listeners that do not need the full context of the healing, but just want to know when HP changes and by how much. An example use case is a UI health bar that needs to update whenever HP changes, regardless of the source of the change

After the healing pipeline completes and the resurrection game action has been dispatched:

- **Entity Resurrected Event** — the dedicated resurrection signal, carrying a `ResurrectionContext` with the following fields:
  - **Target**: the `EntityCore` of the resurrected entity (convenience accessor for `ReceivedHeal.Target`)
  - **PreviousValue**: HP immediately before resurrection (at or below the death threshold)
  - **NewValue**: HP after resurrection, computed as `PreviousValue + ReceivedHeal.HealAmount.NetAmount`
  - **AbsAmount**: `NewValue - PreviousValue`
  - **ReceivedHeal**: the full `ReceivedHealContext` produced by the internal `Heal` call, exposing:
    - **HealAmount**: a `HealAmountContext` with `RawAmount` (base amount before modifiers), `AfterModifierAmount` (after flat and percentage modifiers), and `NetAmount` (final HP actually added, capped to max HP)
    - **HealSource**: the `HealSourceSO` used for this resurrection
    - **Performer**: the entity that performed the resurrection (`null` for system-initiated resurrections, such as those triggered via the convenience overloads or the `ResurrectEntityIHasEntityGameActionSO`)
    - **IsCritical**: whether the resurrection heal was flagged as a critical

Each event has a corresponding Event Channel on `EntityHealth`. When that channel is raised, it resolves and raises the matching global event configured at package level, if any, and also raises any extra events assigned to that channel. Use global events when any GameObject in the game needs to receive the signal; use extra events on the channel for targeted, component-specific or GO-hierarchy-specific communications. See Global Events and Event Channels for guidance on choosing between them.

Game Actions

Astra Health extends the framework's `GameAction` system with a set of health-specific actions for resurrection, override management, and damage reactions. Each action is a `ScriptableObject` built on top of `GameAction<TContext>` and integrates directly with `EntityHealth` and the event system.

For an introduction to the `GameAction` execution model, execution with Unity Events, and the difference between `ExecuteAsync` and `RunFireAndForget`, refer to the Game Actions section of the Astra RPG Framework documentation.

IHasEntity-Based Actions

The recommended way to use the health game actions is through the `IHasEntity`-based variants. Most event context payloads raised by the health package — `DamageResolutionContext`, `EntityDiedContext`, `PreDamageContext`, `ResurrectionContext`, and others — implement the framework's `IHasEntity` interface, which exposes the primary `EntityCore` subject of the event. This means that any `GameEventListener` whose payload implements `IHasEntity` can drive these actions directly without any adapter or component extraction.

The `IHasEntity`-based variants are listed under `Astra RPG Framework/Game Actions/Context: Entity/` in the asset creation menu and should be the first choice whenever actions are wired to a `GameEventListener` response.

Resurrect

`ResurrectEntityGameActionSO<TContext>` is the abstract base action for resurrecting a dead entity. It resolves the `EntityHealth` component from `context.Entity` and calls the appropriate convenience `Resurrect` overload — either with a percentage of max HP or with a flat HP amount — depending on the inspector configuration.

The sealed `ResurrectEntityIHasEntityGameActionSO` is the ready-to-use instantiable variant with `IHasEntity` as the context type.
Relative path: `Astra RPG Framework/Game Actions/Context: Entity/Resurrect`

Its inspector fields, configuration options, and a common automatic-respawn use case are documented in the Resurrect Game Action section of the Resurrection workflow.

> **NOTE**
>
> Because all health event payloads implement `IHasEntity`, `ResurrectEntityIHasEntityGameActionSO` can be connected directly to any `GameEventListener` in the package — for example to an `EntityDiedGameEventListener` to trigger an automatic resurrection on death — without extracting a `Component` reference first.

Set Override On Death

Relative path: `Astra RPG Framework/Game Actions/Context: Entity/Set Override On Death`

`SetEntityOverrideOnDeathGameActionSO<TContext>` assigns or clears the `OverrideOnDeathGameAction` property on an entity's `EntityHealth` component at runtime. It gives any system or workflow the ability to redirect what happens the next time a specific entity dies, without modifying the global configuration or the entity's inspector directly.

The configurable field is:

- **New Override Game Action**: the `GameAction<IHasEntity>` to assign as the entity's next on-death action. Leave this field empty to clear any previously set override, causing the entity to fall back to the **Default On Death Game Action** from the package configuration

Use Cases:

- One-shot death mechanics: place a `SetEntityOverrideOnDeathGameActionSO` inside a `CompositeGameAction` that first resurrects the entity and then clears the override (by leaving **New Override Game Action** empty). The first time the entity dies the override fires — resurrecting it — and removes itself so subsequent deaths use the default behavior
- Dynamic difficulty: assign a stronger or weaker on-death routine to a specific entity at runtime based on game state

> **NOTE**
>
> The action resolves `EntityHealth` from the context via `context.Entity.GetComponent<EntityHealth>()`. If the entity does not have an `EntityHealth` component, a warning is logged and the action does nothing.

The sealed `SetEntityOverrideOnDeathIHasEntityGameActionSo` is the instantiable variant with `IHasEntity` as the context type:

Set Override On Resurrection

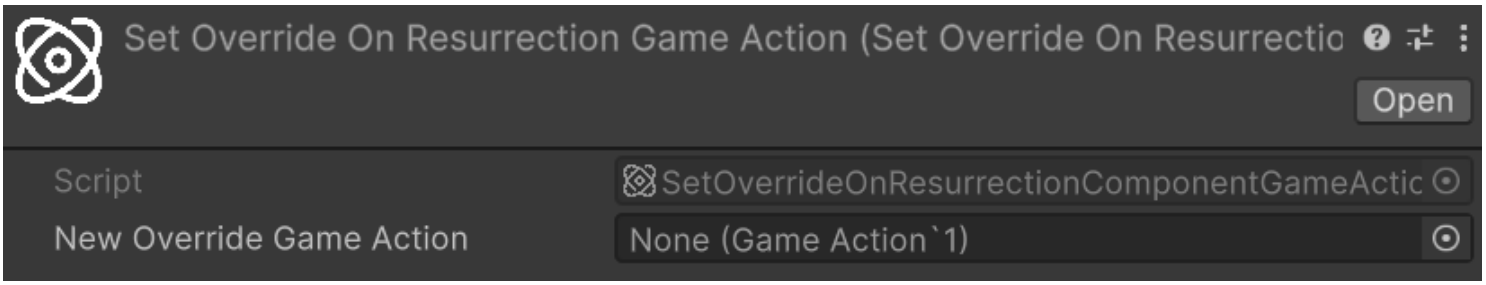
Relative path: `Astra RPG Framework/Game Actions/Context: Entity/Set Override On Resurrection`

`SetEntityOverrideOnResurrectionGameActionSO<TContext>` mirrors its death counterpart: it assigns or clears the `OverrideOnResurrectionGameAction` property on an entity's `EntityHealth` component, redirecting the post-resurrection behavior the next time that entity is resurrected.

The configurable field is:

- **New Override Game Action**: the `GameAction<IHasEntity>` to assign as the entity's next on-resurrection action. Leave this field empty to clear the override and fall back to the **Default On Resurrection Game Action** from the package configuration

The sealed `SetEntityOverrideOnResurrectionIHasEntityGameActionSo` is the instantiable variant with `IHasEntity` as the context type:



Context Projection Actions

Context projection actions allow a broad `IHasEntity` listener to forward its payload to an action that expects a richer, more specific context type. The base class `EntityContextCastProjectionGameAction<TProjectedContext>` attempts a runtime cast from `IHasEntity` to `TProjectedContext` and, if the cast succeeds, delegates to the wrapped inner action; if the cast fails, the action silently does nothing.

Use these actions when a single `GameEventListener` needs to drive logic that lives inside an action typed on a concrete context — for example, routing an `EntityDiedGameEventListener` response through to a `CounterDamageOnDeathGameActionSO` without writing a custom listener.

Three sealed implementations are provided:

- **`EntityContextToDamageResolutionContextProjectionGameAction`** — narrows `IHasEntity` to `DamageResolutionContext` and forwards to a `GameAction<DamageResolutionContext>`. Created via `Astra RPG Framework/Game Actions/Context: Entity/Projections/→ DamageResolutionContext Projection`

- **`EntityContextToEntityDiedContextProjectionGameAction`** — narrows `IHasEntity` to `EntityDiedContext` and forwards to a `GameAction<EntityDiedContext>`. Created via `Astra RPG Framework/Game Actions/Context: Entity/Projections/→ EntityDiedContext Projection`

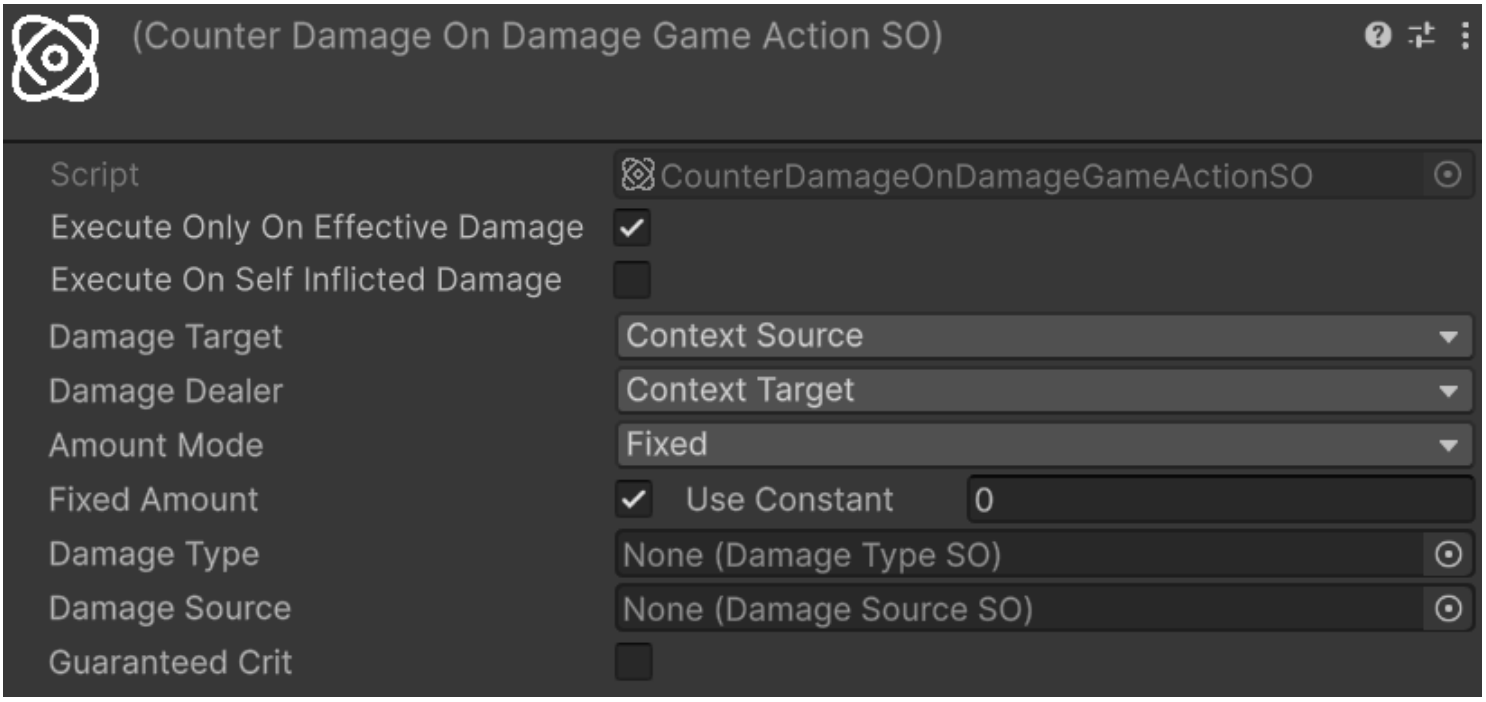
- **`EntityContextToPreDamageContextProjectionGameAction`** — narrows `IHasEntity` to `PreDamageContext` and forwards to a `GameAction<PreDamageContext>`. Created via `Astra RPG Framework/Game Actions/Context: Entity/Projections/→ PreDamageContext Projection`

The configurable field on each projection action is:

- **Inner Action**: the `GameAction<TProjectedContext>` to execute when the cast succeeds. Leave empty to produce a no-op.

Counter Damage

`CounterDamageGameActionSO<TContext>` is the abstract base for actions that react to a damage or death event by immediately inflicting a new hit in return. The action reads role assignments and a damage configuration from the inspector, resolves the correct entities from the event context, and calls `EntityHealth.TakeDamage` on the resolved target.



`CounterDamageGameActionSO<TContext>` implements `IEffectInstigator`, so it registers itself as the instigator of the counter-damage it produces. Downstream listeners can compare `context.Instigator`

against the action asset to identify and filter self-triggered events — the primary mechanism for preventing infinite damage cycles (see Preventing Infinite Cycles below).

`TContext` must implement `IHasTarget`, `IHasPerformer`, `IHasInstigator`, and `IHasAmount`. The compatible built-in context types are `DamageResolutionContext`, `EntityDiedContext`, and `PreDamageContext`.

The top-level configurable fields are:

- **Execute Only On Effective Damage**: when enabled (the default), the counter-damage fires only if the incoming `Amount` is non-null and greater than zero. Disable to also react to prevented or zero-damage hits
- **Execute On Self Inflicted Damage**: when enabled, the counter-damage fires even when the instigator of the incoming event is this same action asset. Leave disabled (the default) to suppress the counter when this action already caused the incoming event, preventing infinite loops
- **Config**: the inline `PreDamageContextConfig` block that describes the counter-damage to apply (see Counter Damage Configuration below)

Counter Damage Configuration

The **Config** block is a serialized `PreDamageContextConfig` that fully defines the counter-damage: who takes it, who is attributed as the dealer, how the amount is computed, and what damage type and source are applied.

- **Damage Target**: which entity from the incoming context receives the counter-damage. `ContextSource` targets the entity that performed the original action (`context.Performer`); `ContextTarget` targets the entity that received it (`context.Target`)
- **Damage Dealer**: which entity from the incoming context is attributed as the dealer. `ContextSource` maps to the performer, `ContextTarget` maps to the original target, and `None` produces system-initiated damage with no attribution
- **Amount Mode**: determines how the counter-damage amount is computed:
  - `Fixed` — a constant value from a **Fixed Amount** `LongRef` asset
  - `ScalingFormula` — computed at runtime from a **Scaling Formula** asset (dealer maps to *Self*, target to *Target*). If the dealer is null (system damage), the formula falls back to `CalculateValue(target, target)` with a warning
  - `DamageFromContext` — a fraction of the incoming `Amount`, scaled by the **Damage From Context Multiplier** (e.g. `0.5` = 50% of the received damage)
- **Damage Type**: the `DamageTypeSO` applied to the counter-damage
- **Damage Source**: the `DamageSourceSO` attributed to the counter-damage
- **Guaranteed Crit**: when enabled, the counter-hit is always flagged as a critical
- **Use Crit Stat**: when **Guaranteed Crit** is active, reads the critical multiplier from the **Crit Multiplier Stat** on the dealer entity instead of from the inline **Critical Multiplier** value

- **Crit Multiplier Stat**: the stat on the dealer entity whose value is used as the critical multiplier (e.g. a stat value of `200` becomes `2.0×`). Active only when both **Guaranteed Crit** and **Use Crit Stat** are enabled
- **Critical Multiplier**: the inline critical multiplier as a percentage (e.g. `150` = `1.5×`). Active when **Guaranteed Crit** is enabled and **Use Crit Stat** is disabled

> **WARNING**
>
> If **Damage Type** or **Damage Source** is left unassigned, the system logs a warning at runtime and the resulting `PreDamageContext` may behave unexpectedly.

Let's see a concrete example of a damage-reflection setup. Entity `A` attacks entity `B`, which has a `CounterDamageOnDamageGameActionSO` listening to its `DamageResolutionGameEventListener`. When the hit resolves:

- `context.Performer` = `A` (the attacker)
- `context.Target` = `B` (the defender)
- `context.Amount` = the damage actually dealt

To reflect damage back at `A` with a fixed amount attributed to `B`, configure:

- **Damage Target** = `ContextSource` (the counter-damage targets the performer, `A`)
- **Damage Dealer** = `ContextTarget` (the counter-damage is dealt by the original target, `B`)
- **Amount Mode** = `Fixed`, with **Fixed Amount** set to the desired flat value

This example assumes no other damage modifications are active.

Real-World Examples

Two passive ability implementations from the sample scene illustrate `CounterDamageGameActionSO` in concrete setups.

Vengeance in Death (Assassin passive) uses `CounterDamageOnDeathGameActionSO` to strike back at the attacker with a guaranteed critical physical hit equal to 80% of the lethal damage received. **Damage Target** is `ContextSource` (the attacker), **Damage Dealer** is `ContextTarget` (the Assassin), **Amount Mode** is `DamageFromContext` with a multiplier of `0.8`, and **Guaranteed Crit** is enabled with **Use Crit Stat** pointing to the Assassin's critical multiplier stat. To avoid firing on every entity's death, the action is wired to a dedicated `EntityDiedGameEvent` assigned as an extra death event on the Assassin's death Event Channel rather than to the global death event. See Vengeance in Death in the Samples page for the full setup.

Glass Cannon (Sorcerer passive) uses `CounterDamageOnDamageGameActionSO` to deal bonus magical self-damage equal to 15% of the Sorcerer's Magic Power every time the Sorcerer is hit. **Damage Target** is

`ContextTarget` (the Sorcerer itself), **Damage Dealer** is `None`, **Amount Mode** is `ScalingFormula` with a formula that evaluates 15% of Magic Power. To avoid reacting to other entities' damage events, the action is wired to a dedicated `DamageResolutionGameEvent` assigned as an extra damage-taken event on the Sorcerer's damage Event Channel. **Execute On Self Inflicted Damage** remains disabled so the self-damage this action produces does not re-trigger the action. See Glass Cannon in the Samples page for the full setup.

Preventing Infinite Cycles

Counter-damage actions can produce infinite loops: `A` damages `B`, `B` counters `A`, `A`'s counter reacts to `B`'s hit, and so on. Three complementary mechanisms help contain runaway chains:

- **Disable Execute On Self Inflicted Damage** (default): because `CounterDamageGameActionSO` implements `IEffectInstigator` and passes itself as the instigator of the counter-damage it produces, any subsequent event caused by that counter carries the action asset as `context.Instigator`. When **Execute On Self Inflicted Damage** is disabled, the action skips execution whenever the incoming instigator matches itself — suppressing a single action from reacting to its own direct output
- **Enable Execute Only On Effective Damage** (default): zero-damage or prevented hits do not trigger a counter, reducing the chance of a chain reaction starting from a suppressed hit
- **Reactability marker**: every damage and heal context carries an `IsReactable` flag. When `CounterDamageGameActionSO` receives a context with `IsReactable == false`, it skips execution immediately. Every `PreDamageContext` built by the action always has `IsReactable` set to `false`, and this propagates automatically to the resulting `DamageResolutionContext` and, if the hit is lethal, to the `EntityDiedContext`

The third mechanism is what breaks the mutual counter-chain that the first two cannot. If entity `A` owns counter action `CDA` and entity `B` owns counter action `CDB`: `A` hits `B` with a first-order `IsReactable = true` event → `CDB` fires, hits `A` with `IsReactable = false` → `CDA` receives a non-reactable context and skips — the chain ends after one counter.

The `IsReactable` flag is also available on `PreHealContext` and `ReceivedHealContext`, following the same convention. Use `.WithIsReactable(false)` on the builder when constructing any secondary heal (for example, a passive ability that heals on receiving damage) to prevent heal-triggered chains.

> **NOTE**
>
> Code that constructs `PreDamageContext` instances directly — for example, a custom passive ability that deals secondary damage in response to an event — should call `.WithIsReactable(false)` on the builder when the resulting hit is a secondary effect and should not trigger further reactive systems.

> **WARNING**
>
> Enabling **Execute On Self Inflicted Damage** removes the self-instigator guard. Only do so when the action is intentionally designed to react to its own outputs, and ensure an alternative termination condition is in place.

Available Variants

Three sealed implementations are provided, each bound to a specific context type:

- **`CounterDamageOnDamageGameActionSO`** — context: `DamageResolutionContext`. Reacts to a resolved damage event. Connect to a `DamageResolutionGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(DamageResolutionContext)` as the dynamic invocation target. Created via `Astra RPG Framework/Game Actions/Context: DamageResolutionContext/Counter Damage`
- **`CounterDamageOnDeathGameActionSO`** — context: `EntityDiedContext`. Reacts when an entity dies. Connect to an `EntityDiedGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(EntityDiedContext)` as the dynamic invocation target. Created via `Astra RPG Framework/Game Actions/Context: EntityDiedContext/Counter Damage`
- **`CounterDamageOnPreDamageGameActionSO`** — context: `PreDamageContext`. Reacts before the damage calculation pipeline runs. Connect to a `PreDamageGameEventListener`'s UnityEvent response and select `ExecuteAsyncForUnityEvent(PreDamageContext)` as the dynamic invocation target. Created via `Astra RPG Framework/Game Actions/Context: PreDamageContext/Counter Damage`

> **NOTE**
>
> When using `CounterDamageOnPreDamageGameActionSO`, `context.Amount` is the pre-modifier base damage amount rather than the final resolved damage. The **Execute Only On Effective Damage** check therefore evaluates the base amount, not the post-calculation result.

Composite Pre-Damage Context

Relative path: `Astra RPG Framework/Game Actions/Context: PreDamageContext/Composite`

`CompositePreDamageContextGameActionSO` is a sealed implementation of the framework's `CompositeGameAction<PreDamageContext>`. It executes a list of `GameAction<PreDamageContext>` actions in sequence, passing the same `PreDamageContext` to each. Use it to chain multiple pre-damage context actions — for example, a `CounterDamageOnPreDamageGameActionSO` followed by a state modifier — within a single `PreDamageGameEventListener` response.

Limitations

Requirements

- Astra RPG Framework
- Unity 6 or later

Package Contents

Samples

No More Utils Folder

Unlike Astra Framework, in this package you will not find a `Utils` folder among the various samples. I made this decision because I believe it is wiser and more robust to let you create your own instances. This is because if you build your project around the instances I provide in the samples, and then decide to import a new version of the package and reimport the samples, you risk getting confused about which instances from which version you are currently using for your project. By letting you create all the framework object instances yourself, this problem is completely avoided. Furthermore, the instances present in the sample scene's samples are marked with a `(Astra Health Samples)` suffix, making them easily identifiable and distinguishable from any instances you might create for your own project.

If you want to draw inspiration from the instances in the samples, feel free to do so, but I recommend creating new instances for your project to avoid any future confusion.

Examples Folder - Overview

Inside the `Examples` folder you will find a sample scene: it is your sandbox for trying out the features of Astra Health. Compared to the minimal one in Astra Framework, this scene has more content and features to explore. The scene covers all the main features of Astra Health:

- Health System
- Damage Types
- Damage Sources
- Damage Mitigation
- Defense Penetration
- Healing System
- Heal and Damage Modifiers
- Passive Regeneration
- Resurrection System
- Event System
- Health Scaling Component
- Experience Collection System

The Scene - Overview

Sample scene as you enter in play mode

If you open and start the sample scene, you will see a stylized fighter on the left (Duelist) and a dummy on the right (Dummy). Without much surprise, you can unleash the wrath of your character against the poor dummy in order to test the features of Astra Health. Before taking it out on the dummy, let's take a look at the rest of the scene. Above the duelist there is a panel containing the character's name, level, optional class, health bar, and finally the character's stat values. A similar panel is present above the dummy. You can scroll with the mouse wheel on these panels to view all the statistics. You will notice that unlike the Astra Framework sample scene, the panels in this scene do not show attributes. This is because I intentionally excluded their use in this scene, in order to focus on the features of Astra Health. Had I included attributes as well, it would have been more complex to reason about the effect of damage calculation formulas, reductions, penetrations and so on, since the stat values would also have been influenced by the attributes. Therefore, for simplicity, in this sample scene you will only need to focus on statistics.

In addition to the stats panels, you will notice that there are also buttons with one or two icons to their left. These represent the abilities that each character can use. And yes, even the dummy has a say in this and can take revenge for the mistreatment it has suffered in various video games over the decades. Each character has 3 abilities; each row of icon(s) + 2 buttons constitutes a single ability. For each ability, the first button contains its name and is the activation point. The button to its right is a toggle that defines whether the ability will perform a critical hit or not. Finally, the icon, or icons, to the left of the ability name represent the damage types that the various effects of the ability can inflict, or a heal if the ability's effect is a healing one. A single icon means the ability has a single effect, while two icons indicate the

ability has two or more effects. If there are 3 or more effects, only the icons of the first 2 effects will be shown. The icons you will find are 4 in total:

- An orange sword, representing physical damage
- A purple flame, representing magical damage
- A white perforated shield, representing pure damage
- A green cross, representing healing

The sword, the flame and the perforated shield therefore represent damage effects, and are associated with the respective damage types. The green cross instead represents a healing effect.

> **WARNING**
> I want to specify that the implementation given for abilities in this sample scene is simplistic and does not represent a complete ability system. It was necessary to introduce a minimal ability system to demonstrate the features of Astra Health, but do not consider this implementation as a reference point for creating a more complex ability system. It will be the responsibility of a future Astra extension package to provide a complete and flexible ability system. For now, consider this implementation as a simple tool for testing the features of Astra Health.

You will also notice that just above the duelist's head there are a couple of buttons, reading "[D] Next Hero" and "[A] Previous Hero". These buttons allow you to change your character. Alternatively to the buttons you can use the D and A keys on the keyboard. In addition to the duelist, who focuses on abilities that deal physical damage and healing, there are 2 other characters, each with a different focus. The second character, the Assassin, focuses on abilities that deal physical and pure damage, sometimes with scaling based on the enemy's health. The third character, the Mage, focuses on abilities that deal magical damage.

The Hierarchy - Overview

If you now take a look at the scene hierarchy you will see that, in addition to the default main objects, it contains:

- the dummy entity
- the dummy's canvas (stats panel and abilities panel)
- a section with the entities of the three playable characters (duelist, assassin and mage). The duelist is active at scene start, while the other two characters are disabled. Changing characters with the "[D] Next Hero" and "[A] Previous Hero" buttons, you will cycle through these three characters, activating one at a time and deactivating the other two.
- a section with the canvases of the three playable characters (stats panel and abilities panel for each character). Also in this case, the duelist's canvas is active at scene start, while the other two canvases

are disabled.
- A `HeroSelector` object. This object has as children the game objects representing the "[D] Next Hero" and "[A] Previous Hero" buttons.
- A `PopupCanvas` object, which contains two sub-objects: `DamagePopupManager` and `HealPopupManager`. These objects are responsible for creating the damage and healing popups that appear above the characters' heads when they take damage or receive healing.

Each entity has the Astra components `EntityCore`, `EntityClass`, `EntityStats`, and the new `EntityHealth` on the root object. The dummy does not have a class. This choice is not to discriminate against our wooden friend, but simply because it simplifies changing the dummy's statistics to facilitate testing the package's features. This way, if you wanted to change the Armor to see how the damage mitigation changes in real-time, you don't need to go through the Growth Formula associated with that stat in the dummy's class, but you can simply modify its value through `EntityStats`. Simple and effective.

Each entity (including the dummy) also has two sub-objects: `Visuals` and `Skills`. The first contains all the objects responsible for the character's visual representation (sprite and health bar for the dummy, sprite and attack sprite for the player characters), while the second is used to specify the character's abilities.

The objects related to the entity canvases instead have the following sub-objects: a `HeroPanel` responsible for showing the panel with the character's name, level, class, health bar and statistics, and a `SkillsPanel`, which contains the buttons and icons related to the character's abilities.

The Project Files - Overview

In the resource explorer, in the package samples, in addition to the sample scene, you will have these folders:

- **Art**: Contains all the artistic resources used in the sample scene.
- **Instances**: This is the most important folder, as it contains all the instances of the objects provided by Astra Health. You will find yourself interacting a lot with these instances to test the package's features, and you can draw inspiration from them to create your own custom instances for your project.
- **Prefabs**: Contains the prefabs of all the objects present in the sample scene.
- **Resources**: Contains the Astra Health configuration instance for the sample scene.
- **Scripts**: Contains all the scripts used in the sample scene.

> **WARNING**
>
> A note regarding `Resources`. As explained in Global Settings, Astra Health uses an `AstraHealthConfigSO` instance to configure its features in your Unity project. This specific instance provided in the samples has been configured for the sample scene. The scene works right out of the box because the configuration resource, being named exactly `Astra Health Config`, is automatically loaded by Astra Health upon package import and assigned in the package's global configuration.
>
> When you go to create your own `AstraHealthConfigSO` configuration instance for your project, you will need to assign it manually to the global configuration of Astra Health, as explained in Project Settings through the project settings, or alternatively, as explained in Manual Configuration, by assigning it directly to the `AstraHealthGlobalSettings` instance found in the `Assets/Resources` folder (this resources folder is located at the root of Assets, not in the package samples).
>
> If you do not assign your configuration instance to the global configuration of Astra Health, the package will continue to use the samples configuration, creating confusion and potential configuration issues.

Instances Folder

Now we will focus on the `Instances` folder, which is the most relevant one for you.

Spare object: Default Damage Calculation Strategy

Let's start with the loose `Default Damage Calculation Strategy` object. It represents the damage calculation strategy used by default in the scene.

Statistics and Stat Sets

In the `Stats` folder you will find all the instances of the `Stat` and `StatSet` used in the sample scene. You can explore the statistics and stat sets in detail yourself; the only thing I want to highlight is that the `Hero Stat Set` is the stat set used by all 4 characters in the sample scene (duelist, assassin, mage and dummy).

Classes

In the `Classes` subfolder you will find all the classes for the 3 playable characters and all the related growth formulas that define the progression of their statistics. There is also a `Common` folder that contains some utilities shared by multiple classes. For example, the `200% Const Critical Multiplier GF` is a growth formula that returns a 200% multiplier for critical hits at all levels. Usually a game uses a fixed multiplier for all entities at all levels, so it makes no sense to duplicate this growth formula in every class, but it is more efficient to create a single instance and share it among all the classes that need it.

Damage Sources

In the `Sources - Damage` subfolder you will find two damage sources: `Skill` and `Environmental`. The scene does not make use of environmental damage, but the damage source is used in Experience Collection, particularly in the `Environmental Kill Exp Strategy` strategy. Therefore, although it cannot be tested in the scene, it has value as an example to show you how to create and configure a multiple strategy that uses both the direct kill and the environmental kill strategies.

Damage Types

In the `Damage Types` subfolder you will find the three damage types used in the sample scene: Physical, Magical and True, and two folders containing the three variants of Damage Mitigation Functions and Defense Penetration Functions. Currently, both physical and magical damage use the logarithmic formula for damage mitigation based on the defensive stat, and percentage reduction for defensive stat penetration. I have provided all three to make it easier for you to test should you want to swap the formulas on the fly.

Events

In `Events` you will find all the instances of the game events used by the sample scene. All the instances belong to Astra Health except for `Entity Leveled Up Game Event` and `Entity Leveled Down Game Event`. These events have been connected as global events in the `AstraHealthConfigSO` configuration.

All communications between the various objects in the sample scene are handled through these events.

Experience Collection

In the `Experience Collection` folder there are all the instances and strategies used for experience collection in the sample scene. As mentioned earlier, the sample scene does not involve the use of environmental damage, but I have nonetheless included an `Environmental Kill Exp Strategy` strategy to show you how to create and configure a multiple strategy that uses both the direct kill and the environmental kill strategies.

Heal Sources

Unlike the damage sources, in the sample scene all 4 `HealSourceSO` instances found in the `Sources - Heal` folder are used:

- `HP Regeneration`: used by passive health regeneration events
- `Lifesteal`: used by lifesteal effects
- `Resurrection`: used by the heal applied upon an entity's resurrection
- `Skill`: used by abilities that have healing effects

Game Actions

In `Game Actions` you will find `On Death Game Actions` and `On Resurrection Game Actions`. These are used to define the behavior of entities in response to their death and resurrection respectively. We will cover

this topic in more detail later on this page.

Skills

Finally, in the `Skills` folder you will find all the instances of the abilities used in the sample scene, divided by character. Each ability has its own `SkillSO` instance, and a `ScalingFormula` with one or more associated `ScalingComponent`s. Some abilities, as mentioned before, can have more than one effect. In that case, they have a scaling formula for each effect.

Passives

In the `Passives` folder you will find the object instances needed for the passive abilities implemented for the sample scene to work. We will look at passive abilities later in Implementing Custom Passive Abilities.

Interacting with the Scene

Now that we have explored the sample scene, the hierarchy and the project files, let me give you some information about some specific configurations that deserve a bit more explanation.

Casting Skills to Deal Damage and Heal

The skills of the sample scene can be summarized, from a technical standpoint, as builders of `PreDamageContext` and `PreHealContext` that route these contexts toward the target entities. The target will process these contexts in its `heal` and `TakeDamage` methods. The damage calculation that takes place inside `TakeDamage`, as we have seen in the workflows, passes through the damage calculation pipeline. The pipeline uses the strategy that has been configured at the `AstraHealthConfigSO` configuration level. When the entity is healed or takes damage, it will invoke the respective Global Events also configured in the package configuration. And the heal and damage pop-up managers found in the hierarchy are listening to these two events respectively. When they receive the event, they create a pop-up above the target character's head showing the amount of damage taken or healing received. The damage pop-ups will have different colors and icons based on the type of damage dealt. The icons (and color) will match those you see to the left of the ability you cast.

Casting Skills and Damage and Heal Pop-Ups

If you hover over an ability button, you will see a tooltip appear showing a description of the ability. If you hold the `Alt` key while the tooltip is visible, you will see the scaling details of the various ability effects appear. These descriptions should help you predict the order of magnitude of the damage and heals that the various ability effects can deal, and understand how the characters' level up influences the power of their abilities.

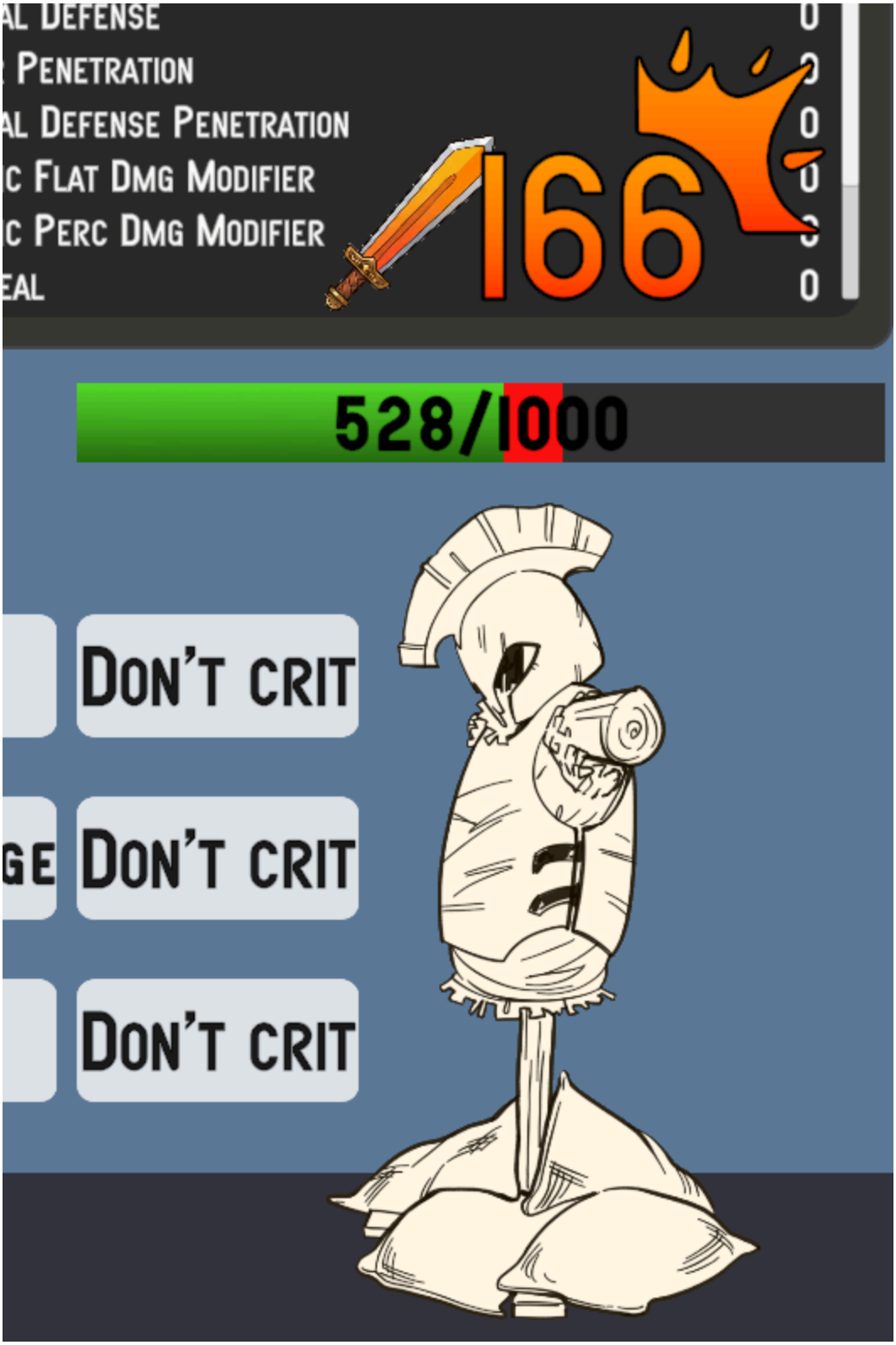
Collapsed tooltip

Expanded tooltip with details

You can also toggle whether to show or not the description by clicking `Ctrl`.

If you press the "Don't crit" button, the text will change to "Do crit", and the ability you cast from that point on will deal a critical hit. The pop-up of a critical hit has a custom icon in the top right, and looks

like this:



Messing Around with Damage Types and Damage Calculation Strategy

Here you can go wild and play with various settings to radically change the way damage is calculated. Let's start by observing the configuration of the damage types:

Damage Type	Defensive Stat	Damage Mitigation Function	Defense Penetration Stat	Defense Penetration Function	Flat Damage Modifier Stat	Percentage Damage Modifier Stat	Ignores Barrier
Physical	Armor	Logarithmic DR	Armor Penetration	Percentage DP	Physical Flat Dmg Mod	Physical Percentage Dmg Mod	✗
Magical	Magic Resist	Logarithmic DR	Magic Penetration	Percentage DP	Magical Flat Dmg Mod	Magical Percentage Dmg Mod	✗
True	None	None	None	None	None	None	✓

You can notice that physical and magical damage both use the same logarithmic formula for damage mitigation and the same one for defense penetration, but clearly use different statistics for reduction and penetration. True damage instead has no associated defensive stat, so it does not suffer any reduction. Having no associated defensive stats, it also has no defense penetration. In addition to the defensive stats and the related reduction and penetration formulas, physical and magical damage also define statistics for flat and percentage modifiers for the specific damage type. I refer you to the documentation on Damage Type's Damage Modifiers for more details on how these stats work and how they are used in the damage calculation pipeline. You can test the impact of these stats by modifying their values in the Dummy's `EntityStats`. **Also remember that the order of application of flat and percentage modifiers is defined in the configured damage calculation strategy. In the sample scene you will find it under `Examples/Instances/Default Damage Calculation Strategy`.** Its initial configuration is:

1. Apply Critical Multiplier
2. Apply Defenses
3. Apply Barrier
4. Apply Percentage Damage Modifiers
5. Apply Flat Damage Modifiers

You can of course reorder these steps as you prefer to see how the final damage result changes based on the order of application of the steps.

To conclude the observations on damage types, you can notice that true damage ignores both flat and percentage modifiers, in addition to the barrier. This means that the damage, besides not being reduced, cannot be amplified either. **The only mechanic that modifies the value of true damage is the critical multiplier.**

There is one last thing worth mentioning on this topic, and it is the configuration of the stats for flat and percentage damage modifiers (this applies to both damage-type specific and generic ones). The stats for percentage damage modifiers are configured differently from flat ones. If you open the `Physical Dmg Perc Mod` stat in the inspector, you will notice that it has a minimum value of -100, while the `Physical Dmg Flat Mod` stat does **not** have a minimum value. This is because a damage modifier percentage can never exceed -100% (complete damage negation), while a flat modifier can theoretically reduce damage by any value. It is the responsibility of the flat damage modifiers step in the damage calculation pipeline to ensure that the final damage does not drop below zero due to a flat damage modifier that exceeds the damage value. An observation you might raise is "What if an entity has the generic damage percentage modifier at `-70`% and the damage-type specific percentage modifier at `-50`%? In that case, the damage will suffer a total reduction of 120%, right?". Correct! However, as explained in ApplyPercentageDmgModifiersStep, the construct used internally by the damage calculation pipeline lower-bounds the damage amount transformations at 0. Therefore, in this case, the final damage will be 0, and the damage will be prevented with `DamagePreventionReason.PipelineReducedToZero`. Therefore, even if you did not set a lower bound of -100% on the percentage damage modifier stats, the system would still ensure that the damage does not drop below zero due to percentage modifiers; however, it is more semantically correct and clearer to set a -100% limit on these stats. This would also be useful when creating game interfaces for displaying these stats, as a player might be confused seeing a damage modifier of -150%, not knowing that in reality the damage cannot drop below zero.

Playing with Passive HP Regeneration

In the sample scene, passive health regeneration is enabled for all characters through the dedicated `EntityPassiveHpRegeneration` component. You can disable it for a specific entity by removing that component or disabling it in the inspector. By default, healing popups will also be shown for each passive regeneration tick. If these popups bother you, you can disable them as follows:

1. From the hierarchy, open the `PopupCanvas` game object
2. Select the `HealPopupManager` game object
3. In the inspector, in the `Heal Popup Manager` component, expand the `Heal Sources To Ignore` section and press the `+` button. Drag the `HP Regeneration HS` Heal Source found in `Examples/Instances/Sources - Heal` here.

This way, the `Heal Popup Manager` will ignore all healing events that have `HP Regeneration` as their heal source, and therefore will not create pop-ups for passive regeneration ticks.

Even from this simple example, the value of creating and using different heal sources becomes clear. Heals coming from different mechanics can raise different use cases.

I have configured the regeneration tick rate to 1 second. If you wanted to change the tick rate of passive regeneration, you can do so by modifying the `Passive Health Regeneration Interval` value in the `Astra Health Config` configuration instance found in `Examples/Resources`. I also refer you to the documentation

on Package Configuration | Health Regeneration and on Healing | Passive Health Regeneration for details.

To modify the passively regenerated health, you need to change the value of the `Passive Regeneration` stat. For the three playable characters, this stat is controlled through the Growth Formulas associated with their respective classes, while for the Dummy you can modify it directly through `EntityStats`.

> **WARNING**
> Remember that, as mentioned in Passive Health Regeneration Stat (HP/10s) in Package Configuration, the value of this stat represents the amount of health regenerated every 10 seconds. Therefore, if you want a character to passively regenerate 5 HP per second, you will need to set the value of `Passive Regeneration` to 50, not 5.

Health Scaling Component

Some abilities use a `HealthScalingComponent` to scale their damage based on the health of themselves or the target.

- The duelist has the `Syphoning Strike` ability, which heals him based on maximum health.
- The Assassin has the `Slice And Dice` ability, where the second effect deals pure damage based on the target's current health. Additionally, `Throat Cut` deals physical damage based on the target's maximum health.
- The Sorcerer has the `Soul Explosion` ability that deals magical damage based on the target's missing health.
- All of the Dummy's abilities scale based on health.

You can explore the details of the abilities through the popup that appears on hover over the respective buttons, and you can inspect the scaling formulas and scaling components associated with each ability to see how the health-based scaling has been configured.

Experience Collection and Death & Resurrection

In the sample scene you have the opportunity to see both the base package's `ExpSource` and the brand new `ExpCollector` of Astra Health in action. I refer you to the documentation on Experience Collection for details on `ExpCollector`.

All three playable characters, who have classes and therefore a level progression, have an associated `ExpCollector`. The Dummy instead has an associated `ExpSource`, and acts as a source of experience for the playable characters. As explained in the Experience Collection documentation, `ExpCollector` relies on an `ExpCollectionStrategySO` to define the experience collection rules. In the sample scene, the playable characters use a multiple strategy that involves both collecting experience through the `Direct Kill Exp`

`Strategy` and through the `Environmental Kill Exp Strategy`, all thanks to the `FirstMatchExpStrategySO` which considers the first strategy that matches the experience collection conditions. I refer you to the configuration documentation Package Configuration | Default Exp Collection Strategy for configuring the package's default experience collection strategy. However, I remind you that the sample scene does not involve the use of environmental damage, so the `Environmental Kill Exp Strategy` will never be activated. I have nonetheless included this strategy in the sample scene to show you how to create and configure a multiple strategy that uses both the direct kill and the environmental kill strategies.

By default the `DirectKillExpStrategySO` marks the Dummy's `ExpSource` as harvested once a playable character kills the Dummy, and therefore does not allow experience to be collected from that `ExpSource` more than once. However, at the package configuration level a multiple Default On Resurrection Game Action has been set which, among other things, resets the `ExpSource` of the resurrected entity. Therefore, since the `Default On Death Game Action` resurrects entities after 3 seconds, and the `Default On Resurrection Game Action` resets the `ExpSource`, you will be able to collect experience from the Dummy every time you kill it, even if you are using the `Direct Kill Exp Strategy`.

Let's spend a few more words on the Game Actions configured for death and resurrection. The `Default On Death Game Action` is a `CompositeComponentGameAction` that contains 2 Game Actions within it:

1. `ToggleActiveGameObjectGameAction`: allows you to decide whether to activate or deactivate the game object related to the `Component` passed as context (in this case the entity that died). In this case, it is configured to deactivate the game object.
2. `DelayedGameAction`: allows you to delay the execution of a Game Action by a certain amount of time. In this case, it is configured to delay by 3 seconds the execution of a `ResurrectGameAction`, which resurrects the dead entity. The `Default On Resurrection Game Action` is also a `CompositeComponentGameAction` that contains 2 Game Actions within it:
3. `ToggleActiveGameObjectGameAction`: used in contrast to the one in the `Default On Death Game Action` that deactivates the game object upon death.
4. `ToggleHarvestedExpSourceGameAction`: allows you to decide whether to mark as harvested or unharvested the `ExpSource` passed as context (in this case, the `ExpSource` of the resurrected entity). In this case, it is configured to mark the `ExpSource` as unharvested.

Implementing Custom Passive Abilities

Although this package does not deal with defining high-level constructs for abilities and passives (and it will be the responsibility of a future framework extension to do so), it is still possible to implement certain passive abilities for your characters in a simple and fast way using the constructs of this package and the base one. Below are some examples.

Passive Ability (Duelist) - Excellent Recovery

Implementation difficulty: easy

The duelist has a passive ability that increases passive health regeneration by 400% at level 10, and by 1000% at level 20.

The implementation of this ability is very simple: I created a specific GrowthFormula for the Duelist for the `Passive Reg Heal Perc Mod` stat, which is the stat that represents the percentage modifier to passive health regeneration that has been assigned to the `HP Regeneration` Heal Source. The growth formula returns a value of 0% up to level 9, a value of 400% from level 10 to level 19, and a value of 1000% from level 20 onwards.

Passive Ability (Assassin) - Vengence In Death

Implementation difficulty: medium

The assassin has a passive ability that causes him, when receiving a fatal blow, to counter-attack the enemy dealing 80% of the lethal damage received as physical damage. This damage is a guaranteed critical hit.

The implementation of this ability is slightly more complex than the previous one. To obtain the described effect we can resort to the `CounterDamageOnDeathGameActionSO` provided by the package. This Game Action takes as input a parameter of type `entityDiedContext`, the same context that is passed by death events. Therefore, through an `EntityDiedGameEventListener` we can intercept the death of an entity and trigger this Game Action passing the intercepted death context. The Game Action uses the `DamageResolutionContext` contained in the death context to trace back to the amount of damage that caused the entity's death, and deals damage equal to 80% of this amount to the perpetrator of the lethal damage (multiplied by the Assassin's critical multiplier). However there is a problem: we don't want to trigger the Game Action at the death of any entity, but only at the death of the Assassin. This is where Event Channels and Extra Events come into play. We can create a dedicated `EntityDiedGameEvent` to communicate only the Assassin's death, and assign it as an extra death event on the Assassin's death Event Channel in `EntityHealth`. When the Assassin dies, the death Event Channel raises both the global `Entity Died Game Event` configured in the package and this dedicated extra event. Therefore, instead of listening to the global `Entity Died Game Event` with our `EntityDiedGameEventListener`, we will listen to this specific extra event for the Assassin. This way, our Game Action will only be triggered when the Assassin dies, and not at the death of other entities.

Passive Ability (Sorcerer) - Glass Cannon

Implementation difficulty: medium

The sorcerer has a passive ability that increases the critical hit multiplier by 75%, but every time he takes damage he also takes 15% of his Magic Power as extra magical damage.

The implementation of this ability is somewhat a union of the previous two. To increase the critical hit multiplier, simply create a specific Growth Formula for the Sorcerer for the `Critical Multiplier` stat, which returns a value of 275% at all levels. As for the second effect, namely taking extra magical damage

equal to 15% of Magic Power every time he takes damage, we can again resort to the `CounterDamageOnDamageGameActionSO` to deal extra magical damage every time the Sorcerer takes damage. However, unlike the Assassin, we want this game action to target the Sorcerer rather than whoever dealt the damage to us. For the damage amount, we use a dedicated `ScalingFormula`. Finally, also in this case, we don't want this game action to be triggered every time any entity takes damage, but only when the Sorcerer takes damage. Also in this case, we can resort to the damage Event Channel and its extra events. I created a dedicated `DamageResolutionGameEvent` to communicate when the Sorcerer takes damage, and assigned it as an extra damage-taken event on the Sorcerer's damage Event Channel in `EntityHealth`.

Passive Ability (Duelist) - Second Chance

Implementation difficulty: advanced

When the duelist reaches level 11, he unlocks a passive ability that grants him a one-time lethal damage prevention. When the duelist receives a lethal blow, he is resurrected with 33% of his maximum health. However, this ability can only trigger once.

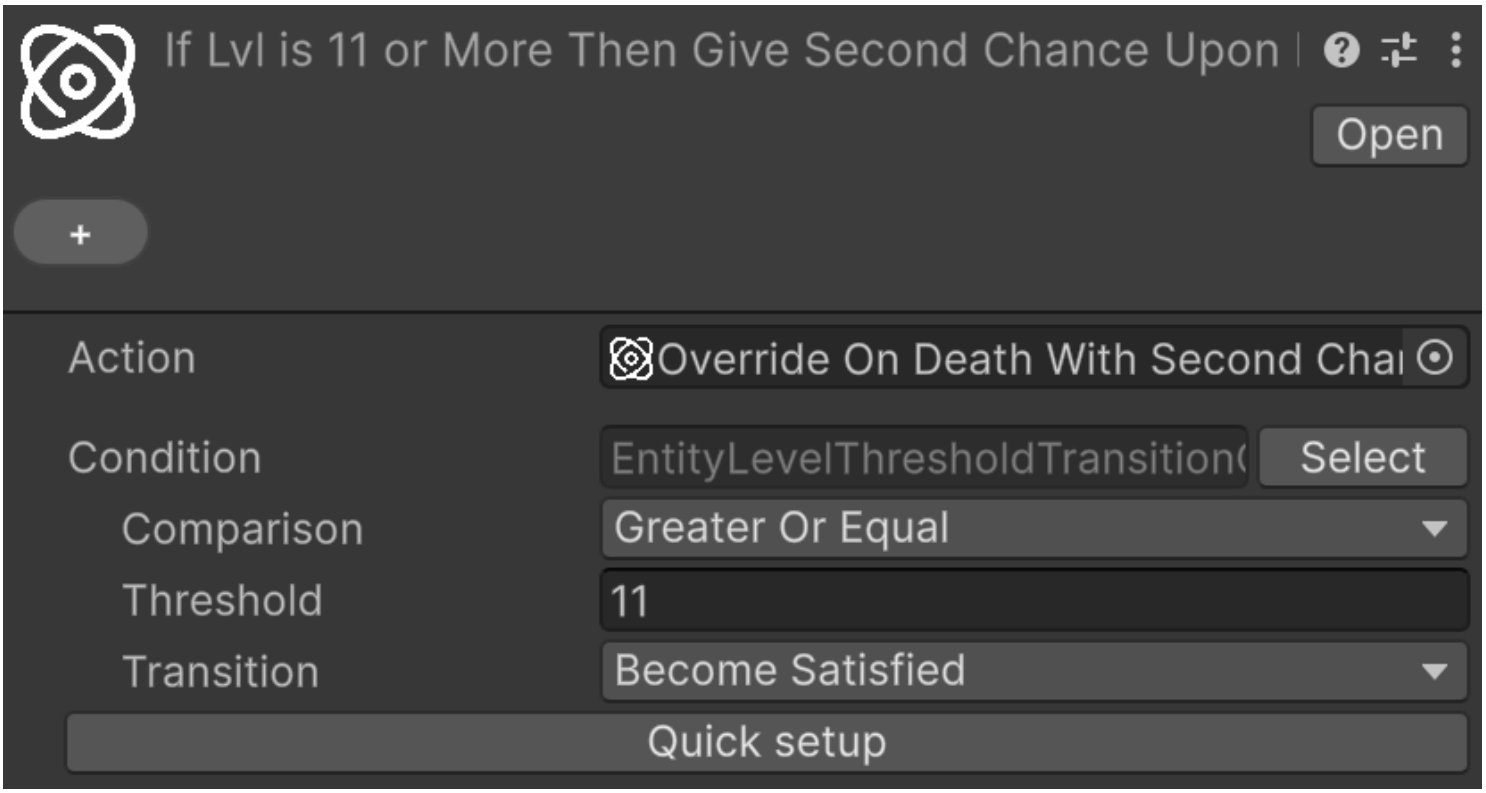
The implementation of this ability is the most complex. The core idea is to rely on the `SetEntityOverrideOnDeathIHasEntityGameActionSO` to override the default On Death Game Action for the duelist. Let's analyze step by step how to implement this:

First of all we need a dedicated Entity Level Up extra event to assign to Duelist's `EntityCore` component, under his extra events. This will ensure that only Duelist's level up will be communicated through this event. Then, we can add a `EntityLevelUpGameEventListener` component to a dedicated child object of the Duelist, and wire the previously created extra event to it. We'll come back later on this listener later when we'll have the Game Action to assign to it ready.

The game action we want to implement shall satisfy the following requirements:

1. When the duelist reaches level 11, the on-death game action override is applied to him. Any level reached beyond 11 should not trigger any change, even if the second chance was already consumed.
2. When the duelist receives a lethal blow, if the second chance is not yet consumed, the on-death behavior should immediately resurrect the duelist with 33% of his maximum health.
3. Upon resurrection, the on-death game action override should be removed from the duelist. Any further lethal blow should cause the duelist's death, without triggering the second chance effect.

The point (1) can be easily achieved by creating first a Conditional Game Action, and by configuring it as follows:



The assigned `Action` is a `SetEntityOverrideOnDeathIHasEntityGameActionSO` that sets the override on death for the duelist to the `Resurrect And Remove Second Chance (AH Samples)` Game Action (we will see soon how it is implemented and what it does). Notice that the used condition is `EntityLevelThresholdTransitionCondition`, which checks whether the entity has just reached a certain level threshold. This ensures that any transition beyond level 11 will not trigger this game action.

The point (2) and (3) are satisfied by the `Resurrect And Remove Second Chance (AH Samples)` Game Action assigned as override on the Duelist's `EntityHealth` component. This Game Action is a `CompositeComponentGameAction` that contains the following Game Actions:

1. `Resurrect Game Action (AH Samples)` (point 2)
2. `Remove Second Chance Upon Death (AH Samples)` (point 3)

The first is just a resurrection game action configured to resurrect with 33% of the Max HP. The second one is, once again, a `SetEntityOverrideOnDeathIHasEntityGameActionSO`, but this time configured to remove the override on death from the duelist, by setting it to `None`.

With this setup, the flow of the second chance ability is as follows:

1. When the duelist reaches level 11, the conditional game action is triggered, and the override on death is applied to the duelist.
2. When the duelist receives a lethal blow, the on-death override triggers, and the `Resurrect And Remove Second Chance (AH Samples)` Game Action is executed.
3. The duelist is immediately resurrected with 33% of his maximum health thanks to the `Resurrect Game Action (AH Samples)`, and the override on death is removed thanks to the `Remove Second`

`Chance Upon Death (AH Samples)`.

4. Any further lethal blow will cause the duelist's death, without triggering the second chance effect, as the override on death has been removed.
5. Any level up beyond 11 will not trigger any change, as the conditional game action will not be triggered thanks to the `EntityLevelThresholdTransitionCondition` condition used.

Installation instructions

Importing Astra Health and its samples

1. From the package manager, import Astra Health. You can find the package in the "My Assets" section.
2. After importing, head to the "In Project" section, always in the Package Manager, and click on "AstraHealth".
3. Click on the "Samples" tab and import the samples you desire. Find out more about the samples in the Samples documentation.
4. If you imported the "Example scene and instances" samples you need to import also TextMeshPro Essentials. Click on "Window > TextMeshPro > Import TMP Essential Resources".

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog.

[1.0.0] - Unreleased

Added

- Initial release of Astra Health.

Migration Guide

Any breaking changes in Astra Health will be documented here, along with instructions on how to update your project to be compatible with the latest version of the package.